

FEC/ARQ Unified Correction Symbol Model

Pierre Kisters

July 2026

Contents

Abstract	5
1. System Model	6
1.1 Notation	6
1.2 Glossary	7
1.3 Components	8
1.4 The Bandwidth / Latency / Reliability Triangle	8
2. Channel Model	10
2.1 Gilbert-Elliott Two-State HMM	10
2.2 Stationary Properties	10
2.3 Burst Length Distribution	11
2.4 Concrete Examples	11
3. Recovery Fundamentals	11
3.1 The Problem: Recovering Lost Symbols	11
3.2 FEC: Proactive Recovery	12
3.3 ARQ: Reactive Recovery	13
3.4 Recovery Latency and the $P_{\text{lost}}(t)$ Model	13
4. The Taper Function	15
4.1 Definition	16
4.2 Why Match the Loss Distribution?	16
4.3 Total Correction Rate	17
4.4 The Taper Never Reaches Zero	17
4.5 Real-Time Adaptation	17
5. Unified Correction Symbol Model	18
5.1 Why Unify FEC and ARQ?	18
5.2 Correction Symbols — The Unified Concept	18
5.3 Three-Stream View: Source, Repair, Retransmit	19
5.4 Per-Slot Decision	20
6. Protocol Mechanics	21
6.1 The Retransmit Buffer	21
6.2 SACK-Extended ACK	22
6.3 Per-Symbol Delivery Outcomes	23
6.4 The Triangle in Action	24
6.5 Four-Mechanism Composition	24
7. Estimation — From Observations to Channel Parameters	25
7.1 What We Observe	25
7.2 EWMA — Fast Point Estimate	25
7.3 Beta-Binomial Posterior — Uncertainty Quantification	25
7.4 BOCD — Regime-Aware Prediction	26
7.5 GE Parameter Estimation	26
7.6 ACK Loss and Self-Healing	27
7.7 Estimation Error and Overhead	27
8. The Optimization Problem	27
8.1 Formal Statement	27
8.2 Corrected Model (canonical)	28

8.3 Burst Variance Correction	29
8.4 The Corrected Optimal Correction Rate	30
8.5 Worked Examples	30
8.6 Three-Variable Optimization	31
8.7 Exact P_{fec} via Transfer-Matrix DP	34
9. Codec Overhead Integration	35
9.1 Decoder Invocation Probability	35
9.2 Effective Codec Overhead	35
9.3 Impact on METTLE at DC	36
10. Multi-Protocol Extension	36
10.1 Per-Symbol Latency Classes	36
10.2 Interleave Before vs After FEC	36
10.3 When Shared Wins	37
10.4 Extending the Formula	37
11. Verification	37
11.1 Simulation Approach	37
11.2 Analytical Predictions to Verify	37
11.3 Boundary Cases	38
11.4 Connection to Existing Benchmarks	38
12. Congestion Control Integration	38
12.1 Why Delay-Based CC is Required	38
12.2 QUIC Datagrams Bypass Quinn's CC	39
12.3 Copa vs BBR	39
12.4 Copa's Rate Formula	40
12.5 CC + Taper: The Complete Architecture	41
12.6 ECN as Opportunistic Enhancement	42
12.7 Application Back-Pressure	42
13. Multi-Path Scheduling	43
13.1 Why FEC Beats MPTCP	43
13.2 Per-Path Model	43
13.3 Unified Symbol Stream and Interleaving	43
13.4 Correction Deficit	44
13.5 Effective Delivery Time and Bandwidth	44
13.6 Scheduler Ratio Adjustment	44
13.7 Burst Protection During Ratio Adjustment	45
13.8 Interpolated Objective Function	45
13.9 QoS Priority Cascade	45
13.10 Cross-Path Retransmit	46
14. Future Directions and Considered Improvements	46
14.1 Reconsidering the Triangle: Bandwidth as Constraint, Not Variable	46
14.2 FEC Latency Is Not Zero	47
14.3 Unified FEC Latency Distribution	47
14.4 Ambient FEC and the Pipeline Effect	48
14.5 Optimal Encoder Window Size	48
14.6 ARQ Latency and the Retransmit Sweet Spot	49
14.7 When FEC Beats ARQ (and Vice Versa)	49

14.8 Per-Symbol Recovery Probability Function	49
14.9 Reconceived Delivery Time Distribution	50
14.10 Latency Tail vs Throughput: Not Always a Trade-off	50
14.11 Application Profiles Revisited	50
14.12 ARQ After FEC Decode: Not Redundant for the Decoder	51
14.13 Proactive Retransmit vs FEC: FEC is Strictly Better	51
14.14 Marginalizing Over Burst Length	52
14.15 In-Burst FEC Survival	52
14.16 The FEC/ARQ Race	52
14.17 Decode-Induced Jitter	53
14.18 Estimator-Rate Feedback Stability	53
14.19 Consistency of P_{fec} Models	53
14.20 The δ Definition Question	54
14.21 Sub-Capacity Operation (Emergent Behavior)	54
14.22 Sequence-Aware P_{lost}	56
14.23 Post-Burst FEC Boost (Reactive Deficit Recovery)	57
14.24 Jitter-Horizon Encoder Lag	57
14.25 Completion-Tail FEC	58
14.26 Completion-Exposure δ (the Bulk glide)	59
14.27 Block-Mode ARQ via Batch Acknowledgements	60
Appendix A: Summary of Key Formulas	62
Appendix B: Related Work	63
ACK-Based vs NACK-Based Recovery	63
Hybrid FEC-ARQ for Lossless Streaming	63
Streaming Codes over Gilbert-Elliott Channels	64
Tail Latency Optimization with Proactive FEC	64
Joint FEC/ARQ under Gilbert-Elliott	64
Water-Filling and Optimal Resource Allocation	64
What This Model Contributes	65
Appendix C: Model Extensions from Related Work	65
C.1 Correction Symbol Preemption of Source Data [Mehrotra2010] (resolved)	65
C.2 Information Debt for Exact P_{fec} [RLC_GE2025]	65
C.3 Analytical P_{fec} Bounds [Vajha2020]	66
C.4 Multipath Diversity Gain [Zeng2021]	66
C.5 DCSW Worst-Case Taper Floor	67
C.6 Burst-Aware Channel Discount (considered refinement)	68
C.7 Summary of Extensions	68
Appendix D: Open Questions	68
Remaining Open Points (full audit)	69
Appendix E: Preliminary Poisson Model (superseded)	71
E.1 FEC Recovery Probability (preliminary – superseded by Section 8.2)	71
E.2 Solving for A^* (preliminary)	71
E.3 The Optimal Correction Rate Formula (preliminary – see Section 8.4)	72
E.4 Comparison with Information-Theoretic Minimum	72
References	73
Channel Models	73

Estimation and Detection	73
FEC Codes	73
Streaming Codes Theory	74
Multipath and Scheduling	74
Hybrid FEC-ARQ	74
Streaming Code Analysis over GE	74
Tail Latency	75
Congestion Control	75
Information Theory	75
TCP SACK	75
ECN and Multicast Feedback	75
Sliding Window Channel Models	75

Abstract

Data traversing lossy multipath channels (WiFi, LTE, satellite) has specific requirements: some needs low latency (VoIP), some needs high throughput (file transfer), some tolerates partial loss (sensor telemetry). Each available path has different characteristics — bandwidth, latency, and loss rate. The goal of this model is to **optimally use each path or combination of paths to satisfy the constraints of the data sent over them**, without ad-hoc tuning knobs.

Three properties — bandwidth, tail latency, and reliability — form a triangle: fix any two, the third is determined by the channel. The protocol hint (Realtime, Bulk, etc.) selects which two to fix. The model computes the optimal correction rate, taper schedule, and path allocation from measured channel parameters — no magic offsets or manual thresholds.

The core mechanism is the **correction symbol** — a unified concept that subsumes both FEC repair symbols (proactive, flexible) and ARQ source retransmits (reactive, immediately decodable). A single taper function, shaped by the Gilbert-Elliott channel model's burst survival function, controls correction density over time. A probabilistic per-slot decision based on loss confidence $P_{\text{lost}}(t)$ — driven by time since send — determines whether each correction is a retransmit or a repair.

The optimal correction rate $r^* = e/(1-e) + z_{\text{delta}} \times \sqrt{e \times s2_{\text{burst}} / (W(1-e))}$ combines the information-theoretic minimum with a burst-variance- corrected tail margin. Copa congestion control is recommended over BBR for its taper-compatible rate oscillation (no FEC protection gaps from ProbeRTT).

For multipath, each path runs its own Copa, taper, and GE estimator with a unified symbol stream preserving interleaved burst protection. A global correction deficit tracks outstanding recovery needs across paths. The scheduler adjusts per-path source/correction ratios using an interpolated latency/bandwidth objective, with QoS priority cascading for mixed traffic.

ACK-based feedback with SACK replaces the NACK mechanism — the same proven approach TCP has used for 40 years, with sender-side loss inference from ACK absence.

1. System Model

1.1 Notation

Symbol	Meaning	Unit	Example
ϵ	Average channel loss rate	probability (0-1)	0.025 (2.5% WiFi)
p	$P(\text{Good} \rightarrow \text{Bad})$ in GE model	probability (0-1)	0.013
q	$P(\text{Bad} \rightarrow \text{Good})$ in GE model	probability (0-1)	0.5
B	Mean burst length = $1/q$	symbols (count)	2.0
W	Encoder window size	symbols (count)	50
RTT	Round-trip time	seconds	0.050 (50ms)
$P_{\text{lost}}(t)$	$P(\text{symbol lost given no ACK after time } t)$	probability (0-1)	$\approx 2\epsilon$ at $t=\text{SRTT}$
t_{fec}	FEC recovery time = $m \times (1+r) / (r \times (1-\epsilon)) \times t_{\text{sym}}$	seconds	0.0013 (1.3ms)
t_{sym}	Symbol transmission time = $\text{symbol_size} / \text{throughput}$	seconds	0.000096 (96 μ s at 100Mbps)
SRTT	Smoothed RTT estimate	seconds	0.050 (50ms)
RTTVAR	RTT variance estimate (standard deviation)	seconds	0.005 (5ms)
r	Total correction rate	ratio (corrections/source)	0.08 (8%)
$\tau(t)$	Taper function: correction density at offset t	ratio (corrections/symbol)	0.04
A	Taper amplitude (scaling factor)	ratio (corrections/symbol)	0.04
P_{fec}	Probability a lost symbol is FEC-recovered	probability (0-1)	0.95
δ	Tail latency target: $P(\text{late delivery}) \leq \delta$	probability (0-1)	$1e-4$
ρ	Reliability target: $P(\text{symbol delivered}) \geq \rho$	probability (0-1)	1.0 (100%)
T_{cut}	Taper cutoff time (stop corrections after this)	seconds	∞ (100% reliability)
B_{max}	99.99th percentile burst length = $\text{ceil}(\ln(0.0001) / \ln(1-q))$	symbols (count)	14 (WiFi, $q=0.5$)
buffer_max	Max retransmit buffer size (derived)	symbols (count)	700 (WiFi 100Mbps)
L_{prop}	Propagation delay (base latency)	seconds	0.025 (25ms)
L_{arq}	ARQ recovery latency (time from loss to retransmit arrival)	seconds	0.075-0.100
σ^2_{burst}	Burst variance inflation factor	dimensionless	2.9

Symbol	Meaning	Unit	Example
z_{δ}	Standard normal quantile for δ	dimensionless	3.72 (for $\delta=1e-4$)
ϵ_{burst}	Current channel loss rate (fast EWMA or GE state, optional — see C.6)	probability (0-1)	0.5 (during burst)
ϵ_{codec}	Codec decode overhead	ratio (0-1)	0.01 (RaptorQ)

1.2 Glossary

Correction symbol — any symbol sent to recover lost data. Either a source retransmit (exact copy, immediate decode) or a repair symbol (FEC linear combination, needs decoder). The taper function controls how many; $P_{\text{lost}}(t)$ controls which type. See Section 5.2.

Repair symbol — a correction symbol generated by the FEC encoder. It is a random linear combination of source symbols in the encoder window over GF(256). The receiver feeds it to the decoder. See Section 3.2.

Source retransmit — a correction symbol that is an exact copy of a previously-sent source symbol from the retransmit buffer. The receiver can use it immediately without decoding. See Section 3.3.

Encoder window — the W most recent source symbols tracked by the FEC encoder. Repair symbols are linear combinations of symbols in this window. When a symbol is evicted (window slides forward), it can no longer be covered by new repair symbols. See Section 3.2.

Taper function $\tau(t)$ — specifies the correction symbol density at time offset t from a source symbol. Shaped to match the channel's burst-length distribution: $\tau(t) = A \times (1-q)^t$ for Gilbert-Elliott channels. See Section 4.

Systematic code — an FEC scheme where source symbols are sent as-is (not encoded). The decoder is only invoked when at least one source symbol is lost. If nothing is lost, the data passes through without any decoding cost.

Codec overhead — the extra symbols the decoder needs beyond the number of losses, because not all random linear combinations are linearly independent. E.g., RaptorQ needs ~1% extra, METTLE needs ~15%. See Section 9.

SACK (Selective Acknowledgement) — an extension to cumulative ACK that reports out-of-order received blocks beyond the cumulative point. Lets the sender know exactly which symbols arrived despite gaps. See Section 6.1.

GE states (Good/Bad) — the two states of the Gilbert-Elliott channel model. In the simplified model: Good = no loss, Bad = total loss. Transition probabilities p (Good->Bad) and q (Bad->Good) determine burst behavior. See Section 2.

$P_{\text{lost}}(t)$ — probability that a specific symbol was lost, given no ACK received after time t . The per-slot mixing probability for repair vs retransmit. See Section 3.4.

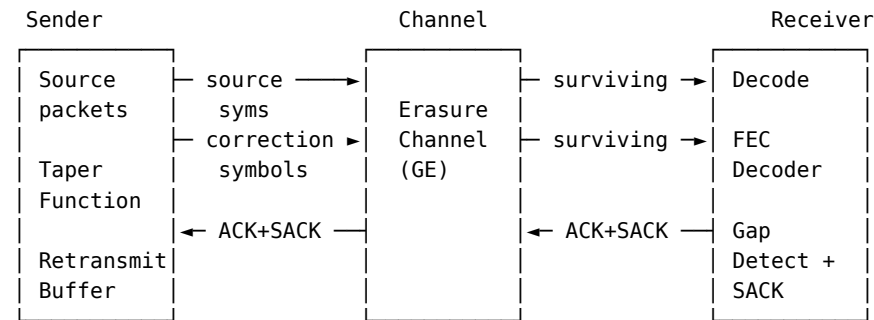
P_{fec} — probability that a lost symbol is recovered by FEC (proactive correction) before ARQ retransmit kicks in. See Section 8.2.

P_arq — probability that ARQ retransmit succeeds, given FEC failed. Derived from reliability target: $P_{arq} = 1 - (1 - \rho) / (\epsilon \times (1 - P_{fec}))$. See Section 6.3.

z_delta — standard normal quantile at probability $(1 - \delta)$. Controls the tail margin in the repair rate formula. See Section 8.4.

σ^2 _burst (s2_burst) — burst variance inflation factor for the GE channel: $1 + 2(1 - p - q) / (p + q)$. Corrects the iid normal approximation for burst-correlated losses. See Section 8.3.

1.3 Components



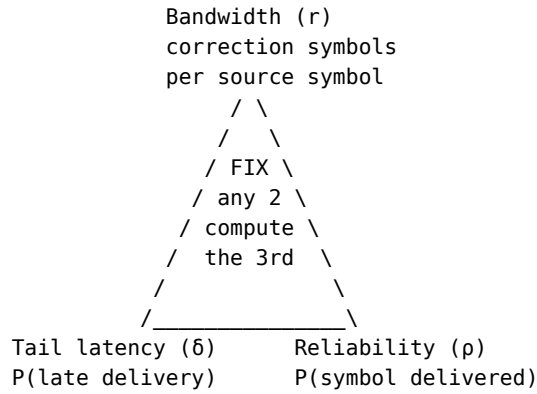
The system provides **reliable** delivery. Three properties form a triangle — bandwidth, tail latency, and reliability. Fix any two, the third is determined by the channel. The protocol hint selects which two to fix.

One unified mechanism — **correction symbols** — handles both proactive and reactive recovery. The taper function controls correction symbol density, and each correction slot either retransmits an exact source symbol from the retransmit buffer (preferred, if old enough) or generates a random repair symbol (FEC fallback):

Aspect	Correction Symbol (retransmit)	Correction Symbol (FEC repair)
When chosen	With probability $P_{lost}(t)$	With probability $1 - P_{lost}(t)$
Content	Exact copy of source symbol	Random linear combination
Decode cost	Zero (immediate use)	Needs FEC decoder
Bandwidth cost	Same (one symbol slot)	Same (one symbol slot)
Latency cost	Depends on when P_{lost} triggers	Zero additional (arrives with source)

1.4 The Bandwidth / Latency / Reliability Triangle

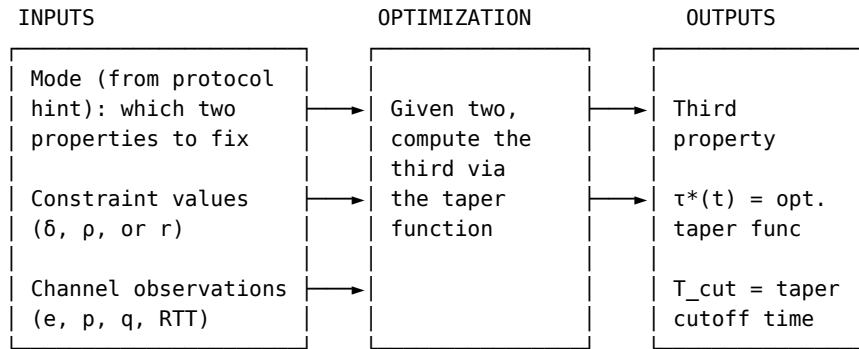
Three properties are linked by the channel. Fix any two, the third is determined:



Mode	Fix	Compute	Use case
Bulk transfer	$\rho=100\%$, minimize r	δ (tail latency)	File transfer, backup
VoIP	δ (max latency), r (codec rate)	ρ (reliability)	Interactive voice
Live video	δ (frame deadline), ρ ($\geq 99.9\%$)	r (bandwidth)	Streaming, conferencing
Gaming	δ (tight), ρ ($\geq 99\%$)	r (bandwidth)	Real-time game state
Sensor/IoT	r (minimal), ρ ($\geq 95\%$)	δ (tail latency)	Periodic telemetry

For Bulk transfer, “minimize r ” is taken literally: the tail target is “late is fine” ($\delta_{\text{bulk}} = \hat{\epsilon} + (0.05 - \hat{\epsilon})\chi$, Sections 5.3 and 14.26), so $r = 0$ identically in the steady state ($\chi = 0$) and the residual is pure ARQ — except over the final ~ 1.5 SRTT of a known-length transfer, where the completion-exposure χ ramps r up to the tail budget and buys completion time (Sections 14.25, 14.26).

The protocol hint selects the mode and constraints:



When $\rho < 100\%$, the taper has a finite cutoff T_{cut} . Symbols not recovered by T_{cut} are permanently lost. When $\rho = 100\%$, $T_{\text{cut}} = \infty$ (correction symbols continue until ACK, the infinite-tailed taper from Section 4).

$$\pi_B = p / (p + q) \quad \text{probability of being in Bad state}$$

$$\pi_G = q / (p + q) \quad \text{probability of being in Good state}$$

Average loss rate:

$$e = \pi_B \times h_B + \pi_G \times h_G = \pi_B = p / (p + q)$$

2.3 Burst Length Distribution

Given we just entered the Bad state, the burst length T follows a geometric distribution:

$$P(T = t) = q \times (1-q)^{(t-1)} \quad \text{for } t = 1, 2, 3, \dots$$

$$P(T \geq t) = (1-q)^{(t-1)} \quad \text{survival function}$$

$$E[T] = 1/q = B \quad \text{mean burst length}$$

This survival function is the key quantity — it tells us, given that a burst started, how likely it is to still be ongoing after t symbols.

2.4 Concrete Examples

Scenario	ε (loss)	p (G→B)	q (B→G)	$B = 1/q$	Character
DC	0.1%	0.0005	0.5	2.0	Rare, short bursts
WiFi	2.5%	0.013	0.5	2.0	Moderate, short
LTE	5%	0.02	0.4	2.5	Moderate, medium
Satellite	9%	0.03	0.3	3.3	Frequent, long
Bad WiFi	15%	0.053	0.3	3.3	Frequent, long

(Consistency check: $\varepsilon = p/(p+q)$. E.g. DC: $0.0005/0.5005 \approx 0.1\%$; WiFi: $0.013/0.513 \approx 2.5\%$; Satellite: $0.03/0.33 \approx 9.1\%$.)

3. Recovery Fundamentals

3.1 The Problem: Recovering Lost Symbols

When a source symbol is erased by the channel, the receiver has a gap in the data stream. There are two fundamentally different ways to fill that gap:

1. **Send extra data proactively** (before knowing what will be lost): the sender generates redundant symbols alongside source data. If a source symbol is lost, the redundant symbols can reconstruct it. This costs bandwidth (the redundant symbols are sent whether needed or not) but adds no latency (they arrive at roughly the same time as the source).
2. **Detect the loss and resend** (after knowing what was lost): the sender discovers that a symbol wasn't received (via ACK timeout) and retransmits it. This costs latency (at least one round-trip to detect + retransmit) but wastes no bandwidth (only lost symbols are resent).

These two approaches are called **FEC** (Forward Error Correction) and **ARQ** (Automatic Repeat reQuest). The challenge is finding the right balance between them — more FEC means lower latency but higher bandwidth; more ARQ means lower bandwidth but higher latency.

3.2 FEC: Proactive Recovery

The encoder window. The sender maintains a sliding window of the W most recent source symbols. This window is the encoder's "memory" — it tracks which source symbols are available for generating repair:

Encoder window ($W = 5$ symbols):

```
sent:  [S1] [S2] [S3] [S4] [S5] [S6] [S7] [S8] ...
        |----- window -----|
        S3  S4  S5  S6  S7
```

As new symbols arrive, old ones are evicted from the left.

Repair symbols are random linear combinations of source symbols in the window. Each repair symbol is computed as:

$$R = c1*S3 + c2*S4 + c3*S5 + c4*S6 + c5*S7$$

where $c1..c5$ are random coefficients in $GF(256)$ (a finite field — arithmetic wraps around so values stay within 0-255). Each repair symbol is a different random combination, giving the decoder independent equations.

How decoding works. If k source symbols in the window are lost, the receiver needs at least k repair symbols that survived the channel. Each repair symbol provides one linear equation relating the lost symbols. With k equations and k unknowns, the decoder solves the system (Gaussian elimination over $GF(256)$) to recover the lost data.

Example: source S3 and S5 lost, S4/S6/S7 received

Time ----->

```
Source:  [S1] [S2] [S3] [S4] [S5] [S6] [S7]
           X       X
Repair:  [R1] [R2] [R3] [R4]
           |   |   |   |
           v   v   v   v
        covers covers covers covers
        S1-S3  S2-S4  S3-S6  S5-S7
```

Decoder receives R1, R2, R3, R4 and knows S4, S6, S7.

Substituting known values into R3 and R4:

$$R3 = c1*S3 + c2*(\text{known } S4) + c3*S5 + c4*(\text{known } S6)$$

$$R4 = c5*S5 + c6*(\text{known } S6) + c7*(\text{known } S7)$$

-> 2 equations, 2 unknowns (S3, S5) -> solve.

Systematic codes. In our system, source symbols are sent as-is — they are not encoded. This means if nothing is lost, the receiver can use the data immediately without any decoding. The decoder is only invoked when at least one source symbol is missing. This is important for efficiency: at low loss rates, the decoder rarely runs.

Codec overhead. Not all random linear combinations are independent. Some may be (near-)linearly dependent, providing redundant equations. This means the decoder sometimes needs slightly more repair symbols than the number of losses. This extra requirement is the codec overhead — e.g., 1% for RaptorQ means the decoder needs 1% more symbols than the theoretical minimum of k .

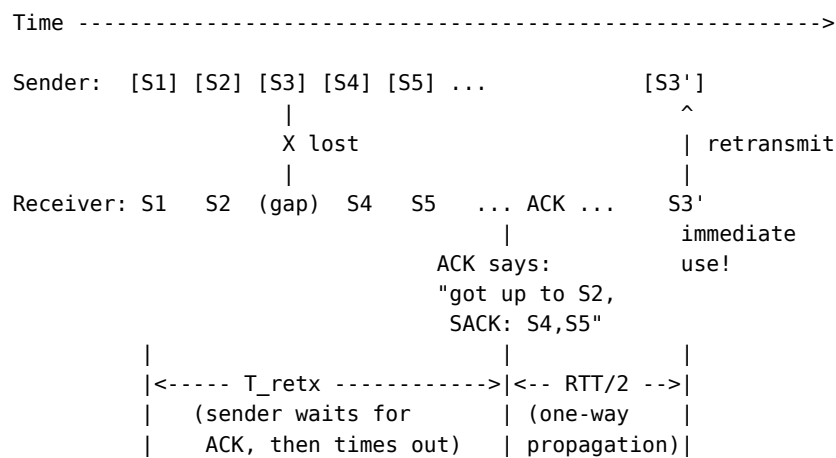
Properties of FEC: - Bandwidth cost: r repair symbols per source symbol (always-on, paid whether loss occurs or not) - Latency cost: zero additional (repair arrives at roughly the same time as source) - Bandwidth fraction used: $r/(1+r)$ of total link capacity

3.3 ARQ: Reactive Recovery

ARQ recovers lost symbols by retransmitting them after detecting the loss. In our system, loss detection is **sender-side**: the sender tracks which symbols have been ACKed and retransmits any that remain un-ACKed after a timeout.

This is how TCP has worked for 40 years [RFC2018] and is proven robust for unicast connections. The alternative (receiver-side detection via NACK messages) adds detection delay and is fragile when the reverse path is lossy.

Timeline of ARQ recovery:



The sender sent S3 at time 0. After T_{retx} (roughly one RTT, enough time for an ACK to return), no ACK has confirmed S3. The sender retransmits S3. The retransmitted symbol travels one-way ($RTT/2$) to the receiver.

Key advantage over FEC: A retransmitted source symbol is immediately usable by the receiver — no decoder needed, no dependency on other symbols. The receiver gets the exact data it was missing and can process it right away.

Key disadvantage: The retransmission adds $L_{\text{arq}} = T_{\text{retx}} + RTT/2$ of latency. For a link with 50ms RTT and $T_{\text{retx}} = 75\text{ms}$: $L_{\text{arq}} = 100\text{ms}$. This is acceptable for bulk transfer but too slow for real-time applications.

Properties of ARQ: - Bandwidth cost: approximately ϵ per source symbol (only retransmit what's actually lost — no waste) - Latency cost: $L_{\text{arq}} = T_{\text{retx}} + RTT/2$ per recovery event - Self-healing: if the ACK itself is lost on the reverse path, the sender just waits longer and retransmits anyway. No special mechanism needed.

3.4 Recovery Latency and the $P_{\text{lost}}(t)$ Model

A lost symbol is recovered by whichever mechanism finishes first — FEC or ARQ. Rather than a hard timeout threshold, the choice between repair and retransmit is **probabilistic**, based on the sender's confidence that a symbol was lost.

FEC recovery time (t_{fec}): After a loss, repair symbols keep arriving. Every repair is a linear combination of the entire encoder window (Section 3.2), so every surviving repair gives the decoder one more equation for the lost symbol — not just the repairs “attributed” to that symbol by its own taper. In steady state, corrections occupy a fraction $r/(1+r)$ of wire slots (Section 5.3), so useful equations arrive at rate $r/(1+r) \times (1-e)$ per slot. For m lost symbols in the window, the decoder needs m surviving repairs:

$$t_{\text{fec}} = m \times (1+r) / (r \times (1-e)) \times t_{\text{sym}}$$

where:

m = number of lost symbols in the window (usually 1)
 r = total correction rate (corrections per source symbol)
 $1-e$ = probability each repair survives the channel
 t_{sym} = symbol_size / throughput (time to transmit one symbol)

Concrete examples for a single loss ($m=1$), $r=0.08$, $e=0.025$:

At 100 Mbps, 1200-byte symbols: $t_{\text{sym}} = 0.096\text{ms}$, $t_{\text{fec}} = 1.3\text{ms}$
 At 10 Mbps, 1200-byte symbols: $t_{\text{sym}} = 0.96\text{ms}$, $t_{\text{fec}} = 13.3\text{ms}$
 At 1 Mbps, 1200-byte symbols: $t_{\text{sym}} = 9.6\text{ms}$, $t_{\text{fec}} = 133\text{ms}$

For burst loss ($m=5$ on WiFi at 100 Mbps): $t_{\text{fec}} = 5 \times 1.3\text{ms} = 6.6\text{ms}$.

$P_{\text{lost}}(t)$: the probability a symbol was lost. At time t after sending a symbol, given no ACK has arrived, what is the probability it was actually lost?

If the symbol was received, the ACK should arrive after roughly one RTT. If the symbol was lost, no ACK will ever come (until a correction recovers it). Using Bayes’ theorem:

$$P_{\text{lost}}(t) = e / [e + (1-e) \times P(\text{RTT} > t)]$$

where:

e = channel loss rate (prior probability of loss)
 $P(\text{RTT} > t)$ = probability the ACK is delayed beyond time t
 = tail of the RTT distribution (from SRTT and RTTVAR)

Assuming RTT is normally distributed with mean SRTT and standard deviation RTTVAR: $P(\text{RTT} > t) = 1 - \Phi((t - \text{SRTT}) / \text{RTTVAR})$, where Φ is the standard normal CDF. This is the same normal approximation used in TCP’s RTO computation [RFC6298].

This gives a smooth transition from “probably fine” to “certainly lost”:

$t = 0$:	$P_{\text{lost}} = e$	(just the base loss rate)
$t = \text{SRTT}$:	$P_{\text{lost}} \approx 2e$	(ACK expected by now)
$t = \text{SRTT} + 2s$:	$P_{\text{lost}} \approx 0.5$	(loss is now the likelier explanation)
$t = \text{SRTT} + 4s$:	$P_{\text{lost}} > 0.99$	(very confident it's lost)
$t \gg \text{SRTT}$:	$P_{\text{lost}} \rightarrow 1.0$	(certainly lost)

Concrete example (WiFi, $e = 0.025$, SRTT = 50ms, RTTVAR = 5ms):

$t = 0\text{ms}$:	$P_{\text{lost}} = 0.025$	-> 97.5% repair, 2.5% retransmit
$t = 40\text{ms}$:	$P_{\text{lost}} = 0.026$	-> 97% repair, 3% retransmit
$t = 50\text{ms}$:	$P_{\text{lost}} = 0.049$	-> 95% repair, 5% retransmit
$t = 55\text{ms}$:	$P_{\text{lost}} = 0.14$	-> 86% repair, 14% retransmit
$t = 60\text{ms}$:	$P_{\text{lost}} = 0.53$	-> 47% repair, 53% retransmit
$t = 70\text{ms}$:	$P_{\text{lost}} = 0.999$	-> ~0% repair, ~100% retransmit

Note how sharp the transition is: P_{lost} stays near ϵ until roughly $\text{SRTT} + 1 \times \text{RTTVAR}$, then rises to near-certainty within another $2-3 \times \text{RTTVAR}$. The smoothness of the mix in practice

comes from the spread of symbol ages in the retransmit buffer, not from the curve itself being gradual.

The per-slot decision uses $P_{\text{lost}}(t)$ directly:

$$P(\text{retransmit}) = P_{\text{lost}}(t_k)$$

No hard threshold. The transition from repair to retransmit is driven by TIME since send. Early (t small): P_{lost} is low, so corrections are mostly repair — flexible coverage for uncertain losses. Late ($t \gg \text{SRTT}$): P_{lost} approaches 1, so corrections are mostly retransmit — targeted recovery of confirmed losses. No tuning knobs.

See Appendix C.6 for an optional channel-state discount (ϵ_{burst}) that provides modest improvement for long-burst scenarios.

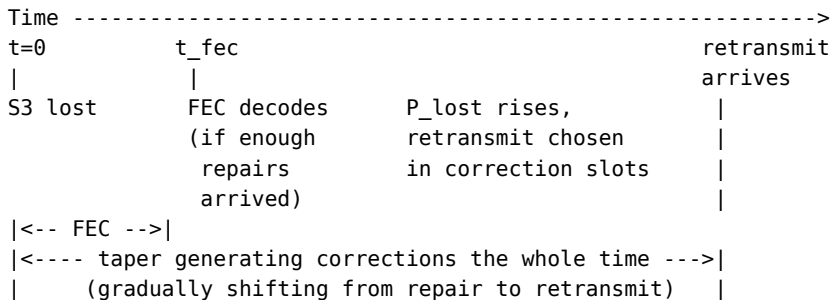
Bandwidth efficiency. A retransmit is wasted if the symbol wasn't actually lost (duplicate). A repair is never wasted (always provides information to the decoder). The expected waste per correction slot is:

$$\begin{aligned} E[\text{waste}] &= P(\text{retransmit chosen}) \times P(\text{symbol not actually lost}) \\ &= P_{\text{lost}}(t) \times (1 - P_{\text{lost}}(t)) \end{aligned}$$

This is maximized at $P_{\text{lost}} = 0.5$ (maximum uncertainty) and zero at the extremes. The probabilistic model automatically minimizes waste.

Proactive retransmit emerges naturally. At high loss rates (e.g., $e = 0.5$), $P_{\text{lost}}(0) = 0.5$ — half the correction slots are retransmits even at $t = 0$. This is proactive redundant source sending, no special mode needed. At low loss rates ($e = 0.001$), $P_{\text{lost}}(0) = 0.001$ — virtually all repair. The model adapts to the channel automatically.

Which mechanism wins on latency? FEC recovery (t_{fec}) is typically much faster than ARQ recovery (waiting for P_{lost} to rise + $\text{RTT}/2$):



At 100 Mbps on WiFi: $t_{\text{fec}} = 1.3\text{ms}$. Most losses are FEC-recovered long before P_{lost} rises high enough for retransmission. ARQ retransmit is only relevant for burst losses that overwhelm the FEC budget.

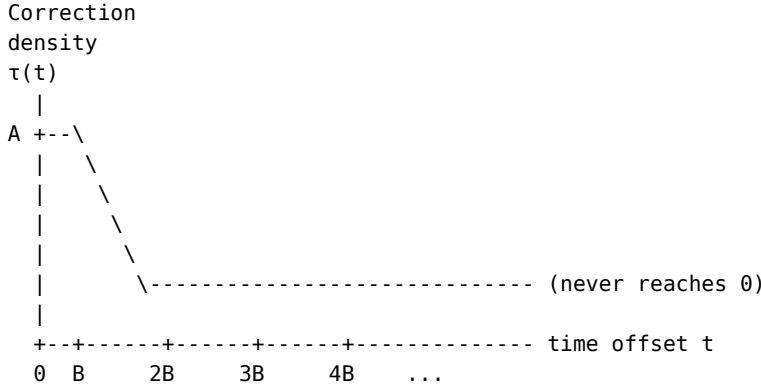
4. The Taper Function

Why not send correction symbols at a constant rate? If loss were uniformly distributed over time (i.i.d.), a constant rate would be optimal — every position is equally likely to need correction. But on real wireless channels, loss comes in **bursts** (Section 2). A burst wipes out consecutive symbols, then the channel recovers. Right after a burst starts, the probability of continued loss is high; as time passes, it decays exponentially.

A constant correction rate misallocates: it sends too many corrections during good periods (wasted) and too few right after a burst (when they're needed most). The taper function solves this by matching correction density to the burst survival probability — more corrections where loss is likely, fewer where it isn't. This minimizes the total correction budget for a given recovery target.

4.1 Definition

The taper function $\tau(t)$ specifies the correction density at time offset t from a source symbol. At offset t after symbol s enters the window, we generate $\tau(t)$ correction symbols covering s .



$$\tau(t) = A \times (1-q)^t$$

A = amplitude (what we solve for)
 $(1-q)^t$ = GE burst survival function

4.2 Why Match the Loss Distribution?

The taper should allocate more correction where it does the most good. Given that a symbol was lost (we're in a burst), the conditional probability that the burst is still active at offset t is:

$$P(\text{burst active at offset } t \mid \text{burst at offset } 0) = (1-q)^t$$

A correction at offset t is useful only if BOTH (a) the lost symbol still needs help at t , and (b) the correction itself survives the channel. These pull in opposite directions: (a) decays like the burst survival function $(1-q)^t$, while (b) — conditioned on the loss at offset 0 — is the probability the burst has ENDED by t , roughly $1 - (1-q)^t$. Corrections sent into an ongoing burst are themselves lost (Section 14.15). The product

$$\text{benefit}(t) \propto (1-q)^t \times (1 - (1-q)^t)$$

is hump-shaped: zero at $t = 0$, peaking around one mean burst length ($t \approx B$), then decaying like $(1-q)^t$. The taper's exponential decay matches the correct TAIL behavior; its peak at $t = 0$ is an approximation that over-weights the first few offsets, where corrections have the lowest conditional survival probability. Two effects limit the cost of this approximation:

1. **Repairs are window-fungible.** Every repair covers the whole encoder window (Section 3.2), so a correction “attributed” to offset 0 of a fresh symbol simultaneously sits at larger offsets for every older symbol in the window. Per-symbol attribution is bookkeeping, not physics — recovery depends on the aggregate correction rate (Section 8.2), which the shape does not change in steady state (see below).

2. **The exponential tail is the operationally important part.** The $(1-q)^t$ decay controls how long un-ACKed (increasingly likely lost) symbols keep drawing correction coverage after everything else has been ACKed.

Steady-state shape invariance. With a continuous source stream, the aggregate correction rate per wire slot is $\sum_t \tau(t) = r$ regardless of the taper's shape — every in-window symbol contributes its taper at a different age, and the sum over ages telescopes to the same total. The shape only affects behavior in transients: when the source pauses, when the window is not yet full, and — most importantly — through ACK truncation (Section 4.4): once a symbol is ACKed, its taper contribution stops, so beyond one RTT only un-ACKed (likely lost) symbols continue to generate corrections. The decay rate q determines how aggressively that residual coverage tapers off. The taper shape is therefore a policy for allocating corrections across UNCERTAIN symbols, not a mechanism that changes the steady-state budget.

For an i.i.d. channel ($q = 1$, no burst memory): $\tau(t) = \text{constant}$ (flat taper). This is correct — every position is equally likely to need correction.

4.3 Total Correction Rate

The total correction rate (correction symbols per source symbol) is:

$$r = \sum_{t=0}^{\infty} \tau(t) = A \times \sum_{t=0}^{\infty} (1-q)^t = A / q$$

Since $0 < q \leq 1$, this geometric series converges. Therefore:

$$A = r \times q$$

The amplitude is uniquely determined by the correction rate and the GE parameter.

4.4 The Taper Never Reaches Zero

The exponential $(1-q)^t$ is always positive for $0 < q < 1$. This is correct behavior: there is always a nonzero probability of a burst still continuing. As long as a symbol has not been ACK'd, there is a nonzero probability it was lost, so we should continue generating (increasingly rare) correction coverage.

$t = 0:$	$\tau(0) = A$	peak correction density
$t = B:$	$\tau(B) = A \times e^{-1} \approx 0.37 \times A$	one mean burst length
$t = 2B:$	$\tau(2B) = A \times e^{-2} \approx 0.14 \times A$	two mean burst lengths
$t = 5B:$	$\tau(5B) = A \times e^{-5} \approx 0.007 \times A$	five mean burst lengths
$t \rightarrow \infty:$	$\tau(t) \rightarrow 0$	but never zero

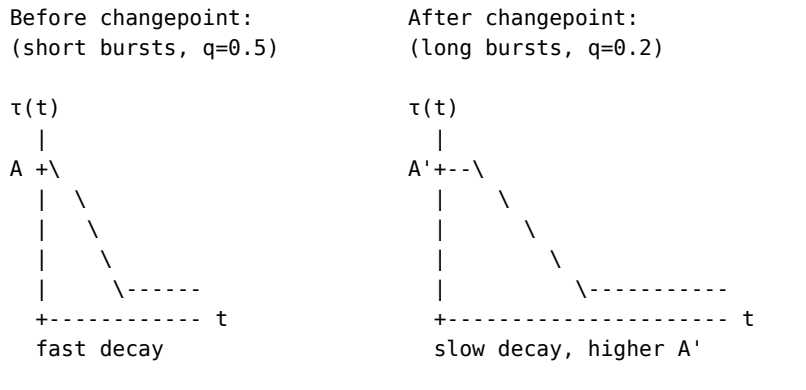
In practice, once a symbol is ACK'd, we stop generating correction for it (the encoder window advances past it). The theoretical infinite tail is truncated by the ACK mechanism.

4.5 Real-Time Adaptation

The taper function adapts in real time through two mechanisms:

1. **GE parameter updates:** The estimator continuously tracks q (and p) from observed loss patterns. As q changes, the taper shape changes — slower decay for longer bursts, faster decay for shorter bursts.
2. **BOCD changepoint detection:** If the loss regime changes abruptly (e.g., path switches from WiFi to LTE), BOCD detects the changepoint within 5-15 batches (a batch = one ACK feedback cycle, typically at intervals of 10-100ms depending on sending rate and path

RTT) and widens the posterior, increasing the correction budget until the new regime is characterized.



5. Unified Correction Symbol Model

5.1 Why Unify FEC and ARQ?

Both FEC and ARQ produce one symbol on the wire. Both cost the same bandwidth per symbol. The difference is timing and content:

FEC repair: generated immediately, random combination, needs decoder
 ARQ retransmit: generated after timeout, exact source copy, immediately usable

Both occupy one symbol slot on the wire.
 Both are subject to the same channel loss rate e .
 Both serve the same purpose: recover a lost source symbol.

The key insight: **the taper function controls WHEN to generate correction symbols** (the density schedule). **The per-slot decision controls WHAT to generate** (repair or retransmit). These are orthogonal choices:

- **Early in the taper** (right after sending source): $P_{\text{lost}}(t)$ is low, so almost all correction symbols are FEC repair. This is pure proactive protection — we don't yet know what's lost.
- **Late in the taper** ($t \gg \text{SRTT}$): $P_{\text{lost}}(t)$ approaches 1, so correction symbols are almost all retransmits. We're confident about which symbols are lost and send the exact data the receiver needs.
- **In between**: a smooth probabilistic mix. No hard phase switch.

This means the taper function **naturally transitions from FEC to ARQ** as time passes, driven by the $P_{\text{lost}}(t)$ posterior. The unified mechanism is called a **correction symbol** — it is either a repair symbol or a source retransmit, decided probabilistically at generation time.

5.2 Correction Symbols — The Unified Concept

A **correction symbol** is any symbol sent to recover lost data. It occupies one symbol slot on the wire and serves one of two purposes:

Aspect	Source retransmit (ARQ)	Repair symbol (FEC)
When chosen	With probability $P_{\text{lost}}(t)$	With probability $1-P_{\text{lost}}(t)$
Content	Exact copy of source symbol	Random linear combination

Aspect	Source retransmit (ARQ)	Repair symbol (FEC)
Receiver action	Immediate use, no decoder	Feed to FEC decoder
Bandwidth cost	Same (one symbol slot)	Same (one symbol slot)
Waste risk	Duplicate if symbol wasn't lost	Never wasted (always useful)
Best for	Confirmed loss + good channel	Uncertain loss or burst (flexible)

The taper function (Section 4) determines the **density** of correction symbols over time. This section explains what happens in each slot.

5.3 Three-Stream View: Source, Repair, Retransmit

Conceptually, the sender manages **three streams** that compete for wire capacity. Each stream has a different effect on the bandwidth/latency/ reliability triangle:

Stream	Latency	Bandwidth	Reliability
Source	++ immediate	neutral	neutral
Repair (FEC)	- decoder wait	+ (no waste if taper matched)	++ (covers any loss)
Retransmit (ARQ)	+ immediate decode	- (duplicate risk)	+ (targeted recovery)

The hierarchical model (taper + P_lost) determines the three-stream mix:

Total capacity: C (from Copa)

```

source_rate      = C / (1 + r)                [from taper ratio r]
retransmit_rate  = C x r / (1+r) x P_lost(t)   [confirmed losses]
repair_rate       = C x r / (1+r) x (1 - P_lost(t)) [uncertain losses]

```

where r = taper correction ratio (Section 4)
P_lost(t) = loss confidence (Section 3.4)

The three rates sum to C. The taper ratio r controls the source/correction split. P_lost(t) controls the repair/retransmit split within corrections.

How the protocol hint shifts the mix:

Realtime (low latency):

- > tight delta -> high r (more corrections for fast FEC recovery)
- > BUT also high P_retx when confident (retransmit = immediate decode)
- > Mix: moderate source, moderate repair, moderate retransmit

Bulk (high bandwidth):

- > loose delta -> low r (fewer corrections, more source)
- > low P_retx (prefer repair over retransmit, less duplicate risk)
- > Mix: lots of source, some repair, little retransmit

Concretely, Bulk's tail target is "late is fine", weighted by the completion exposure chi (Section 14.26):

$$\text{delta_bulk} = \text{epsilon_hat} + (\text{delta_tail} - \text{epsilon_hat}) \times \text{chi}(T_{\text{rem}})$$

Mid-stream chi = 0, so delta_bulk = epsilon_hat and the continuous r* formula (Section 8.4, $z_{\{\text{delta}/\text{epsilon}\}} = \Phi^{-1}(1 - \text{delta}/\text{epsilon}) = \Phi^{-1}(0) = -\text{inf}$) yields r = 0 IDENTICALLY in the

steady state – independent of estimator uncertainty: pure ARQ, wire volume at parity with retransmission transports ($\sim 1 + \epsilon/(1-\epsilon)$ per source symbol). A mid-transfer loss recovered one RTT late costs a bulk transfer nothing – recovery overlaps ongoing sends. The one place FEC still buys completion time is the stream tail, where χ rises smoothly to 1 and δ_{bulk} glides to the $\delta_{\text{tail}} = 0.05$ completion budget (Sections 14.25, 14.26).

VoIP (fixed bandwidth + latency, variable reliability):
 -> r constrained by bandwidth budget
 -> high P_{retx} when confident (immediate decode matters)
 -> Mix: fixed source rate, split corrections by P_{retx}

Per-path three-stream fractions: Each path has its own r_i and P_{retx_i} , so the three-stream fractions naturally differ per path. A low-latency path with low loss has: high source, low repair, low retransmit. A high-loss path has: lower source, high repair, moderate retransmit. The hierarchical model produces these automatically – no separate three-way optimization needed.

5.4 Per-Slot Decision

When the taper function (Section 4) decides to generate a correction symbol, the sender makes a probabilistic choice based on $P_{\text{lost}}(t)$ for the oldest un-ACKed symbol in the retransmit buffer:

Taper decides: "generate a correction symbol now"

```

      |
      v
+-----+
| Compute mixing probability:          |
|                                     |
|    $P_{\text{retx}} = P_{\text{lost}}(t_k)$           |
|                                     |
| With probability  $P_{\text{retx}}$ :              |
|   -> Retransmit exact source          |
|       (immediate decode at receiver)  |
|                                     |
| With probability  $1 - P_{\text{retx}}$ :          |
|   -> Generate random repair symbol    |
|       (FEC, covers any loss in window)|
+-----+

```

$P_{\text{lost}}(t_k) = e / [e + (1-e) \times P(\text{RTT} > t_k)]$ per-symbol loss confidence

The FEC/ARQ mix is regulated by TIME since send. Early correction slots (t small) have low P_{lost} – the ACK hasn't had time to return, so loss is uncertain. Almost all corrections are FEC repair, which flexibly covers any lost symbol. Late correction slots ($t \gg \text{SRTT}$) have high P_{lost} – the ACK should have arrived by now, so loss is confirmed. Corrections shift to retransmit, which is immediately decodable.

This naturally produces Mehrotra & Li's optimal policy [Mehrotra2010]: during bursts (before RTT elapses), P_{lost} stays low → mostly repair. After bursts (ACKs reveal gaps), P_{lost} rises → targeted retransmit.

The expected WASTE, $E[\text{waste}] = P_{\text{lost}} \times (1 - P_{\text{lost}})$ (Section 3.4), is zero at the extremes ($P_{\text{lost}} = 0$: all repair, no waste; $P_{\text{lost}} = 1$: all retransmit, definitely lost so no waste) and

maximized at $P_{\text{lost}} = 0.5$ (maximum uncertainty). The model automatically allocates the right mix.

See Appendix C.6 for an optional channel-state discount (ϵ_{burst}) that provides modest improvement for long-burst scenarios.

6. Protocol Mechanics

6.1 The Retransmit Buffer

The sender maintains a **retransmit buffer**: an ordered list of source symbols that have been sent but not yet ACKed.

Retransmit buffer (ordered by send time, oldest first):

```
+-----+-----+-----+-----+-----+-----+
| S12 | S13 | S17 | S18 | S19 | S24 | S25 | <- un-ACKed symbols
+-----+-----+-----+-----+-----+-----+
  ^                               ^
oldest                         newest
(first retransmit               (too recent,
candidate if                    wait for ACK)
age > T_retx)
```

Operations: - **Enqueue**: every sent source symbol is added to the tail - **Dequeue (ACK)**: when an ACK or SACK confirms receipt, remove the symbol - **Peek (retransmit)**: when generating a correction symbol, check the head — if its age exceeds T_{retx} , retransmit it (but keep it in the buffer until ACKed)

The retransmit buffer is NOT bounded by the encoder window. The encoder window (W symbols) bounds what FEC repair can cover. The retransmit buffer bounds what ARQ retransmit can cover. They serve different purposes:

- Symbol in encoder window AND buffer: protected by FEC + ARQ
- Symbol evicted from window, still in buffer: protected by ARQ only
- Symbol ACKed: removed from buffer (delivered)

Two independent mechanisms manage the buffer:

Age eviction (T_{cut}): Symbols older than T_{cut} (derived from the reliability target ρ in Section 8.6) are evicted from the buffer — accepted as permanently lost. T_{cut} is derived from the triangle optimization (Section 8.6): for reliability target ρ , T_{cut} is the time where $P(\text{not recovered}) = 1 - \rho$. When $\rho = 100\%$, the equation has no finite solution, so $T_{\text{cut}} = \infty$ (never evict). This emerges from the math — not a special case.

Size backpressure (buffer_max): When the buffer reaches buffer_max entries, the sender pauses accepting new source symbols from the application (backpressure). Correction symbols continue being generated for existing buffer contents — corrections recover lost symbols, ACKs arrive, and the buffer drains. Once space is available, source resumes.

buffer_max is derived, not configured:

For $\rho < 100\%$: $\text{buffer_max} = \text{source_rate} \times T_{\text{cut}}$
 (eviction keeps buffer within this bound)

For $\rho = 100\%$: $\text{buffer_max} = \text{source_rate} \times (\text{RTT} + B_{\text{max}} / (r^* \times (1-e)) \times t_{\text{sym}})$
 where $B_{\text{max}} = \text{ceil}(\ln(0.0001) / \ln(1-q))$
 (99.99th percentile burst length from GE model;
 $\approx 9.2/q$ for small q , = 14 exactly at $q = 0.5$)

Both mechanisms always run. T_{cut} determines which triggers first: - Finite $T_{\text{cut}} \rightarrow$ eviction keeps buffer small $\rightarrow$ backpressure rarely triggers - $T_{\text{cut}} = \infty \rightarrow$ no eviction $\rightarrow$ backpressure is the only limit

What bends when: - $\rho < 100\%$: reliability bends (eviction at T_{cut}) - $\rho = 100\%$: latency bends (backpressure at buffer_max , rare)

6.2 SACK-Extended ACK

The receiver sends an **ACK per received symbol** — every incoming symbol triggers an immediate ACK in the reverse direction. This provides the fastest possible loss detection and the most RTT samples for estimation.

Overhead: Each ACK is ~80 bytes (fields + QUIC/UDP headers). With 1200-byte symbols, per-symbol ACKs generate reverse-path traffic of about $80/1200 \approx 6.7\%$ of the forward data rate. On symmetric links this is an acceptable price for per-symbol loss detection and RTT sampling. On strongly asymmetric uplinks (LTE, satellite, DOCSIS) it can be significant, and the per-batch or piggyback alternatives below become attractive; the model itself is agnostic to ACK batching (it only slows the P_{lost} transition by the batch interval).

Alternatives considered: - Per-batch ACK: lower overhead, but slower loss detection (batch granularity) - Delayed ACK (TCP style, every 200ms): 50% less traffic, but adds 200ms detection delay — too slow for our P_{lost} model - Piggyback ACK on reverse-direction data: zero overhead for bidirectional traffic — a future optimization for the bidirectional case

ACK contents (5 fields):

```
+-----+
| cumulative_ack:      u64    all seqs <= this received |
| sack_ranges:         [(u64,u64)] additional ranges   |
| echo_timestamp:      u64    sender's timestamp       |
| jitter_us:           u32    interarrival jitter (us)  |
| cumulative_received: u64    total symbols received    |
+-----+
```

- **cumulative_ack**: highest sequence number such that ALL symbols up to and including it have been received. An optimization — equivalent to a SACK range starting at 0, but compressed to one number.
- **sack_ranges** (Selective ACK [RFC2018]): out-of-order ranges received beyond the cumulative point. Tells the sender exactly which symbols arrived despite gaps. Cumulative within the T_{cut} window — all received fragments are reported, not just the most recent (unlike TCP which limits to 3-4 blocks).
- **echo_timestamp**: sender's own send timestamp echoed back for RTT measurement. $\text{RTT} = \text{now} - \text{echo}$. No clock synchronization needed.

- **jitter_us**: interarrival jitter in microseconds [RFC3550 A.8]. u32 holds up to 4300 seconds (matches RFC 3550's 32-bit field). Used for reorder buffer sizing, congestion detection, and real-time jitter budgets.
- **cumulative_received**: running total of symbols received. Self-healing counter that survives ACK loss and SACK pruning. Gives the sender a reliable aggregate reliability metric: $\rho_{\text{actual}} = \text{received} / \text{sent}$.

Gap pruning at T_{cut} : For $\rho < 100\%$, gaps older than T_{cut} are pruned by the receiver — cumulative_ack advances past abandoned symbols. Both sender (buffer eviction) and receiver (gap pruning) use the same T_{cut} , communicated at connection start and updatable mid-connection via control message.

For $\rho = 100\%$ ($T_{\text{cut}} = \infty$): no pruning. cumulative_ack advances only when gaps are filled by corrections. SACK ranges are temporary.

ACK loss handling: Cumulative ACKs are self-healing — each new ACK supersedes all previous ones. If ACK_5 is lost but ACK_6 arrives, the sender gets all information from ACK_6. No ACK retransmission needed.

6.3 Per-Symbol Delivery Outcomes

A lost symbol has three possible outcomes, depending on whether the taper function's correction symbols recover it before the cutoff T_{cut} :

Outcome 1: FEC-recovered (proactive repair arrives before T_{retx})
 Latency: $L_{\text{prop}} + \text{small delay (same as source)}$
 Probability: $e \times P_{\text{fec}}$

Outcome 2: ARQ-recovered (retransmit arrives after T_{retx} , before T_{cut})
 Latency: $L_{\text{prop}} + L_{\text{arq}}$ where $L_{\text{arq}} = T_{\text{retx}} + \text{RTT}/2$
 Probability: $e \times (1 - P_{\text{fec}}) \times P_{\text{arq}}$

Outcome 3: Lost (not recovered by T_{cut})
 Latency: infinity (never delivered)
 Probability: $e \times (1 - P_{\text{fec}}) \times (1 - P_{\text{arq}})$

P_{arq} is the probability that ARQ retransmission succeeds, given that FEC failed to recover the symbol. It is a conditional probability in this chain:

```

Symbol sent
|
+-- (1-e): arrives directly      -> delivered (on-time)
|
+-- e: lost on the channel
    |
    +-- P_fec: FEC recovers it    -> delivered (on-time)
    |
    +-- (1-P_fec): FEC failed
        |
        +-- P_arq: ARQ succeeds    -> delivered (late, L_arq)
        |
        +-- (1-P_arq): ARQ fails    -> permanently lost
  
```

P_{arq} is derived from the reliability target ρ , not computed independently:

$$P(\text{lost}) = e \times (1 - P_{\text{fec}}) \times (1 - P_{\text{arq}}) = 1 - \rho$$

$$\text{Solving: } P_{\text{arq}} = 1 - (1 - \rho) / (e \times (1 - P_{\text{fec}}))$$

For $\rho = 100\%$: $P_{\text{arq}} = 1$. We keep retransmitting until ACK ($T_{\text{cut}} = \text{infinity}$), so ARQ eventually succeeds. FEC is still active — it handles most recoveries fast. ARQ is the backstop for what FEC misses.

For $\rho < 100\%$: $P_{\text{arq}} < 1$. We stop retransmitting at T_{cut} . Symbols not recovered by then are permanently lost.

The full delivery distribution:

$$\begin{aligned} P(\text{on-time delivery}) &= (1 - e) + e \times P_{\text{fec}} && \text{not lost, or FEC} \\ P(\text{late delivery}) &= e \times (1 - P_{\text{fec}}) \times P_{\text{arq}} && \text{ARQ retransmit} \\ P(\text{permanent loss}) &= e \times (1 - P_{\text{fec}}) \times (1 - P_{\text{arq}}) && \text{not recovered} \end{aligned}$$

$$\text{Reliability: } \rho = 1 - P(\text{permanent loss})$$

$$\text{Tail latency: } \delta = P(\text{late delivery}) / \rho \quad \text{among delivered symbols}$$

6.4 The Triangle in Action

Under **100% reliability** ($\rho = 1$, $T_{\text{cut}} = \text{infinity}$): outcome 3 never occurs. “Tail loss from FEC” equals “tail latency events” — they are the same thing. This is the special case from Section 8.

Under **variable reliability** ($\rho < 1$, $T_{\text{cut}} < \text{infinity}$): the taper is cut off. Symbols beyond T_{cut} are permanently lost. This saves bandwidth (fewer correction symbols) and bounds latency (no recovery beyond T_{cut}), at the cost of reliability.

$\rho = 100\%$: ----- (taper runs until ACK)
all symbols eventually delivered

$\rho = 98\%$: -----+
98% delivered | T_{cut}
2% lost +-- (taper stops, accept loss)

$\rho = 95\%$: -----+
95% | T_{cut} (shorter)
5% lost +-- accept loss (sensor/VoIP)

6.5 Four-Mechanism Composition

The complete system is four independent mechanisms that compose without interfering:

Mechanism	Controls	Derives from
Copa	Total wire rate	RTT, bandwidth (delay-based)
Taper	Source/correction ratio	e , q , δ (GE + triangle)
T_{cut} + buffer	Reliability guarantee	ρ (triangle), GE B_{max}
P_{lost}	Repair/retransmit mix	ACK timing, e

Each mechanism has one job and one set of inputs. None needs to know about the others:

- Copa doesn’t know about FEC — it just limits total rate
- The taper doesn’t know about Copa — it just sets the ratio
- T_{cut} doesn’t know about the taper — it just evicts old symbols
- P_{lost} doesn’t know about T_{cut} — it just picks repair vs retransmit

The protocol hint flows through the triangle to set delta and rho, which determine the taper amplitude and T_cut. Copa independently discovers the pipe capacity. P_lost independently tracks per-symbol loss confidence. The four mechanisms compose to produce the correct behavior for each protocol class without branching or mode switches.

7. Estimation — From Observations to Channel Parameters

7.1 What We Observe

At the sender, we receive periodic feedback:

Observation	Source	Frequency
(sent, received) per batch	ACK messages	Every batch (~10-100ms)
RTT	Echoed timestamps in ACK	Every batch
SACK ranges (out-of-order blocks)	ACK+SACK messages	Every ACK
Throughput	Delivery rate tracking	Continuous

The sender infers losses from gaps: symbols not covered by the cumulative ACK or any SACK range are presumed lost after T_retx has elapsed.

7.2 EWMA — Fast Point Estimate

Exponentially Weighted Moving Average of the loss rate:

$$e_hat_ewma(n) = \alpha \times (lost/sent) + (1-\alpha) \times e_hat_ewma(n-1)$$

$$\alpha = 0.1 \rightarrow \text{time constant of 10 samples (half-life} \approx 7 \text{ samples)}$$

Strengths: Simple, fast, responsive to changes.

Limitations: Single number — cannot express “confident at 2%” vs “uncertain, somewhere between 0% and 10%”. This inability to express uncertainty is why the old system needed three stacked mechanisms (EWMA + Beta margin + PI controller) to avoid under-provisioning.

7.3 Beta-Binomial Posterior — Uncertainty Quantification

The Beta distribution is the conjugate prior for Binomial observations:

Prior: Beta(a, b) (a = received, b = lost)

Update: a' = a x decay + received

b' = b x decay + lost

Posterior: Beta(a', b')

Mean loss rate: b' / (a' + b')

Variance: a'b' / ((a'+b')² (a'+b'+1))

Upper quantile: beta_quantile(b', a', confidence)

The decay factor (0.995) causes old observations to fade, allowing adaptation. This gives a half-life of approximately 138 observations ($\ln(0.5)/\ln(0.995) \approx 138$). The value is not sensitive — decay factors between 0.99 and 0.999 produce similar steady-state behavior. The choice trades off adaptation speed (lower decay = faster forgetting) against estimation stability (higher decay = smoother estimates).

7.6 ACK Loss and Self-Healing

The feedback channel (receiver $\rightarrow$ sender) may also be lossy. If ACKs get lost, the sender lacks up-to-date knowledge of what the receiver has.

Self-healing property: Unlike a NACK-based system where a lost NACK means the sender never learns about the loss, an ACK-based system is inherently self-healing:

- If an ACK is lost, the sender simply does not advance its knowledge of receiver state. Symbols remain in the retransmit buffer.
- After T_{retx} elapses without acknowledgment, the sender retransmits — this is correct behavior whether the original was lost or just the ACK was lost.
- If the original arrived but the ACK was lost, the receiver gets a duplicate (harmless — deduplicated by sequence number) and sends another ACK.
- Eventually an ACK gets through, and the sender's state catches up.

ACK lost scenario:

```
Sender:    [S1] [S2] ... wait  $T_{\text{retx}}$  ... [S1'] ... receives ACK ... done
                                     ↑
                                     retransmit (safe: either original
                                     or ACK was lost, both handled)
```

No separate mechanism needed — the retransmit timeout handles both cases.

This eliminates the need for RX path loss estimation, NackAck echo, and NACK effectiveness tracking. The system is simpler and more robust.

7.7 Estimation Error and Overhead

Estimation error directly maps to overhead:

```
If  $e_{\text{hat}} > e_{\text{true}}$ : over-provisioning  $\rightarrow$  wasted bandwidth
If  $e_{\text{hat}} < e_{\text{true}}$ : under-provisioning  $\rightarrow$  more ARQ latency events
```

Overhead gap = $(e_{\text{hat}} - e_{\text{true}}) / e_{\text{true}}$

BOCD minimizes this gap by adapting the estimation confidence to the regime: - Steady state: $\hat{\epsilon} \approx \epsilon_{\text{true}}$ (tight posterior) - Transition: $\hat{\epsilon} > \epsilon_{\text{true}}$ (conservative, correct behavior)

8. The Optimization Problem

8.1 Formal Statement

```
minimize:     $r = A/q$                                 (correction rate = bandwidth cost)

subject to:   $e \times (1 - P_{\text{fec}}(A, q)) \leq \delta$     (tail latency constraint)

where:        $\tau(t) = A \times (1-q)^t$                 (taper function)
              $P_{\text{fec}}$  depends on  $A, q, e, W$           (FEC recovery probability)
```

Input: δ (tail latency target, from protocol hint)

Output: A^* (optimal taper amplitude), $r^* = A^*/q$ (optimal correction rate)

Note on the constraint: Section 8.2 actually derives r^* from the stricter per-window surrogate $P(\text{repairs} < K) \leq \delta$ rather than $e \times (1 - P_{\text{fec}}) \leq \delta$ — see the “Which δ is this?” note there.

8.2 Corrected Model (canonical)

Let's reconsider. The taper generates correction symbols at known positions. For a window of size W , the taper generates exactly $r \times W$ correction symbols total. The question is: given that a source symbol is lost, how many correction symbols covering it will arrive at the receiver?

In a sliding window code, repair symbols are linear combinations of all source symbols in the window. A repair at offset t from source s covers s as long as s is still in the window ($t < W$).

Number of correction symbols generated while s is in the window:

$$N_{\text{correction}} = \sum_{t=0}^{W-1} \tau(t)$$

Of these, each survives independently with probability $(1-\epsilon)$. Expected arrivals:

$$R = N_{\text{correction}} \times (1-\epsilon) = (A/q) \times (1 - (1-q)^W) \times (1-\epsilon)$$

For the decoder to recover s , we need at least 1 linearly independent repair (with systematic code, if all other source symbols arrived, one repair suffices).

But the actual requirement is more nuanced: we need as many repair symbols as there are lost source symbols in the window. If k symbols are lost in the window, we need k repair arrivals.

Average case: In a window of size W with loss rate ϵ , expected losses = ϵW . Expected repair arrivals = $r \times W \times (1-\epsilon)$. For recovery: $r \times W \times (1-\epsilon) \geq \epsilon W$, giving $r \geq \epsilon/(1-\epsilon) = r_{\text{IT}}$. This confirms the IT minimum.

Tail case: We need recovery even when more symbols are lost than average. The number of losses in a window follows a distribution determined by the GE model. The tail latency constraint requires recovery at the δ -th percentile.

Let K be the number of lost symbols in a window. For the GE model:

$P(K = k)$ depends on the burst structure
For large W : $K \approx e \times W$ with variance determined by burstiness

For recovery: number of arriving repairs $\geq K$.

Repairs arrive = $\text{Binomial}(r \times W, 1-\epsilon)$, approximately $\text{Normal}(rW(1-\epsilon), rW(1-\epsilon)\epsilon)$.

Constraint: $P(\text{repairs} < K) \leq \delta$.

Which δ is this? Section 6.3 defines δ as $P(\text{late delivery}) = e \times (1 - P_{\text{fec}})$. Since a symbol can only be late if it was lost first, the constraint on the window is $P(\text{repairs} < K) \leq \delta/e$ — the canonical form used in Section 8.4, giving $z_{\{\delta/e\}}$ in the margin. Two consequences:

1. **Continuity.** As the channel improves relative to the target ($e \rightarrow \delta$), $z_{\{\delta/e\}}$ falls through zero and the required rate decreases continuously to $r^* = 0$ — pure ARQ already meets the tail target. No cutoff rule is needed; the $\delta = e$ boundary behavior of Section 11.3 emerges from the closed form.
2. **A conservative variant exists.** Imposing $P(\text{repairs} < K) \leq \delta$ directly (z_{δ} instead of $z_{\{\delta/e\}}$) is stricter — it partially compensates for the size-bias of conditioning on “this symbol was lost” (a window known to contain a loss has more losses than average). At WiFi ($e = 0.025$, $\delta = 1e-4$): $z_{\{\delta/e\}} = 2.65$ vs $z_{\delta} = 3.72$. Deployments wanting extra safety can use z_{δ} ; for exactness, use the transfer-matrix computation (Appendix D, item 4) instead of either approximation.

Using normal approximation:

$$P(\text{repairs} < K) \approx P(\text{Normal}(rW(1-e), rW(1-e)e) < K)$$

This requires the δ -quantile of repairs to exceed the $(1-\delta)$ -quantile of losses. The algebra gives:

$$r \geq e/(1-e) + z_{\delta} \times \sqrt{e/(W(1-e))}$$

where z_{δ} is the standard normal quantile [Abramowitz1964] for probability δ .

The explicit P_{fec} formula. Given correction rate r , loss rate e , burst variance $s2_{\text{burst}}$, and window size W :

Losses in window: $K \sim \text{Normal}(We, We(1-e)s2_{\text{burst}})$
Surviving corrections: $C \sim \text{Normal}(rW(1-e), rWe(1-e))$

$$P_{\text{fec}} = P(C \geq K) = \Phi(z)$$

$$\text{where } z = \sqrt{W} \times (r(1-e) - e) / \sqrt{e(1-e)(r + s2_{\text{burst}})}$$

At the IT minimum ($r = e/(1-e)$): $z = 0$, $P_{\text{fec}} = 0.5$ (coin flip — half the time corrections suffice, half the time they don't). The margin term in Section 8.4's r^* pushes z positive, giving $P_{\text{fec}} > 0.5$.

Note: The r^* formula from Section 8.4 is a first-order approximation of inverting this P_{fec} formula. It drops r from the variance denominator (assumes $r \ll s2_{\text{burst}}$). For practical r values (0.05-0.25) this is accurate. For exact computation, invert the P_{fec} formula numerically.

This normal approximation is valid when $W \gg B$ (window much larger than mean burst) — which covers all practical scenarios ($W=50$, $B=2-3$). For edge cases where $W \approx B$, the exact GE distribution is computable via transfer matrix dynamic programming in $O(W^2)$. See Appendix D item 4.

8.3 Burst Variance Correction

The normal approximation in 8.2 assumes iid losses (Binomial variance). On a GE channel, losses are correlated — bursts inflate the variance.

The GE autocorrelation decays with eigenvalue $(1-p-q)$. The variance of losses in a window of size W is:

$$\text{Var}_{\text{iid}}(K) = W \times e \times (1-e) \quad (\text{independent losses})$$

$$\text{Var}_{\text{GE}}(K) = W \times e \times (1-e) \times \sigma^2_{\text{burst}} \quad (\text{burst-correlated losses})$$

$$\sigma^2_{\text{burst}} = 1 + 2(1-p-q)/(p+q) \quad (\text{variance inflation factor})$$

Scenario	$p+q$	σ^2_{burst}	Meaning
DC	0.5005	3.0	3× wider variance than iid assumption
WiFi	0.513	2.9	similar
LTE	0.42	3.8	significant inflation
Satellite	0.33	5.1	iid approximation seriously wrong

We compute σ^2_{burst} directly from the GE estimator's $\hat{p}$ and $\hat{q}$. Degenerate estimates need care: when the estimator has observed no Bad-state transitions (very clean channels — the decayed counters empty out), $\hat{q} = 0$ is a NO-DATA sentinel, not a measurement of infinite bursts. Treating it as a measurement makes $\sigma^2_{\text{burst}} = 1 + 2(1-\hat{p})/\hat{p} \approx 2/\hat{p}$ explode (≈ 4000 at $\varepsilon = 0.1\%$) and massively over-provisions exactly the cleanest links. No-data must map to the iid default $\sigma^2 = 1$.

Known limitation — repair survival is treated as i.i.d. The model inflates the variance of the loss count K by σ^2_{burst} but keeps the repair count $C \sim \text{Binomial}(rW, 1-\varepsilon)$ with independent survival. Repairs cross the same GE channel: a burst that inflates K simultaneously kills interleaved repairs, so $\text{Var}(C)$ is also burst-inflated and $\text{Cov}(K, C) < 0$. Both effects widen $\text{Var}(K - C)$ beyond what the formula assumes, making P_{fec} optimistic in exactly the bursty regime σ^2_{burst} is meant to protect. Monte Carlo validation shows the normal approximation diverging by up to $\sim 12\%$ on high-loss/long-burst channels (LTE-like). For implementation-grade precision, use the exact $O(W^2)$ transfer-matrix computation (Section 8.7), which captures loss/repair correlation exactly.

8.4 The Corrected Optimal Correction Rate

$$r^* = \max(0, \underbrace{\varepsilon/(1-\varepsilon)}_{\text{IT minimum}} + \underbrace{z_{\{\delta/\varepsilon\}} \times \sqrt{\varepsilon \times s2_{\text{burst}} / (W \times (1-\varepsilon))}}_{\substack{\text{tail margin} \\ \text{(accounts for burst correlation)}}})$$

$$s2_{\text{burst}} = 1 + 2(1-p-q)/(p+q)$$

$$z_{\{\delta/\varepsilon\}} = \text{standard normal quantile at } (1 - \delta/\varepsilon)$$

Continuity — no cutoffs, the boundary emerges from the closed form. The quantile is taken at $1 - \delta/\varepsilon$: the margin depends on how tight the target is RELATIVE to the channel. When $\delta \ll \varepsilon$ (target much tighter than the loss rate), z is large and the margin dominates. As the channel improves toward the target ($\varepsilon \rightarrow \delta$), z falls through zero, r^* drops below the IT minimum, and reaches 0 continuously — at which point pure ARQ meets the tail target with no FEC at all. The $\max(0, \cdot)$ is the physical floor (repair counts cannot be negative), not a policy cutoff; there is no mode switch anywhere on the path from “heavy FEC” to “pure ARQ”.

Properties: - $r^*(\delta, \varepsilon)$ is continuous — it decreases to 0 as $\delta \rightarrow \varepsilon$ (Section 11.3 boundary) with no branch or threshold; all regime behavior comes from one closed-form expression - IT minimum $\varepsilon/(1-\varepsilon)$ is the dominant term when the margin is active - Tail margin scales as $1/\sqrt{W}$ — larger windows need proportionally less margin - $z_{\{\delta/\varepsilon\}}$ controls the margin: tighter δ RELATIVE to $\varepsilon \rightarrow$ larger $z \rightarrow$ more margin - σ^2_{burst} amplifies the margin for bursty channels (large for small $p+q$) - On strongly bursty channels this closed form UNDER-provisions the tail (Gaussian tail + ignored loss/repair correlation) — see Section 8.7 for the exact computation and the size of the gap

Note: This formula uses the raw loss rate ε . For the canonical production formula including codec overhead, replace ε with $\varepsilon_{\text{hat}} = \varepsilon + \varepsilon_{\text{codec}} \times (1-(1-\varepsilon)^W)$ from Section 9.2. The codec-adjusted version accounts for decoder invocation probability on systematic codes.

8.5 Worked Examples

Using $z_{\{\delta/\varepsilon\}} = \Phi^{-1}(1 - \delta/\varepsilon)$ — the margin depends on how tight the target is relative to the channel loss rate.

The margin term is: $z_{\{\delta/\epsilon\}} \times \sqrt{(\epsilon \times \sigma^2_{\text{burst}} / (W \times (1-\epsilon)))}$

DC ($\epsilon=0.001$, $W=50$, $\sigma^2_{\text{burst}}=3.0$):

Bulk ($\delta=1e-2$): $\delta \geq \epsilon \rightarrow r^* = 0$ (pure ARQ already meets the target)
 Auto ($\delta=1e-4$): $r^* = 0.1\% + 1.28 \times 0.78\% = 0.1\% + 1.0\% = 1.1\%$
 Realtime ($\delta=1e-6$): $r^* = 0.1\% + 3.09 \times 0.78\% = 0.1\% + 2.4\% = 2.5\%$

WiFi ($\epsilon=0.025$, $W=50$, $\sigma^2_{\text{burst}}=2.9$):

Bulk ($\delta=1e-2$): $r^* = 2.6\% + 0.25 \times 3.86\% = 2.6\% + 1.0\% = 3.5\%$
 Auto ($\delta=1e-4$): $r^* = 2.6\% + 2.65 \times 3.86\% = 2.6\% + 10.2\% = 12.8\%$
 Realtime ($\delta=1e-6$): $r^* = 2.6\% + 3.94 \times 3.86\% = 2.6\% + 15.2\% = 17.8\%$

Satellite ($\epsilon=0.09$, $W=50$, $\sigma^2_{\text{burst}}=5.1$):

Bulk ($\delta=1e-2$): $r^* = 9.9\% + 1.22 \times 10.0\% = 9.9\% + 12.3\% = 22.2\%$
 Auto ($\delta=1e-4$): $r^* = 9.9\% + 3.06 \times 10.0\% = 9.9\% + 30.7\% = 40.6\%$
 Realtime ($\delta=1e-6$): $r^* = 9.9\% + 4.24 \times 10.0\% = 9.9\% + 42.6\% = 52.5\%$

Note the emergent behavior across rows. At DC, a Bulk target is looser than the channel loss itself, so FEC vanishes entirely — ARQ suffices. On satellite the SAME $\delta = 1e-2$ sits 9× below ϵ , so even Bulk needs a 12% margin. The margin responds to the ratio δ/ϵ , not to δ alone — one protocol hint adapts continuously across channels. The σ^2_{burst} factor still amplifies the margin on bursty channels (satellite’s unit margin is 10% vs WiFi’s 3.9% per unit z).

8.6 Three-Variable Optimization

Section 8.4 solves for r^* given δ (with $\rho=100\%$). Here we generalize to all three modes of the bandwidth/latency/reliability triangle.

Taper with cutoff

When $\rho < 100\%$, the taper is truncated at T_{cut} :

$$\begin{aligned} \tau(t) &= A \times (1-q)^t & \text{for } t \leq T_{\text{cut}} \\ \tau(t) &= 0 & \text{for } t > T_{\text{cut}} \end{aligned}$$

Total correction rate with cutoff:

$$r = A \times \sum_{t=0}^{T_{\text{cut}}} (1-q)^t = A \times (1 - (1-q)^{T_{\text{cut}}+1}) / q$$

For $T_{\text{cut}} = \infty$ ($\rho = 100\%$): reduces to $r = A/q$ (Section 4.3).

Mode 1: Given $(\delta, \rho) \rightarrow$ compute r

Fix tail latency target δ and reliability ρ . Compute minimum bandwidth r^* .

Step 1: From ρ , find T_{cut} . The reliability $\rho = P(\text{recovered within } T_{\text{cut}})$. Using the corrected model (Section 8.4):

$$T_{\text{cut}} \text{ such that: } e \times (1 - P_{\text{fec}}(T_{\text{cut}})) \times (1 - P_{\text{arq}}(T_{\text{cut}})) = 1 - \rho$$

For $\rho = 100\%$: $T_{\text{cut}} = \infty$ (no finite solution — see Section 6.4). For $\rho < 100\%$: solve via binary search. $P(\text{recovered by } T_{\text{cut}})$ is monotone increasing in T_{cut} (more time = more corrections = higher recovery):

Algorithm: find T_{cut} from ρ

$P_{\text{fec}} = \text{Phi}(\text{sqrt}(W) \times (r(1-e)-e) / \text{sqrt}(e(1-e)(r+s2_{\text{burst}})))$ (Section 8.2, time-independent)

```

lo = 0
hi = W x 10                                (upper bound: many window lengths)
while hi - lo > tolerance:
    mid = (lo + hi) / 2
    corrections_by_mid = r x (1 - (1-q)^(mid+1))    (taper integral up to mid)
    P_arq(mid) = min(corrections_by_mid / (e/(1-e)), 1) (fraction of needed
corrections available)
    P_recovered(mid) = 1 - e x (1-P_fec) x (1-P_arq(mid))
    if P_recovered(mid) < ρ:
        lo = mid                                (need more time)
    else:
        hi = mid                                (enough time)
T_cut = hi

```

Convergence is guaranteed because $P(\text{recovered})$ is monotone in T_{cut} . Typically converges in ~20 iterations ($\log_2$ of search range).

Step 2: From δ , find A using the tail latency constraint (Section 8.4):

A^* such that: $e \times (1 - P_{\text{fec}}(A^*)) \leq \delta$ (among delivered symbols)

Step 3: Compute $r^* = A^* \times (1 - (1-q)^{\{T_{\text{cut}}+1\}}) / q$.

Special case $\rho = 100\%$: $T_{\text{cut}} = \infty$, $r^* = A^*/q = \varepsilon/(1-\varepsilon) + z_{\delta}\sqrt{(\dots)}$. This is the formula from Section 8.4.

Mode 2: Given $(r, \rho) \rightarrow \text{compute } \delta$

Fix bandwidth r and reliability ρ . Compute resulting tail latency δ .

```

From ρ: find T_cut (same as Mode 1, Step 1)
From r and T_cut: A = r x q / (1 - (1-q)^(T_cut+1))
From r and W: P_fec = Phi(sqrt(W) x (r(1-e)-e) / sqrt(e(1-e)(r+s2_burst)))
(Section 8.2)
Result: δ = e x (1 - P_fec) x P_arq / ρ

```

Mode 3: Given $(r, \delta) \rightarrow \text{compute } \rho$

Fix bandwidth r and tail latency δ . Compute resulting reliability ρ .

```

From r: A = r x q / (1 - (1-q)^(T_cut+1))    (depends on T_cut)
From δ: determine how much of the taper is "on-time" vs "late"
From A and the taper integral: ρ = total recovery probability within T_cut

```

With r fixed, the implied tail latency is monotone in ρ : writing $F = e(1-P_{\text{fec}})$, Mode 2 gives $\delta(\rho) = F \times P_{\text{arq}}(\rho) / \rho = 1 - (1-F)/\rho$, which increases with ρ (demanding higher reliability forces more of the FEC misses to be recovered late by ARQ instead of dropped). So binary search on ρ finds the largest reliability whose implied lateness stays within the δ budget:

Algorithm: find ρ given (r, δ)

$P_{\text{fec}} = \text{Phi}(\text{sqrt}(W) \times (r(1-e)-e) / \text{sqrt}(e(1-e)(r+s2_{\text{burst}})))$ (Section 8.2)

$lo = 0.5$

$hi = 1 - 1e-12$

while $hi - lo > \text{tolerance}$:

$mid = (lo + hi) / 2$ (candidate ρ)

$P_{\text{arq}}(mid) = \text{clamp}(1 - (1-mid)/(e \times (1-P_{\text{fec}})), 0, 1)$ (Section 6.3)

$\delta(mid) = e \times (1-P_{\text{fec}}) \times P_{\text{arq}}(mid) / mid$ (Mode 2 with $\rho = mid$)

 if $\delta(mid) > \delta$:

$hi = mid$ (candidate ρ implies more lateness than allowed)

 else:

$lo = mid$ (budget allows higher ρ)

$\rho = lo$

$T_{\text{cut}} = \text{find } T_{\text{cut}} \text{ from } \rho$ (Mode 1, Step 1)

Convergence guaranteed by monotonicity. The resulting ρ is the maximum reliability achievable within the bandwidth budget r at tail latency δ .

Worked examples

Example 1: Bulk file transfer (WiFi, $\epsilon=0.025$)

Fix: $\rho = 100\%$, minimize r

Compute: δ (tail latency)

$r^* = 2.6\% + z_{\{\delta/\epsilon\}} \times 3.9\%$ (from Section 8.5, WiFi row)

At minimum $r = r_{\text{IT}} = 2.6\%$: $P_{\text{fec}} = 0.5$ (Section 8.2: $z = 0$), so

$\delta = e \times (1-P_{\text{fec}}) = 1.25\%$ – half the lost symbols go to ARQ

Tail latency $\approx T_{\text{retx}} + \text{RTT}/2$ for those symbols

For $\text{RTT} = 50\text{ms}$: $L_{\text{arq}} \approx 100\text{ms}$ for 1.25% of symbols

To get $\delta = 1e-2$: $r^* = 3.5\%$ (Section 8.5, WiFi Bulk row)

Example 2: VoIP (WiFi, $\epsilon=0.025$)

Fix: $\delta = 150\text{ms}$ budget $\rightarrow T_{\text{cut}} = 150\text{ms} / \text{symbol_time}$

$r = 5\%$ (codec + small overhead)

Compute: ρ (reliability)

With $r = 5\%$, the correction budget gives (Section 8.2, $W=50$, $\sigma^2=2.9$):

- $P_{\text{fec}} \approx 0.73$ (73% of lost symbols FEC-recovered, no added latency)

- Remaining 0.68% of all symbols go to ARQ; one retransmit round

($L_{\text{arq}} \approx 100\text{ms}$) fits the 150ms budget, succeeding w.p. $\approx 1-e$

- $P(\text{miss deadline}) \approx e \times (1-P_{\text{fec}}) \times e \approx 0.02\% \rightarrow \rho(150\text{ms}) \approx 99.98\%$

The 150ms budget is generous at $\text{RTT} = 50\text{ms}$: reliability is limited by double losses (symbol AND its retransmit both lost), not by the correction budget. The VoIP codec conceals the residual frame loss.

Example 3: Live video (WiFi, $\epsilon=0.025$)

Fix: $\delta = 33\text{ms}$ (one frame at 30fps), $p = 99.9\%$
 Compute: r (bandwidth)

Need 99.9% of symbols delivered within 33ms.
 T_{cut} determined by $p = 99.9\%$: $T_{\text{cut}} \approx 3 \times \text{RTT}$
 A determined by δ : need $P(\text{recovery within } 33\text{ms}) \geq 0.999$
 $33\text{ms} < L_{\text{arq}}$, so recovery must be FEC – $r^* \approx 13\text{-}18\%$
 (between the Auto and Realtime rows of Section 8.5)

Example 4: Gaming (LTE, $\epsilon=0.05$)

Fix: $\delta = 20\text{ms}$ (tight), $p = 99\%$ (1% loss acceptable)
 Compute: r (bandwidth)

Very tight latency + moderate reliability $\rightarrow$ aggressive FEC
 $T_{\text{cut}} \approx 2 \times \text{RTT}$ (short: accept 1% loss)
 $r^* \approx 11\%$ ($\delta/\epsilon = 0.2 \rightarrow z = 0.84 \rightarrow \text{margin} \approx 5.3\%$ over the 5.3% IT minimum; the $p = 99\%$ cutoff trims the taper tail further)

Example 5: Sensor telemetry (Satellite, $\epsilon=0.09$)

Fix: $r = 5\%$ (minimal bandwidth), $p = 95\%$
 Compute: δ (tail latency)

Low bandwidth budget + high loss $\rightarrow$ many symbols go to ARQ
 T_{cut} determined by $p = 95\%$
 $\delta \approx 15\%$ (15% of delivered symbols arrive late)
 Acceptable for periodic sensor readings.

8.7 Exact P_{fec} via Transfer-Matrix DP

Both gaps in the normal model – burst-inflated repair variance and the negative K–C correlation (Section 8.3) – vanish if we compute the joint distribution of losses and surviving repairs EXACTLY, by walking the GE chain across the interleaved wire sequence.

Setup. A window of W source symbols with $R = \text{round}(r \times W)$ repairs interleaved evenly: $N = W + R$ wire slots with fixed types $T_i \in \{\text{source}, \text{repair}\}$ (slot i is a repair iff $\lfloor (i+1)R/N \rfloor > \lfloor iR/N \rfloor$). The channel is the two-state GE chain started from its stationary distribution; a symbol is lost iff the chain is Bad at its slot. Because ONE chain walks across both slot types, burst-correlated source losses, burst-correlated repair erasures, and the negative correlation between them are all captured.

Recursion. Track the running deficit $D = (\# \text{source losses}) - (\# \text{surviving repairs})$. FEC succeeds for the window iff $D \leq 0$ at the end – the same criterion as Section 8.2, applied to the exact joint distribution. With $f_i(x, d) = P(\text{chain in state } x, \text{ deficit } d \text{ after slot } i)$:

$$f_0(G, 0) = \pi_G = q/(p+q), \quad f_0(B, 0) = \pi_B = p/(p+q)$$

step to slot $i+1$ of type T :

$$\begin{aligned} \text{mass arriving in state } x': & \sum_x f_i(x, d) \times P(x \rightarrow x') \\ d' = d + 1 & \quad \text{if } T = \text{source and } x' = B \quad (\text{source lost}) \\ d' = d - 1 & \quad \text{if } T = \text{repair and } x' = G \quad (\text{repair survives}) \\ d' = d & \quad \text{otherwise} \end{aligned}$$

$$P_{\text{fec}} = 1 - \sum_{\{d > 0\}} \sum_x f_N(x, d)$$

State space $2 \times (W+R+1)$ over N slots $\rightarrow O(W^2)$ work ($\approx 6,000$ operations at $W = 50, r = 0.1$)
 – trivially cheap. Validation: on a memoryless channel ($p + q = 1$) the DP reproduces the

independent-Binomial reference to machine precision, and against Monte Carlo it agrees to sampling error (< 0.002 at 20k trials), where the normal approximation errs by $\approx 1\text{--}2\%$:

Scenario (W=50)	r	exact	normal(8.2)	error
WiFi (p=.013, q=.5)	0.10	0.9522	0.9695	0.017
LTE (p=.02, q=.4)	0.12	0.8868	0.8694	0.017
Sat (p=.03, q=.3)	0.25	0.9180	0.9272	0.009

($R = \text{round}(r \times W)$, half rounding away from zero — e.g. Sat: $R = 13$.)

****Exact r^* **** $P_{\text{fail}}(r) = 1 - P_{\text{fec}}(r)$ decreases in r (up to the $1/W$ rounding of R), so binary search yields the exact minimum rate for $P_{\text{fail}} \leq \delta$. The tail is where the normal approximation is weakest, and the effect is material:

$\delta = 1e-2$, $W = 50$	r^*_{exact}	$r^*_{\text{normal}} (8.4)$
WiFi	0.170	0.116
LTE	0.270	0.193
Satellite	0.450	0.334

The closed-form r^* UNDER-provisions by $\sim 30\text{--}50\%$ of itself on bursty channels: the K–C correlation widens $\text{Var}(K - C)$ beyond the model, and the true tail of $K - C$ is heavier than Gaussian. The closed form remains valuable for insight and cheap incremental updates; rate selection on strongly bursty channels should use the exact computation (implemented as `p_fec_exact / compute_r_star_exact` in `raptorpath-math`).

Caveats. Codec overhead is not modeled (apply $\epsilon_{\text{codec_eff}}$ from Section 9.2 on top). The success criterion inherits Section 8.2’s per-window block view of the sliding-window code. R is rounded to an integer, so r^* is resolved in steps of $1/W$.

9. Codec Overhead Integration

9.1 Decoder Invocation Probability

For systematic codes (METTLE, RLC, RaptorQ), the decoder is only invoked when at least one source symbol in the window is lost:

$$P(\text{decoder invoked}) = 1 - (1-e)^W$$

Scenario	ϵ	W=50	W=200
DC	0.001	4.9%	18.1%
WiFi	0.025	72.1%	99.4%
Satellite	0.09	99.1%	$\approx 100\%$

9.2 Effective Codec Overhead

The codec overhead (ϵ_{codec}) represents extra symbols the decoder needs beyond the information-theoretic minimum:

Backend	ϵ_{codec}	Reason
Reed-Solomon	0.0%	MDS: any k of n suffices
RLC	0.4%	Near-MDS: rare rank deficiency [RFC8681]

Backend	ϵ_{codec}	Reason
RaptorQ	1.0%	Fountain code convergence overhead [RFC6330]
METTLE	15.0%	Sparse random matrix: needs more eqns [Yu2025]
Streaming	0.0%	Rate-optimal by construction [Badr2017]

Weighted by decoder invocation probability:

$$e_{\text{codec_eff}} = e_{\text{codec}} \times P(\text{decoder invoked}) = e_{\text{codec}} \times (1 - (1-e)^W)$$

The corrected correction rate becomes:

$$r^* = \max(0, (e + e_{\text{codec_eff}})/(1-e) + z_{\{\delta/e_{\text{hat}}\}} \times \sqrt{(e + e_{\text{codec_eff}}) \times \sigma^2_{\text{burst}} / (W \times (1-e))})$$

(the same margin structure as Section 8.4, with e replaced by $e_{\text{hat}} = e + e_{\text{codec_eff}}$ — including inside the quantile ratio δ/e_{hat}).

9.3 Impact on METTLE at DC

At DC ($\epsilon=0.1\%$, $W=50$, $\sigma^2=3.0$, Auto $\delta=1e-4$ — under Bulk the DC rate is 0 with or without weighting, so Auto is the informative case):

Without weighting: $\epsilon_{\text{hat}} = 0.1\% + 15\% = 15.1\%$, $z_{\{\delta/\epsilon_{\text{hat}}\}} = 3.21 \rightarrow r^* = 15.1\% + 30.6\% = 45.7\%$. With weighting: $e_{\text{codec_eff}} = 0.15 \times 0.049 = 0.74\%$, $\epsilon_{\text{hat}} = 0.84\%$, $z_{\{\delta/\epsilon_{\text{hat}}\}} = 2.26 \rightarrow r^* = 0.8\% + 5.1\% = 5.9\%$.

The weighting reduces METTLE's DC overhead by $\sim 8\times$.

10. Multi-Protocol Extension

10.1 Per-Symbol Latency Classes

Different traffic types have different δ targets:

Symbol s has latency class $c(s)$ with target $\delta_{\{c(s)\}}$

10.2 Interleave Before vs After FEC

After FEC (separate streams):

Realtime packets $\rightarrow$ [FEC encoder ($\delta=1e-6$)] $\rightarrow$ channel
Bulk packets $\rightarrow$ [FEC encoder ($\delta=1e-2$)] $\rightarrow$ channel

Each stream has its own taper. No correction sharing. Simple.

Before FEC (shared stream):

All packets $\rightarrow$ [mixed FEC encoder] $\rightarrow$ channel

One taper covers everything. Repair symbols are linear combinations of ALL source symbols [RFC8681] — a repair can recover ANY lost symbol regardless of class.

Advantage of shared: repair symbols are fungible. A repair generated “for” a Realtime symbol can recover a Bulk symbol if needed. Total correction budget can be lower than the sum of separate budgets (statistical multiplexing).

Disadvantage of shared: the taper must be sized for the tightest class. If 1% of traffic is Realtime ($\delta=1e-6$) and 99% is Bulk ($\delta=1e-2$), you pay Realtime-level overhead on everything.

10.3 When Shared Wins

Shared FEC is cheaper when the traffic mix is balanced or dominated by the tight class. Separate streams are cheaper when the tight class is a small fraction. The crossover depends on the specific δ values and loss rate.

Decision rule: Compare total correction bandwidth:

$$\begin{aligned} \text{shared_cost} &= r*(e, \min(\delta_c)) && \text{(one encoder, tightest } \delta) \\ \text{separate_cost} &= \sum_c f_c \times r*(e, \delta_c) && \text{(per-class, weighted by fraction } f_c) \end{aligned}$$

Choose whichever is lower.

10.4 Extending the Formula

For shared FEC with per-symbol δ , the constraint becomes:

$$\text{For each class } c: e \times (1 - P_fec) \leq \delta_c$$

Since P_fec is the same for all symbols (shared repair), the binding constraint is the tightest: $\delta_min = \min(\delta_c)$. The formula reduces to the single-class case with $\delta = \delta_min$.

11. Verification

11.1 Simulation Approach

Generate synthetic loss traces from the GE model, apply the taper function, and measure whether the actual tail latency matches the theoretical prediction.

- For each trial:
1. Generate GE loss trace: {lost_1, lost_2, ..., lost_N}
 2. Generate correction schedule from taper: {correction_1, correction_2, ...}
 3. Apply channel loss to both source and correction symbols
 4. For each lost source symbol:
 - a. Check if enough repair symbols arrived (FEC recovery)
 - b. If not, mark as ARQ-recovered (retransmit needed)
 5. Count ARQ events (symbols not FEC-recovered)
 6. Measure: $\text{actual_arq_fraction} = \text{arq_events} / N$

Over many trials:

Verify: $P(\text{actual_arq_fraction} > \delta)$ is small

Verify: mean correction rate $\approx r*$

11.2 Analytical Predictions to Verify

Prediction	Formula	Test
IT minimum dominates at high loss	$r^* \approx \epsilon/(1-\epsilon)$ when W large	Satellite scenario
Tail margin scales as $1/\sqrt{W}$	$r(W=200) < r(W=50)$	Compare window sizes
Taper shape matches GE	Decay rate = $\hat{q}$ from estimator	Compare simulated vs theoretical
Codec overhead weighting	METTLE DC overhead $\sim 0.74\%$ not 15%	DC scenario with METTLE
Protocol hint only affects δ	$r(\text{Realtime}, \delta=1e-6)$ vs $r(\text{Auto}, \delta=1e-4)$ differ only via z_delta	Same estimator, different hints

11.3 Boundary Cases

Case	Expected behavior	Why
$\epsilon = 0$ (no loss)	$r^* = 0$	No correction needed
$\epsilon \rightarrow 1$ (total loss)	$r^* \rightarrow \infty$	Can't recover anything with FEC alone
$\delta = \epsilon$ (every loss to ARQ)	$r^* = 0$	No FEC needed, all ARQ
$\delta \rightarrow 0$ (zero ARQ tolerance)	$r^* \rightarrow \infty$	Must FEC-recover everything
$W = 1$ (no window)	Margin term large	Single-symbol recovery needs more redundancy
$q = 1$ (no burst memory)	$\tau(t) = \text{flat}$	Reduces to iid case

11.4 Connection to Existing Benchmarks

The `bench_suite` already measures `overhead_pct` and `recovery_pct` per scenario. To verify the model:

1. Compute r^* from the formula for each scenario's (ϵ , δ , W)
2. Compare with the `bench_suite`'s measured overhead
3. The gap between r^* and measured overhead = estimation tax + implementation overhead
4. Track this gap across benchmark runs — it should decrease as we improve the implementation

Canonical evaluation suite. The scenario matrix, baseline fidelity ladder (in-process model $\rightarrow$ real TCP/QUIC/MPTCP stacks over netem $\rightarrow$ real links), and the quantitative win conditions that make “surpasses TCP/MPTCP” a falsifiable claim are defined in ADR-0051 (canonical evaluation scenarios). The channel rows there are exactly the Section 2.4 parameterizations, so the model, the simulator, and the benchmarks share one vocabulary. The suite deliberately includes cells where naive FEC loses (clean links, congestion-dominant loss with a competing TCP flow, mid-transfer path outage) — ties are the win condition there.

12. Congestion Control Integration

12.1 Why Delay-Based CC is Required

On lossy links (WiFi, LTE, satellite), loss-based CC (NewReno, CUBIC) is catastrophic. Every random channel loss triggers `cwnd` halving, collapsing throughput on exactly the links raptorpath is designed for:

NewReno on 2.5% WiFi loss:

- > `cwnd` halves on every loss event
- > throughput oscillates wildly, average $\ll$ capacity
- > our FEC tries to compensate -> more traffic -> more congestion

Delay-based CC on 2.5% WiFi loss:

- > loss + stable RTT = channel loss -> ignore, let FEC handle it
- > loss + rising RTT = congestion -> reduce rate
- > throughput stays near capacity

Delay-based CC distinguishes congestion (queue buildup = rising RTT) from channel loss (random drops = stable RTT). This is essential for raptorpath.

12.2 QUIC Datagrams Bypass Quinn's CC

Raptorpath sends symbol data as QUIC unreliable datagrams, which bypass Quinn's built-in congestion control (NewReno). Our own CC is the sole rate limiter for data traffic. Quinn's CC only applies to QUIC streams (handshake, reliable control messages — small and infrequent).

This means our CC is solely responsible for not flooding the network.

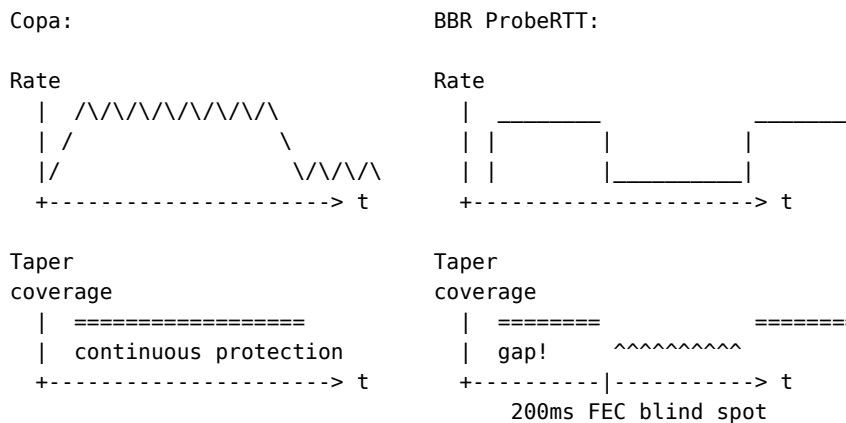
12.3 Copa vs BBR

The current implementation uses BBR (ADR-0019). We recommend migrating to Copa [Copa2018] because of better interaction with the taper function:

Aspect	BBR	Copa
Core idea	measure $\max_bw \times \min_rtt = BDP$	$rate = 1/(d \times dq)$
Queue draining	ProbeRTT: 200ms forced drain every 10s	Natural oscillation
Phases	Startup -> ProbeBw -> ProbeRtt (state machine)	No phases, one formula
Lossy links	RTT-trend check to ignore random loss	Delay-based, same effect
Complexity	~300 lines, multiple edge cases	One rate formula
Taper interaction	ProbeRTT creates FEC protection gap	Smooth, no gaps

Implementation status: Copa is recommended as the target congestion control algorithm. The current codebase uses BBR (ADR-0019). Migration to Copa is a future implementation task. The model and formulas in this paper are CC-agnostic — they work with any delay-based CC that provides a total sending rate.

The critical difference — taper compatibility:



Copa's rate oscillation is smooth and fast (order of RTT). The taper function uses RATIO (corrections per source symbol), which stays constant. Absolute rates oscillate gently, but the EWMA smooths throughput estimates. The taper doesn't notice Copa's oscillation.

BBR's ProbeRTT drops cwnd to 4 symbols for 200ms. During this period:

- Source rate drops to nearly zero
- Correction rate drops proportionally (ratio constant, absolute count tanks)
- Source symbols sent during ProbeRTT get almost no FEC protection
- If a burst loss hits

during ProbeRTT, recovery is severely degraded - After ProbeRTT exits: data burst -> potential queue buildup

12.4 Copa's Rate Formula

Copa targets a queue occupancy that balances throughput and delay:

$$\text{rate} = 1 / (\text{d_copa} \times \text{dq})$$

where:

d_copa = Copa parameter (controls target queue depth; default 0.5)
 dq = queuing_delay = $\text{RTT_current} - \text{RTT_min}$

When dq is small (queue empty): rate is high -> fill the pipe. When dq is large (queue building): rate drops -> drain the queue.

This naturally oscillates: send fast -> queue builds -> send slow -> queue drains -> send fast again. The oscillation frequency is $\sim 1/\text{RTT}$ and amplitude depends on d_copa. No periodic forced drain phase needed.

Copa's d_copa is NOT the same as our tail latency target δ . Copa's d_copa controls the target queue depth at the bottleneck ($1/\text{d_copa}$ packets). Our δ controls the tail probability of late delivery. They are independent parameters that happen to use similar notation in their respective papers.

d_copa = 0.5 (from the Copa paper [Copa2018]) nominally targets a queue of 2 packets. That does NOT mean the realized cost is ~ 1 packet of delay: at high utilization the sender oscillates around the controller's queue target, so the median delivery latency carries a standing queue on the order of that target — not of a single packet. Measured in the gate simulation (100 Mbps WiFi under sustained load), the tolerated standing queue adds ~ 9 ms to the median latency versus an idle baseline (p50 ~ 17.6 ms loaded vs ~ 8.1 ms idle) — a dominant latency term, not a negligible one. The protocol hint must therefore set the delay target too, not only the FEC rate: Realtime $\rightarrow$ a near-empty queue target (accepting some utilization loss), Bulk $\rightarrow$ a deeper queue target for better link utilization, and Auto keeps the d_copa = 0.5-style default. A continuous mapping from the tail latency budget to d_copa is the natural refinement; a coarse per-hint target already recovers most of the standing-queue latency.

min_rtt estimation: Copa uses the minimum observed RTT in a 10-second sliding window (same duration as BBR's min_rtt window). Copa's natural rate oscillation periodically reduces the queue to near-empty, refreshing the minimum within this window. If min_rtt has not been refreshed for an extended period (> 20 s), the system can force a brief rate reduction as a fallback — though this is rarely needed because Copa's oscillation provides natural refreshing.

Production implementation notes (P7 port). The production scheduler implements the Copa-lite scheme exactly as proven in the gate driver (windowed-min queue signal, hint-coupled queue target $x\{1.08, 1.125, 1.25\}$, two-speed ramp $x1.5+1$ then $+2/x0.92$, per-SRTT update cadence, token-bucket pacing at cwnd/SRTT with burst $\max(10, \text{cwnd}/8)$), with these honest deviations in constants and mechanics:

- **Propagation floor** = min RTT sample over the 10 s sliding window (the driver used the lifetime min because its trials were shorter than any sane window; the 10 s window is the faithful production analogue and self-heals after a route change).

- **dq clamp at 0.1 ms**, applied to BOTH the queuing-delay estimate and the backoff threshold ($\text{queue_mult} - 1 \times \text{floor}$). This keeps the closed-form rate $1/(\text{d_copa} \times \text{dq})$ finite on LAN-class floors and makes the backoff comparison continuous there (dq at the clamp cannot exceed a threshold at the same clamp) — no branch cliffs. The rate formula itself is retained as a diagnostic only; the cwnd dynamics are the ramp/backoff scheme.
- **cwnd floor = 8 symbols** (the driver backed off to ≥ 4). An L1 run on a real emulated link (100 Mbit, 10 ms RTT) showed the pre-P7 rate-formula collapse crawling at $\text{cwnd} = 2$; the raised floor guarantees a trickle that keeps RTT samples, and therefore recovery, flowing.
- **Ramp fast-exit**: during the ramp the backoff check also runs per ACK (not only per SRTT), so the exponential phase ends within one feedback message of the first standing-queue evidence.
- **Pacing is symbol-level via a carry queue**: the interleaver's drain is all-or-nothing, so drained symbols land in a per-path carry queue and each pace tick sends only $\text{floor}(\text{tokens})$ symbols; the remainder waits for the next tick ($\text{wakeup} = \text{next-token refill time}$, clamped 0.5-50 ms). The first cut gated at batch granularity and let a whole block overdraft; measured at L1 (C2, Bulk) every 56-symbol block burst serialized ~5.4 ms of self-queue — above Bulk's 2.5 ms backoff threshold — so every block bought a $\times 0.92$ backoff and cwnd pinned just under one block (~34 symbols). Symbol-level pacing removes the self-queue entirely. Carried symbols count toward the TUN-read gate ($\text{in_flight} + \text{carried} \geq \text{cwnd}$), which remains the outer backpressure.
- **Bulk flush timeout 5 ms** (was 50 ms): while the CC gate pauses TUN reads, 64KB block assembly stalls mid-block; a 50 ms flush serialized with the congestion window and clumped the pipeline into ~300 ms ACK bursts at L1. 5 ms bounds the assembly wait well under one C2 RTT.
- **in_flight is a schedule-time budget with time-based release**. Each symbol is charged exactly once when scheduled (covering interleaver + pacing carry + wire) and released by ACK feedback. Because ACKs are best-effort datagrams, a lost ACK strands its release; stranded budget expires after $\max(4 \times \text{SRTT}, 250 \text{ ms})$ (RFC 9002-style time threshold at budget granularity) instead of jamming the TUN gate until a 2 s decay backstop. The second L1 round found the gate cycling at exactly that 2 s cadence (~30 KB/s): the send path was charging the budget a SECOND time at wire time, leaking +1 per symbol. Pacing tokens, not the gate, remain the wire-rate limiter, so an early expiry can only let the encoder run ahead, never the wire.
- **Loss reaction** (the driver has none, by design): a decode FAILURE with the standing queue above target takes the same $\times 0.92$ backoff as the delay signal; a decode failure with an empty queue ends the ramp and steps cwnd down by 1 (FEC under-provision, not congestion). Loss that FEC recovered never touches cwnd.

12.5 CC + Taper: The Complete Architecture

Copa determines: total_rate (symbols/sec on the wire)

Taper determines: r^* (correction symbols per source symbol)

$\text{source_rate} = \text{total_rate} / (1 + r^*)$

$\text{correction_rate} = \text{total_rate} \times r^* / (1 + r^*)$

When channel worsens: e rises $\rightarrow r^*$ rises $\rightarrow \text{source_rate}$ falls

When congestion: dq rises $\rightarrow \text{total_rate}$ falls $\rightarrow$ both fall

When channel clears: e drops $\rightarrow r^*$ drops $\rightarrow \text{source_rate}$ rises

The two controllers are orthogonal: - Copa controls HOW FAST (total symbols per second) - Taper controls HOW MUCH redundancy (correction per source ratio)

Neither needs to know about the other. Copa sees total traffic; taper sees loss rate. They compose naturally.

Bulk operating point. Under the Bulk hint the taper side degenerates by design: the tail target is “late is fine” weighted by completion exposure ($\delta_{\text{bulk}} = \hat{\epsilon} + (0.05 - \hat{\epsilon})\chi$, see Sections 5.3, 14.26), so the continuous r^* is 0 identically mid-stream ($\chi = 0$) and the steady state is pure ARQ — $\text{source_rate} \approx \text{total_rate}$, wire volume at parity with a retransmission transport. FEC reappears only near a known end of stream, where χ ramps to 1 and recovery can no longer overlap sending (Sections 14.25, 14.26).

12.6 ECN as Opportunistic Enhancement

If the network path supports ECN [RFC3168], congestion is signaled by router marking (CE bit) instead of dropping. This provides: - Congestion detection without loss -> even better for delay-based CC - Positive identification: marked = congestion, dropped = channel loss - No need to distinguish via RTT trends (direct signal)

QUIC validates ECN support at connection startup. If supported, use it. If not (common on wireless), fall back to Copa’s delay-based detection.

12.7 Application Back-Pressure

When r^* is so high that $\text{source_rate} = \text{total_rate}/(1+r^*)$ drops below the application’s minimum (e.g., VoIP codec needs 64kbps), the system signals back-pressure: “the channel cannot support your required quality at this loss rate.”

The application must respond: - Reduce quality (lower bitrate codec, lower video resolution) - Accept lower reliability (increase d , decrease p) - Wait for better conditions

This is a resource allocation problem, not a CC problem. The CC works correctly; it’s the application that needs to adapt.

Back-pressure mechanism: The sender stops reading from the source (TUN interface, application socket) when the retransmit buffer reaches `buffer_max`. This causes the kernel’s write buffer to fill, which causes the application’s `write()` to block — identical to TCP behavior when the send buffer is full.

No special API is needed for basic operation: applications using the TUN interface experience standard blocking `write()` semantics. Data is accepted when the channel can handle it and blocked when it can’t.

Optional stats API (for advanced applications): Applications that want finer control can subscribe to channel statistics: - Per-path: ϵ , RTT, throughput, correction rate r , Copa rate - Aggregate: effective reliability ρ , tail latency δ , correction deficit - Events: backpressure start/end, path added/removed, regime change

The API also allows overriding protocol characteristics mid-connection: - Adjust δ (tail latency target) or ρ (reliability target) - Change `T_cut` (affects eviction and gap pruning) - Force path preferences

This API is implementation-specific and not part of the core model.

13. Multi-Path Scheduling

13.1 Why FEC Beats MPTCP

MPTCP (Multi-Path TCP) schedulers are fundamentally limited by **head-of-line (HOL) blocking**: TCP requires in-order delivery, so a packet on the slow path blocks all fast-path packets at the receiver. MPTCP schedulers (round-robin, weighted, BLEST [Ferlin2016]) try to minimize this by avoiding slow paths, but they can't eliminate it.

Our FEC-based model is fundamentally different:

MPTCP:	Raptorpath:
Path A: [P1] [P2] [P5] [P6] Path B: [P3] [P4]	Path A: [S1] [C] [S3] [C] Path B: [S2] [S4] [C]
Receiver must wait for P3 before delivering P4,P5,P6. HOL blocking on slow path.	Decoder needs ANY k of n symbols. Order doesn't matter. No HOL blocking.

The decoder doesn't care WHICH symbols arrive — just HOW MANY. A repair symbol on Path A can recover a lost source on Path B. Source symbols can arrive in any order. The reorder buffer handles sequencing after decode.

13.2 Per-Path Model

Each path i runs independently with its own:

Copa _{i} :	$\text{rate}_i = 1 / (d \times dq_i)$	total sending rate
GE _{i} :	(e_i, p_i, q_i)	loss model
Taper _{i} :	$\tau_i(t) = A_i \times (1 - q_i)^t$	correction density
r _{i} :	correction rate = A_i / q_i	source/correction ratio

All paths share: - **One source stream**: source symbols are distributed across paths - **One retransmit buffer**: any path can retransmit any un-ACKed symbol - **One FEC encoder window**: repair symbols cover the same source window

13.3 Unified Symbol Stream and Interleaving

Each path carries a **unified stream** of source + correction symbols, interleaved at that path's own taper ratio. The interleaving is essential for burst protection — corrections scattered among source symbols survive bursts that wipe out consecutive symbols:

```
Path A (e=0.05, r=0.05): [S][S][S][S][C][S][S][S][S][C]... 5% corrections
Path B (e=0.10, r=0.11): [S][S][C][S][S][C][S][S][C]... 11% corrections

During burst on Path A: [S][X][X][X][C][S]...
                        lost      ^ this correction survives!
                                -> decoder can recover
```

Source and corrections must NOT be separated between paths. If source goes on one path and corrections on another, the source path has no interleaved corrections, and burst protection fails:

```
BAD: source/correction separation
Path A: [S][S][S][S][S][S]... <- burst wipes everything, no protection
Path B: [C][C][C][C][C][C]... <- corrections exist but on wrong path
```

The scheduler distributes the combined stream across paths. Each path independently interleaves at its own ratio.

13.4 Correction Deficit

The **correction deficit** is the total expected corrections still needed across all paths:

$$\text{deficit} = \text{SUM}_{\{s \text{ in un-ACKed}\}} e_s$$

where e_s = channel loss rate of path(s) at the time symbol s was sent

Each source or correction symbol sent on path i adds e_i to the deficit (it might be lost). Each ACKed symbol removes its send-time e_s (confirmed survived).

Why $e/(1-e)$ is a geometric series: When a correction symbol is itself lost, it needs replacement. This creates a chain:

Source lost:	0.30	(30% loss)
Corrections also lost:	$0.30 \times 0.30 = 0.09$	
Replacements also lost:	$0.30^3 = 0.027$	
...		
Total =	$0.30 + 0.09 + 0.027 + \dots = 0.30 / (1 - 0.30) = 0.4286$	

This is why $r_{IT} = e/(1-e)$, not just e . The $(1-e)$ denominator accounts for the infinite chain of correction-of-correction loss. The deficit counter captures this naturally: lost corrections add to the deficit, generating more corrections, until enough survive.

Cross-path correction deficit: When source is on path A (e_A) and corrections go on path B (e_B), the formula becomes:

$$r = e_A / (1 - e_B)$$

This emerges from the deficit dynamics: source adds e_A , each surviving correction on path B removes $(1-e_B)$. Equilibrium: $e_A = r \times (1-e_B)$.

13.5 Effective Delivery Time and Bandwidth

For each path i :

E_i	$= \text{RTT}_i/2 + e_i \times t_{\text{recovery}_i}$	effective delivery time [sec]
B_{eff_i}	$= C_i / (1 + r_i)$	source-carrying capacity [sym/s]
e_{combined}	$= \text{SUM}(C_i \times e_i) / \text{SUM}(C_i)$	throughput-weighted loss [prob]

where $t_{\text{recovery}_i} = P_{\text{fec}_i} \times t_{\text{fec}_i} + (1 - P_{\text{fec}_i}) \times L_{\text{arq}_i}$ is the expected recovery time on path i if the symbol is lost. t_{fec_i} is the FEC recovery time (Section 3.4) and L_{arq_i} is the ARQ recovery latency on that path. The delay spectrum concept [Facenda2022] provides related per-link rate allocation theory for multi-path streaming.

13.6 Scheduler Ratio Adjustment

The scheduler can adjust the per-path source/correction ratio to favor certain paths for source symbols. This is only beneficial for **latency-sensitive** traffic (source symbols are immediately processable, corrections are not — so putting more source on a fast path saves latency):

Default (natural taper):
 Path A (fast, $e=0.05$): [S][S][S][S][C][S][S][S][S][C]... $r=0.05$

Latency-optimized (scheduler shifts source to fast path):
 Path A (fast): [S][S][S][S][S][S][S][S][S][C]... $r'=0.02$
 Path B (slow): [S][C][C][S][C][C][S][C][C]... $r'=0.30$
 (absorbs deficit)

For **bandwidth-optimized** traffic: no adjustment needed. Source and corrections cost the same bandwidth. The natural taper ratio is optimal.

Constraint: Each path must maintain minimum interleaving for burst protection. The scheduler cannot reduce the correction ratio below a floor that leaves the path unprotected during bursts.

13.7 Burst Protection During Ratio Adjustment

When the scheduler reduces a path's correction ratio (more source, fewer corrections), that path has fewer same-path corrections to survive a burst. But burst protection is already handled by two existing mechanisms:

Global correction deficit (Section 13.4): When one path's corrections are reduced, the deficit grows — and other paths absorb it by generating more corrections. The total correction budget across all paths remains matched to the total loss. No per-path floor is needed because the correction budget is global, not per-path.

Cross-path diversity (Section 13.10): Corrections on the slow path survive bursts on the fast path (different channel, different burst pattern). A burst on path A is covered by corrections from path B. $P(\text{both paths burst simultaneously}) = \epsilon_A \times \epsilon_B$ — negligible for independent paths.

No correction floor ($r_{\min}$) is needed. The global deficit and cross-path diversity provide burst protection without any per-path minimum. The taper self-corrects via BOCD if loss increases, and $P_{\text{lost}}(t)$ naturally produces Mehrotra's optimal policy (repair during uncertainty, retransmit after confirmation).

13.8 Interpolated Objective Function

One parameterized objective with weights from the protocol hint:

```

minimize:  $\hat{w}_{\text{lat}} \times \text{SUM}(x_i \times E_i) + \hat{w}_{\text{bw}} \times \text{SUM}(x_i \times r_i)$ 
           latency cost                bandwidth overhead cost

subject to:  $\text{SUM}(x_i) = 1$                 all source distributed
             $x_i \times \text{source\_rate} \leq B_{\text{eff}_i}$  per-path capacity

Realtime:   $w_{\text{lat}} = 1.0, w_{\text{bw}} = 0.0$   minimize latency at any bandwidth cost
Balanced:   $w_{\text{lat}} = 0.5, w_{\text{bw}} = 0.5$   balance latency and bandwidth
Bulk:       $w_{\text{lat}} = 0.0, w_{\text{bw}} = 1.0$   minimize bandwidth waste (overhead)

```

13.9 QoS Priority Cascade

When multiple protocol classes share the same paths, they pick in priority order from tightest to loosest latency requirement:

1. Realtime picks first: lowest E_i paths, up to its source volume
2. Balanced picks next: best remaining path capacity
3. Bulk gets the rest: whatever capacity remains

13.10 Cross-Path Retransmit

The shared retransmit buffer enables recovery across paths. When source symbol S_k was sent on path A and lost, any path's correction slot can retransmit it:

$$P(\text{retransmit on path } j) = P_{\text{lost}}(t_k, e_A)$$

Cross-path diversity: $P(\text{both fail}) = e_A \times e_j$. For $e_A=0.10$, $e_j=0.02$: $P(\text{both fail}) = 0.002$ — 50x improvement over single-path.

No explicit path routing needed. Cross-path retransmit emerges from the shared buffer. Each path independently generates correction symbols from its taper. When a correction slot produces a retransmit (via P_{lost}), it pulls from the shared buffer — which may contain symbols from any path. P_{lost} uses per-path ϵ , so symbols from lossier paths have higher P_{lost} and are retransmitted first.

Work-stealing analogy. Paths with spare Copa capacity (typically low-loss paths whose tapers need fewer corrections) have room to pull retransmits from the shared pool — like idle threads stealing work. High-loss paths are busy with their own corrections. This naturally routes retransmits through reliable paths without explicit selection.

Potential refinement: For latency-sensitive traffic, weighting retransmit pulls toward the lowest-RTT path would reduce recovery time. This follows the same interpolated objective as source scheduling (Section 13.8).

14. Future Directions and Considered Improvements

The following sections document insights discovered through building and testing the interactive visualizer simulation. They represent deeper considerations about how FEC latency, ARQ latency, bandwidth, and the encoder window interact — areas where the current model can be refined in future work.

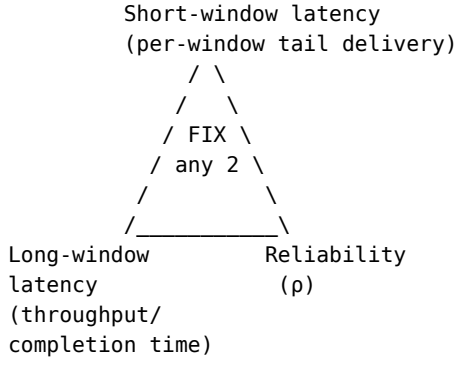
14.1 Reconsidering the Triangle: Bandwidth as Constraint, Not Variable

The current triangle (Section 1.4) treats r (bandwidth overhead) as a variable alongside δ (tail latency) and ρ (reliability). But Copa (Section 12) discovers a fixed link capacity C . We cannot create more bandwidth — we can only choose how to fill the pipe. Every FEC symbol displaces a source symbol:

$$\text{source_rate} = C / (1 + r)$$

More FEC (higher r) → faster per-window recovery → but slower completion

This means “bandwidth overhead” is not an independent dimension — it IS long-window latency at a fixed link rate. The real trade-off space may be:



r is the internal mechanism that trades short-window for long-window latency. The user sets latency targets; the system computes r . This reframing may lead to more intuitive protocol hint mappings.

14.2 FEC Latency Is Not Zero

Section 3.2 states “Latency cost: zero additional (repair arrives at roughly the same time as source).” This is an approximation. FEC repair symbols arrive AFTER the source symbols they cover. A lost symbol S_3 is only recovered when enough repairs covering S_3 have arrived AND the decoder resolves S_3 ’s equation. This takes time $t_{\text{fec}} > 0$.

Per-symbol: FEC can never make a single symbol arrive faster than if it wasn’t lost. The repair always arrives after the source it covers.

Per-window: FEC can recover all losses within a window once enough repairs accumulate. The decode latency depends on the encoder window size W and the correction rate r .

The distinction matters for real-time applications where per-symbol latency is the constraint. For file transfers, only per-window/total completion time matters, and the approximation is acceptable.

14.3 Unified FEC Latency Distribution

For m concurrent losses (a burst of length m), the time until all m are recovered follows a counting process. Every repair covers the entire encoder window (Section 3.2), so the useful-equation arrival rate after a loss is the AGGREGATE correction rate — corrections occupy a fraction $r/(1+r)$ of wire slots, each surviving with probability $(1-\epsilon)$ — not the lost symbol’s own taper density. Using the Poisson approximation (valid when per-slot correction probabilities are small):

$$\lambda(T) = r \times (1-\epsilon) \times T / (1+r) \quad T \text{ in wire slots after the loss}$$

$$P(t_{\text{fec}} \leq T \mid m) = P(\text{Poisson}(\lambda(T)) \geq m) \\ = 1 - \sum_{k=0}^{m-1} e^{-\lambda(T)} \times \lambda(T)^k / k!$$

$\lambda(T)$ grows linearly until the window slides past the lost symbol — after W further source symbols, i.e. $T \approx W(1+r)$ wire slots — at which point $\lambda = rW(1-\epsilon)$, exactly the per-window Binomial mean of Section 8.2.

(An earlier formulation summed only the lost symbol’s own taper, $\lambda(T) = A(1-\epsilon)(1-(1-q)^{(T+1)})/q$, which saturates at $r(1-\epsilon) < 1$ and wrongly predicts that even a single loss is usually unrecoverable — the same per-symbol undercount that invalidates Appendix E.)

This is the regularized incomplete gamma function $Q(m, \lambda(T))$.

Unified across codec types:

- Block codec (RaptorQ, RS): m = losses in block of size K . All m symbols decode simultaneously when the m -th repair arrives.
- Window codec (RLC, Streaming): m = losses in window of size W . Decoder can cascade — recovering one symbol may immediately resolve others via Gaussian elimination. The block CDF is a conservative lower bound (window codecs recover faster due to cascade).

14.4 Ambient FEC and the Pipeline Effect

With a sliding window codec, repair symbols are generated continuously as part of the steady-state $r/(1+r)$ interleaving (Section 5.3). When a loss occurs, there are already repair symbols “in the pipeline” that cover the lost position — they were generated while the lost symbol was in the encoder window.

```
Window:  [S1][S2]...[S50][R1][S51][R2]...[S100][R3]...
           ^         ^
           S50 lost  R1 already covers [S1..S100]
                       → decoder has one equation instantly
```

For a symbol that has been in the window for T_w ticks before loss:

```
λ_total(T) = λ_prior(T_w) + λ_new(T)

λ_prior(T_w) = accumulated surviving repairs from before the loss
              = r × (1-ε) × T_w / (1+r)
λ_new(T)     = new corrections generated after the loss (Section 14.3)
              = r × (1-ε) × T / (1+r)
```

Larger windows accumulate more ambient FEC → faster recovery. This means FEC latency is not just about new repairs generated after the loss — it includes the pipeline of repairs already in flight.

14.5 Optimal Encoder Window Size

The window should be large enough that ambient FEC covers typical bursts. For a burst of length B , we need B surviving repairs in the pipeline:

```
W × r × (1-ε) / (1+r) ≥ B

W_min(B) = B × (1+r) / (r × (1-ε))
```

With $r = \varepsilon/(1-\varepsilon)$: $r(1-\varepsilon) = \varepsilon$ and $1+r = 1/(1-\varepsilon)$, so $W_{\min}(B) = B/(\varepsilon(1-\varepsilon))$. For the mean burst ($B = 1/q$), dropping the small $(1-\varepsilon)$ correction:

```
W_min = 1 / (q × ε × (1-ε)) ≈ 1 / (q × ε)
```

Using the Section 2.4 scenario parameters ($W_{\min} \approx B/\varepsilon$):

Scenario	ε	q	W_min(mean)	B_99	W_min(p99)
WiFi	2.5%	0.50	80	7	~280
LTE	5%	0.40	50	10	~200
Satellite	9%	0.30	37	13	~144

Note the two opposing effects: longer bursts (small q) raise $W_{\min}$ via B , but higher loss ALSO means more repairs per window at $r = \epsilon/(1-\epsilon)$, which lowers it — so satellite (high ϵ) needs a smaller minimum window than WiFi (low ϵ) despite its longer bursts.

Setting W so that $W \times t_{\text{sym}} \approx \text{RTT}$ gives FEC and ARQ roughly equal recovery latency. Below that threshold, FEC is strictly faster than ARQ for all burst lengths up to the pipeline capacity.

14.6 ARQ Latency and the Retransmit Sweet Spot

ARQ latency is well-defined: $L_{\text{arq}} = T_{\text{retx}} + \text{RTT}/2 \approx 1.5 \times \text{RTT}$. But T_{retx} depends on confidence that the symbol is actually lost. $P_{\text{lost}}(t)$ (Section 3.4) models this confidence.

As T_{retx} shrinks (retransmit sooner):

- $\uparrow$ Faster recovery for truly lost symbols
- $\downarrow$ More false-positive retransmits (duplicates)
- $\downarrow$ Duplicates waste bandwidth $\rightarrow$ worse long-window latency

$P_{\text{lost}}(t)$ is exactly this probability/waste trade-off curve. The sweet spot is where $P_{\text{lost}}(T_{\text{retx}})$ is high enough that few retransmits are wasted, but low enough for timely recovery.

For $T_{\text{retx}} < \text{RTT}$: **proactive retransmit** territory. We retransmit before knowing if the symbol was lost. This is bandwidth-expensive but latency-optimal for the individual symbol. Viable only at high ϵ where most symbols are lost anyway ($P_{\text{lost}}(0) = \epsilon$ is already high).

14.7 When FEC Beats ARQ (and Vice Versa)

FEC wins when: $t_{\text{fec}}(W) < L_{\text{arq}} \approx 1.5 \times \text{RTT}$

- High RTT (satellite): ARQ is slow, FEC's window decode is faster
- Short bursts: m small, few repairs needed, quick decode
- High bandwidth: t_{sym} small, window fills quickly

ARQ wins when: $L_{\text{arq}} < t_{\text{fec}}(W)$

- Low RTT (datacenter): ARQ round-trip is fast
- Very long bursts: m large, FEC needs many repairs
- Low bandwidth: large t_{sym} , window takes long to fill

The crossover point $t_{\text{fec}}(W) = 1.5 \times \text{RTT}$ determines the optimal FEC/ARQ balance for a given link. The window size optimization (Section 14.5) can be tuned to align this crossover with the link's RTT, making FEC optimal for all bursts shorter than the pipeline.

14.8 Per-Symbol Recovery Probability Function

Given GE parameters (p , q), the taper, and the encoder window size, we can compute a per-symbol recovery probability as a function of time:

$$P_{\text{recovery}}(T \mid m, T_w) = P(\text{Poisson}(\lambda_{\text{total}}(T)) \geq m)$$

where:

$$\begin{aligned} \lambda_{\text{total}}(T) &= \lambda_{\text{prior}}(T_w) + \lambda_{\text{new}}(T) \\ m &= \text{burst length (geometric}(q) \text{ from GE model)} \\ T_w &= \text{time symbol has been in window before loss} \end{aligned}$$

This is computable analytically via the Poisson CDF (regularized incomplete gamma function). No Monte Carlo simulation needed, though a Markov chain on GE states would give the joint distribution of (burst_length, recovery_time) for more precise analysis.

Since we measure p and q directly from the GE estimator (Section 7.5), these parameters are available at runtime. This opens the possibility of computing the recovery CDF for each symbol to make optimal FEC/ARQ decisions — a refinement of the current P_{lost} heuristic.

14.9 Reconceived Delivery Time Distribution

The corrected per-symbol delivery time CDF:

$$\begin{aligned}
 P(\text{delivered by } T) = & \\
 & (1-\epsilon) \quad \text{not lost (arrives at RTT/2)} \\
 & + \epsilon \times P(t_{\text{fec}} \leq T \mid m) \quad \text{lost, FEC recovers by } T \\
 & + \epsilon \times (1-P(t_{\text{fec}} \leq T \mid m)) \times I(T \geq L_{\text{arq}}) \quad \text{lost, ARQ recovers by } T
 \end{aligned}$$

The tail: $\delta(T) = 1 - P(\text{delivered by } T) = P(\text{delivery takes } > T)$

This allows the triangle to use a **time budget** T_{budget} instead of the current binary FEC/ARQ classification:

```

Fix  $T_{\text{budget}} + \rho \rightarrow$  compute  $r$    (minimum  $r$  for the latency target)
Fix  $r + \rho \rightarrow$  compute  $T$        (what latency does this  $r$  achieve?)
Fix  $r + T_{\text{budget}} \rightarrow$  compute  $\rho$  (reliability at this latency budget)

```

The time-based formulation connects naturally to application requirements: “99% of packets within 33ms” maps directly to $\delta(33\text{ms}) \leq 0.01$, $\rho \geq 0.999$.

14.10 Latency Tail vs Throughput: Not Always a Trade-off

With perfectly matched FEC ($r = \epsilon/(1-\epsilon)$, zero waste):

- Short-window latency: FEC recovers bursts within the window ✓
- Long-window latency: minimal overhead ($\sim \epsilon/(1-\epsilon)$ extra symbols) ✓
- Both improve simultaneously vs the no-FEC baseline

The trade-off only appears when:

1. **Over-provisioning FEC:** wastes bandwidth $\rightarrow$ worse completion time
2. **Over-provisioning ARQ:** duplicates waste bandwidth
3. **Under-provisioning either:** tail latency degrades

The taper function’s role (Section 4) is to match FEC exactly to the loss distribution, avoiding both over- and under-provisioning. When well-matched, there is no latency-vs-throughput trade-off — only the inherent cost of channel loss ($\epsilon/(1-\epsilon)$ overhead is the theoretical minimum regardless of mechanism).

14.11 Application Profiles Revisited

With the refined latency model, the application profiles from Section 1.4 gain additional precision:

VoIP/Gaming ($T_{\text{budget}} \approx 20\text{ms}$, $\rho = 98\%$):

```

FEC window should contain  $< 20\text{ms}$  of data.
At high  $\epsilon$ : accept 2% loss rather than ARQ delay.
Proactive retransmit viable if  $\epsilon > 30\%$  ( $P_{\text{lost}}(0) = \epsilon$  is already high).
 $W_{\text{optimal}}$ : small (minimize  $t_{\text{fec}}$ ), accept higher per-window loss.

```

Large file transfer (T_{budget} = relaxed, $\rho = 100\%$):

Only throughput (long-window latency) matters.
Minimize $r \rightarrow$ maximize $\text{source_rate} = C/(1+r)$.
ARQ handles the tail – per-symbol latency irrelevant.
 $r = \epsilon/(1-\epsilon)$ is optimal (information-theoretic minimum).
 W_{optimal} : large (maximize pipeline, minimize FEC overhead variance).

Live video ($T_{\text{budget}} \approx 33\text{ms}$ per frame, $\rho = 99.9\%$):

Per-frame deadline. FEC window $\approx$ one frame of data.
Moderate FEC for fast burst recovery within frame.
ARQ backstop for rare multi-frame bursts.
 W_{optimal} : $\approx \text{frame_size} / \text{symbol_size}$ (natural alignment).

Sensor/IoT (T_{budget} = relaxed, $\rho = 95\%$):

Bandwidth-constrained (low-power link).
Minimal FEC, minimal ARQ. Accept 5% loss.
 r kept minimal to maximize source throughput.
 W_{optimal} : small (save memory on constrained device).

14.12 ARQ After FEC Decode: Not Redundant for the Decoder

When the decoder recovers a lost symbol S_3 via FEC, the application can consume S_3 immediately. But if an ARQ retransmit of S_3 was already in flight, the retransmit still arrives at the receiver.

This ARQ is NOT redundant at the receiver. The decoder should still feed it as an equation – it is a source symbol (unit vector, guaranteed linearly independent) that can help decode OTHER lost symbols still in the decoder window. From the receiver’s perspective, every arriving symbol is useful to the decoder until the window slides past it.

The ARQ IS redundant at the sender – once the sender knows (via ACK) that the receiver has decoded S_3 , further retransmits of S_3 waste bandwidth. The sender should stop retransmitting ACKed symbols. This is already handled by the existing ACK mechanism: once the sender receives an ACK covering S_3 , it removes S_3 from the retransmit buffer.

The interesting case: the sender sends an ARQ for S_3 BEFORE learning that FEC decoded it. This is unavoidable due to ACK delay ($\sim$ RTT). The “wasted” bandwidth is bounded by one RTT worth of unnecessary ARQ. But the receiver still benefits from the equation – so the waste is only sender-side bandwidth, not receiver-side utility.

14.13 Proactive Retransmit vs FEC: FEC is Strictly Better

At the same overhead, FEC is strictly superior to proactive retransmit (sending every source symbol twice):

- One FEC repair covers ANY single loss in the window (W positions)
- One retransmit covers exactly ONE specific position

For a window of $W=50$, a single FEC repair is 50x more flexible than a single retransmit. Proactive retransmit only approaches FEC efficiency when $\epsilon \rightarrow 50\%$ (half of all symbols are lost, so random “insurance” is as good as targeted repair). At typical loss rates (1-20%), FEC dominates by a factor of $W/1$.

Many real-world systems use packet duplication for simplicity. This model shows the quantitative cost of that simplicity.

14.14 Marginalizing Over Burst Length

The FEC latency CDF (Section 14.3) conditions on m (burst length). The unconditional CDF marginalizes over the GE burst length distribution:

$$\begin{aligned} P(t_{\text{fec}} \leq T) &= \sum_{m=1}^{\infty} P(\text{burst} = m) \times P(t_{\text{fec}} \leq T \mid m) \\ &= \sum_{m=1}^{\infty} (1-q)^{m-1} \times q \times Q(m, \lambda(T)) \end{aligned}$$

where $Q(m, \lambda)$ is the regularized incomplete gamma function.

In practice, truncate at $m_{\text{max}} = B_{99}$ (99th percentile burst length). The tail $P(\text{burst} > B_{99}) < 1\%$ contributes negligibly to the CDF.

14.15 In-Burst FEC Survival

The Poisson model uses $(1-\epsilon)$ survival probability for all repairs. But during a burst (channel in Bad state), ALL symbols are lost — including FEC repairs. Only POST-burst repairs survive.

The corrected $\lambda(T)$ should split into two phases:

For a burst starting at $t=0$ with length B :

$$\begin{aligned} \lambda(T) &= 0 && \text{for } T < B \text{ (in-burst)} \\ &= \sum_{t=B}^T \tau(t) \times (1-\epsilon_{\text{good}}) && \text{for } T \geq B \text{ (post-burst)} \end{aligned}$$

where $\epsilon_{\text{good}} \approx p$ (probability of re-entering Bad at a post-burst slot;
the Good state itself is loss-free, $h_G = 0$) ≈ 0 for packet erasure

The practical effect: recovery begins only after the burst ends. The longer the burst, the later recovery starts. This makes burst length m doubly important — it determines both HOW MANY repairs are needed AND HOW LONG before repairs start arriving.

For the ambient pipeline (Section 14.4): repairs generated BEFORE the burst that are still in transit may arrive during or after the burst. These contribute if they survive the channel at the receiver — which they do, because they were transmitted during the Good state before the burst. The pipeline effect thus provides a head start on recovery.

14.16 The FEC/ARQ Race

FEC and ARQ work in parallel for a lost symbol. The actual delivery time is $\min(t_{\text{fec}}, L_{\text{arq}})$ — whichever recovers the symbol first.

$$P(\text{delivered by } T) = 1 - P(t_{\text{fec}} > T) \times P(L_{\text{arq}} > T)$$

$P_{\text{lost}}(t)$ (Section 3.4) currently decides the MIX in correction slots — FEC or ARQ per slot. But both mechanisms are running simultaneously: FEC repairs accumulate regardless of whether ARQ fires, and ARQ retransmits arrive regardless of FEC state.

The $\min()$ combination means the actual tail latency is BETTER than either mechanism alone. The two mechanisms are complementary, not alternatives. The P_{lost} mix controls the bandwidth split, but both contribute to recovery probability.

14.17 Decode-Induced Jitter

FEC cascade decoding produces micro-bursts: when the m -th repair arrives and the decoder resolves a pivot row, cascade recovery may immediately decode several other symbols. Multiple symbols become available to the application simultaneously.

This creates “negative jitter” — symbols arriving too close together. While not harmful (data is available earlier than expected), it means the delivery time distribution has a step-function component at decode events, not a smooth CDF.

For jitter-sensitive applications (VoIP): a de-jitter buffer after the decoder smooths the micro-bursts into a steady stream. The buffer adds a constant baseline latency (typically one frame period). The net effect on jitter depends on whether the tail improvement from FEC exceeds the baseline latency added by the de-jitter buffer.

Tail latency improvement always reduces jitter in the upward direction (fewer late symbols). Decode micro-bursts create jitter in the downward direction (symbols arriving early). The de-jitter buffer absorbs the downward jitter at the cost of baseline latency. As long as:

$$\text{de_jitter_buffer_delay} < \text{tail_latency_improvement}$$

the net result is lower jitter. For most FEC-protected links, this condition holds because FEC reduces tail latency by an RTT (replacing ARQ with FEC recovery), while the de-jitter buffer adds only a few ms.

14.18 Estimator-Rate Feedback Stability

In “compute r ” mode, the estimator’s ϵ feeds into the triangle solver which produces r . But r affects correction volume, which affects how many symbols the ESTIMATOR observes as lost-and-recovered vs lost- permanently.

Potential oscillation:

high r $\rightarrow$ good recovery $\rightarrow$ estimator sees low ϵ $\rightarrow$ solver reduces r
 $\rightarrow$ poor recovery $\rightarrow$ estimator sees high ϵ $\rightarrow$ solver increases $r \rightarrow \dots$

This feedback loop is stabilized by two properties:

1. **BOCD’s inertia:** the Bayesian changepoint detector maintains a run-length distribution with prior mass. It doesn’t react instantly to a few ticks of different ϵ — it integrates over many observations before shifting the predictive quantile.
2. **The estimator observes CHANNEL loss, not APPLICATION loss:** the estimator tracks which symbols were lost on the channel (not received at the receiver), not which symbols the application eventually got (via FEC recovery). FEC recovery doesn’t reduce the estimator’s ϵ .

Property 2 is critical. If the estimator instead measured “application- level loss” (post-FEC), the feedback loop would be unstable. The estimator must measure RAW channel loss, independent of FEC recovery.

14.19 Consistency of P_fec Models

The paper has two FEC recovery models:

- **Section 8.2:** $P_{\text{fec}} = \Phi(\sqrt{W} \times (r(1-\epsilon)-\epsilon) / \sqrt{(\epsilon(1-\epsilon)(r+\sigma^2_{\text{burst}}))})$ Normal approximation to binomial. Answers: “given r and W , what’s the probability that FEC recovers ALL losses in the window?”
- **Section 14.3:** $P(t_{\text{fec}} \leq T \mid m) = Q(m, \lambda(T))$ Poisson CDF of the correction arrival process. Answers: “given m losses, what’s the probability that m surviving repairs arrive by time T ?”

These answer different questions:

- Section 8.2: probability of FEC success (regardless of time)
- Section 14.3: time distribution of FEC recovery

They should be consistent: by the time the window slides past the lost symbol, the Poisson CDF should approach the Section 8.2 P_{fec} .

Verify: the lost symbol stays in the window for W further source symbols = $W(1+r)$ wire slots, so $\lambda(\text{window exit}) = r(1-\epsilon)/(1+r) \times W(1+r) = rW(1-\epsilon)$ — exactly the mean of the Binomial repair count in Section 8.2. The Poisson(λ) tail $P(\geq m)$ matches the first moment with a slightly wider distribution than the Binomial. The Monte Carlo suite compares the two models numerically across scenarios (tolerance 0.15).

14.20 The δ Definition Question

The paper currently has two candidate definitions for δ :

- **Section 6.3:** $\delta = P(\text{late delivery}) / \rho$ — a probability (dimensionless)
- **Section 14.9:** $\delta(T) = P(\text{delivery} > T)$ — a function of time

These serve different purposes:

- The probability definition is useful for the triangle solver (binary search on r to achieve target δ)
- The time-based definition connects directly to application requirements (“99% of packets within 33ms”)

Recommendation: keep both. The probability δ is the triangle variable. T_{budget} is the application requirement. They’re connected by:

$$\delta = \delta(T_{\text{budget}}) = P(\text{delivery} > T_{\text{budget}})$$

The user specifies T_{budget} (from application requirements). The system computes $\delta = \delta(T_{\text{budget}})$ using the delivery time CDF, then uses δ in the triangle solver. This preserves the triangle’s mathematical structure while connecting to real-world requirements.

14.21 Sub-Capacity Operation (Emergent Behavior)

When the application data rate is below link capacity, the model should handle this naturally without a separate code path.

In the current model, Copa (Section 12) controls the total sending rate C . The taper determines the source/correction split: $\text{source_rate} = C/(1+r)$. If the application produces data at rate $S < C/(1+r)$, the sender has idle capacity.

In the current architecture, Copa already handles this: if the application has no data to send, Copa’s window isn’t fully utilized, and the sending rate naturally drops to match. FEC symbols

are only generated when source symbols are sent (the taper is driven by source symbols entering the window).

An interesting possibility: when idle slots exist, the sender COULD generate additional FEC repairs proactively — filling spare capacity with extra protection. The taper's r determines the minimum FEC rate; spare capacity allows exceeding it at no throughput cost.

This should emerge from the triangle solver: when solving for r with a tight δ target, the solver may compute $r > r_{\min}$. If the link has capacity for this higher r without reducing source throughput (because $\text{source_rate} < C/(1+r)$), the system naturally operates at the higher r .

The diminishing return: when the FEC pipeline $\lambda \gg B_{99}$ (more ambient repairs than any likely burst needs), additional FEC provides negligible improvement. The marginal benefit approaches zero exponentially (Poisson tail). The triangle solver would compute r at this saturation point when δ is set very tight — the solver naturally stops increasing r when further increase doesn't improve δ .

Open question: should the taper continue generating repairs during idle periods (no source data)? These repairs cover the existing window and strengthen the pipeline. The current model ties correction slots to source slots — decoupling them would allow “idle FEC” without changing the architecture, only the scheduling policy.

A quantitative saturation model. The L0 gate measured the diminishing return turning NEGATIVE: at C4-Satellite, Realtime ($\delta = 1e-7 \rightarrow r \approx 0.49$) has a WORSE p99 than Auto (412 ms vs 297 ms). Past ~43% overhead the extra repairs displace source symbols and pressure the queue — more FEC hurts the tail. The controller increases r monotonically with hint tightness and needs a saturation point. The model below is continuous (no mode switch) and uses only estimator-known quantities: ϵ (loss), σ^2_{burst} (Section 8.3), W (window), SRTT, and $t_{\text{sym}} = \text{symbol_size} / \text{throughput}$ (the wire slot time).

The p99 tail has components that pull in opposite directions in r :

```
tail_fec(r) = (1 - P_fec(r)) × L_arq [decreasing]
FEC-miss cost: misses fall through to ARQ. P_fec from the
Section 8.2 normal formula; L_arq ≈ 1.5 × SRTT (Section 14.9).

tail_rec(r) = B × t_sym × (1+r) / (r × (1-ε)), B = (σ²+1)/2 [decreasing]
Recovery wait: repairs occupy an r/(1+r) share of wire slots and
survive with probability (1-ε), so the wait for the B surviving
repairs a mean burst needs is B × t_sym × (1+r)/(r(1-ε)). This is
why moderate r keeps helping even after P_fec ≈ 1: repairs arrive
SOONER. B is the mean burst length recovered from the variance
factor (σ² = 2B - 1 when p ≪ q, Section 8.3).

tail_svc(r) = c × (1+r) × W × t_sym, c = 1/2 [increasing]
Dilution cost: corrections consume wire share r/(1+r), stretching
the effective per-source service time — the recovery window
passes at the diluted source rate, taking (1+r) × W × t_sym.

p99_model(r) = tail_fec(r) + tail_rec(r) + tail_svc(r)
```

The sum has an interior minimum r_{sat} , and the controller should emit $\min(r_{\text{hint}}, r_{\text{sat}})$: the hint still picks the operating point on the rising side of reliability, but cannot push past the

point where more FEC hurts. The cap is hint-independent — saturation is a channel property, not a preference.

Worked example (C4-Satellite). $\epsilon \approx 9\%$, $\sigma^2 \approx 5$ ($B = 3$), $W = 64$, $\text{SRTT} \approx 0.21$ s ($L_{\text{arq}} = 315$ ms), throughput 2.5 MB/s, symbol 1225 B $\rightarrow t_{\text{sym}} = 4.9\text{e-}4$ s, $W \times t_{\text{sym}} = 31.4$ ms:

r	tail_fec	tail_rec	tail_svc	total	
0.20	40.9 ms	9.7 ms	18.8 ms	69.4 ms	
0.30	4.1 ms	7.0 ms	20.4 ms	31.5 ms	
0.35	0.9 ms	6.2 ms	21.2 ms	28.3 ms	
0.40	0.2 ms	5.7 ms	22.0 ms	27.8 ms	<- r_sat
0.49	0.0 ms	4.9 ms	23.4 ms	28.3 ms	

$r_{\text{sat}} = 0.40$, below Realtime's uncapped 0.49 — consistent with the measured reversal. At C2-WiFi-like numbers ($\epsilon = 2.5\%$, $\sigma^2 = 3$, $W = 64$, $\text{SRTT} = 13$ ms, $t_{\text{sym}} = 1\text{e-}4$ s) the minimum sits at $r_{\text{sat}} \approx 0.255$, ABOVE Realtime's ~ 0.20 request there: the cap is non-binding exactly where the measurements show more FEC still helping (C2 Realtime p99 28.1 ms beats Bulk's 31.5 ms).

Honesty about roughness. The dilution constant $c = 1/2$ is a fitted scale, not derived; queueing is ignored beyond linear dilution, so the model's minimum is much shallower than the measured one (the model says +0.5 ms from r_{sat} to 0.49 at C4; the gate measured +115 ms — the real cost past saturation is a queueing knee, not a line). The model is therefore trusted for the LOCATION of the minimum, not the depth. B from σ^2 assumes $p \ll q$. Estimation error in ϵ moves r_{sat} by a few percent (higher $\epsilon \rightarrow$ later saturation), which is the safe direction: when in doubt the cap loosens. When the estimator has no throughput sample, t_{sym} is unknown and no cap applies (degenerate inputs $\rightarrow$ no cap): the monotone hint behavior of the base model is preserved.

14.22 Sequence-Aware P_{lost}

The current $P_{\text{lost}}(t)$ (Section 3.4) uses only time since send. But SACK feedback (Section 6.2) provides stronger evidence: if subsequent symbols have been ACKed, an un-ACKed symbol is almost certainly lost.

$$P_{\text{lost_seq}}(k) = P(S_{\text{n lost}} \mid S_{\{n+1\}..S_{\{n+k\}}} \text{ all received})$$

On a FIFO channel (no reordering), if the next symbol arrived, the previous one was lost — $P_{\text{lost_seq}}(1) \approx 1.0$. Real links are not perfectly FIFO: network jitter, multipath, and switch buffering can reorder packets.

Reorder probability estimation: the system should track the observed reorder rate (fraction of symbols that arrive out of order) as a running estimate. Starting assumption: FIFO ($\text{reorder_rate} = 0$). The SACK ranges provide the observation data — each out-of-order arrival updates the estimate.

$$P_{\text{lost_seq}}(k, \text{reorder_rate}) = 1 - \text{reorder_rate}^k$$

$$\begin{aligned} \text{For FIFO (reorder_rate} = 0\text{):} \quad & P_{\text{lost_seq}}(1) = 1.0 \\ \text{For mild reorder (rate} = 0.05\text{):} \quad & P_{\text{lost_seq}}(1) = 0.95 \\ & P_{\text{lost_seq}}(3) = 0.9999 \end{aligned}$$

The combined P_{lost} uses both time AND sequence evidence:

$$P_{\text{lost_combined}} = \max(P_{\text{lost_time}}(t), P_{\text{lost_seq}}(k, \text{reorder_rate}))$$

This makes the FEC/ARQ decision faster: instead of waiting for P_lost_time to rise (which takes $\sim SRTT$), a single subsequent ACK gives near-certain evidence on a FIFO link. Correction slots can switch to ARQ sooner for confirmed losses, freeing FEC capacity for uncertain losses.

14.23 Post-Burst FEC Boost (Reactive Deficit Recovery)

After an unexpectedly long burst (longer than the taper was provisioned for), the system has a correction deficit — more losses occurred than the ambient FEC pipeline can cover. The BOCD estimator will adapt ϵ within 5-15 batches, but during that lag the system is under-protected.

The GE model's state transitions provide a faster signal: the Bad→Good transition indicates burst end. At that moment, the deficit is known:

```
On Bad→Good transition:
burst_length = observed consecutive losses
repairs_in_pipeline =  $\lambda\_prior$  (ambient FEC accumulated)
deficit = max(0, burst_length - repairs_in_pipeline)
```

If deficit > 0, the system should temporarily boost FEC:

```
boost_duration = deficit / (r × (1- $\epsilon$ ))    ticks
boost_r = r + deficit / boost_duration    (spread the extra FEC)
```

This is faster than waiting for BOCD to increase $\epsilon \rightarrow$ solver to increase $r \rightarrow$ taper to generate more corrections. The GE state transition is observable within one symbol interval; BOCD takes 5-15 batches.

Interaction with ARQ: at burst end, the oldest lost symbols are $\sim$ burst_length ticks old. If burst_length > SRTT, P_lost_time is already high for those symbols — ARQ fires naturally. The boost is needed for the symbols where FEC should have covered them but didn't (pipeline was insufficient).

Open question: should the boost be additive (extra FEC on top of normal taper) or multiplicative (temporarily higher r)? Additive is simpler and doesn't affect the taper shape for new symbols. The extra repairs specifically target the deficit, not general protection.

14.24 Jitter-Horizon Encoder Lag

Discovered while building the L0 gate suite (ADR-0051): with per-packet jitter J , a repair symbol can overtake up to $J \times \text{send_rate}$ of the source symbols it covers. At arrival, those covered-but-still-in-flight symbols are unknowns to the decoder, so the repair parks as a deep pivot equation instead of decoding the actual loss — measured hole-fill p50 at C2-WiFi was ~ 21 ms instead of the expected ~ 2 ms.

A repair covering symbols that cannot yet have arrived carries no usable information at arrival time. The encoder should therefore TRAIL the send stream by the jitter horizon:

```
L = ceil(J × send_rate)    [symbols; 2..48 in practice]

repairs cover [sent - L - W, sent - L]    instead of    [sent - W, sent]
```

With the lag, a repair's unknowns at arrival are true losses, decodable immediately. The correction budget is unchanged (Section 4.2 shape invariance) — only the window placement moves. L adapts with measured jitter and send rate; on a jitter-free channel $L \rightarrow 0$ and the windows coincide. This composes with Section 14.5's window sizing: the effective protection span for fresh symbols shrinks by L , which matters only if L approaches W .

14.25 Completion-Tail FEC

For a finite transfer, completion time decomposes as:

$$T_{\text{completion}} = T_{\text{send}} + T_{\text{tail_recovery}}$$

$$T_{\text{send}} = N \times t_{\text{sym}} / (1 + r)^{-1} \text{ -adjusted source rate}$$

(the time to push all N source symbols)

$$T_{\text{tail_recovery}} = \text{the time to recover losses among the LAST window's symbols after the send stream ends}$$

The two terms have completely different loss economics. A mid-transfer loss is recovered in parallel with ongoing sends: ARQ (or a later repair) rides alongside new source symbols, so the recovery consumes wire budget but adds ZERO completion time — the link never goes idle waiting for it. A tail loss is different: once the last source symbol has been sent there is nothing left to overlap with, so every ARQ round on a final-window hole is serial — ~ 1.5 RTT each (detection at $\sim 9/8$ SRTT plus the retransmit flight), and a retransmit that is itself lost pays the full round again.

This yields an end-of-stream policy that is nearly free:

At end-of-stream (last source symbol sent), send a burst of
 $n_{\text{tail}} = \text{ceil}(r_{\text{tail}} \times W)$ repair symbols covering the final window.

Cost: $r_{\text{tail}} \times W$ symbols $\approx$ negligible vs N for a large transfer
 (e.g. $0.2 \times 64 = 13$ symbols on a 1500-symbol transfer $< 1\%$)

Saving: $P(\geq 1 \text{ tail loss}) \times \sim 1.5$ RTT of completion time per avoided
 serial ARQ round, where
 $P(\geq 1 \text{ tail loss}) = 1 - (1 - \epsilon)^W \quad (\approx 80\% \text{ at } \epsilon=2.5\%, W=64)$

r_{tail} comes from the exact transfer-matrix computation (Section 8.7): the smallest r such that $P_{\text{fail}}(r, W) \leq \delta_{\text{tail}}$ for a modest tail-failure budget (e.g. $\delta_{\text{tail}} = 0.05$ — one residual serial ARQ round in 20 transfers). The exact DP matters here: the tail burst is a one-shot, small- W event where the normal approximation's 30-50% tail under-provisioning (Section 8.7) directly converts into completion regressions.

Composition with Bulk's $r \rightarrow 0$ steady state (Sections 5.3, 12.5). Bulk maps δ to “late is fine” ($\delta_{\text{bulk}} = \hat{\epsilon}$ mid-stream, Section 14.26), so the continuous r^* formula (Section 8.4) is 0 mid-transfer: pure ARQ, volume parity with retransmission transports. Completion-tail FEC is the complement, not a contradiction: FEC vanishes in the steady state (where recovery overlaps sending and buys nothing) and reappears exactly at the one place it buys completion time — the stream tail. The r - δ - ρ triangle is respected at both operating points; only the effective δ differs, because the COST MODEL differs between mid-transfer (parallel recovery, late is genuinely fine) and the tail (serial recovery, late is 1.5 RTT each). Section 14.26 makes this composition CONTINUOUS: the one-shot burst described above is the limiting case of a δ glide driven by a completion-exposure kernel χ , and the glide supersedes the burst wherever T_{rem} is known.

Multipath corollary — tail reinjection. On asymmetric paths the same end-of-stream logic applies to slow-path IN-FLIGHTS, not just losses: once nothing overlaps recovery, an undelivered symbol whose path's residual wait (queue + propagation) exceeds a fast-path flight is worth duplicating onto the fast path (cross-path retransmit, Section 13.10). The duplicate rides spare end-of-stream tokens, so the cost is a few symbols of fast-path capacity against

a saving of the slow path's queue drain (~10-25 ms measured on WiFi+LTE). Same principle, ARQ flavor: completion is bought exactly at the stream tail, nowhere else.

14.26 Completion-Exposure δ (the Bulk glide)

Section 14.25 established the two-regime cost model for Bulk: a mid-transfer loss recovers in parallel with ongoing sends (zero completion cost), a tail loss recovers serially (~1.5 SRTT per ARQ round). It implemented the tail side as a one-shot repair burst at end-of-stream. This section replaces the sharp mid/tail boundary with a continuous kernel and, in doing so, fixes two measured flaws in the original Bulk hint mapping $\delta_{\text{bulk}} = \min(0.1, \hat{\epsilon})$.

The two flaws of $\min(0.1, \hat{\epsilon})$ (measured in the wasm simulator, which shares `controller_rate` with production):

M1 – cold start. The loss estimator's Beta(1,1) prior puts the 95% predictive upper quantile at $\hat{\epsilon}_{.95} \approx 0.975$ for the first ~1.5-3 RTT. The clamp maps this to $\delta_{\text{eff}} = 0.1 \ll \hat{\epsilon}_{.95}$, so $z_{\{\delta/\epsilon\}}$ is large, the IT term $\epsilon/(1-\epsilon) \approx 39$ dominates, and r pins at $\text{max_overhead} = 0.5$ – wasting ~1/3 of the wire for 2-3 RTTs on a channel about which nothing bad is actually known. Measured: a FIXED $r = 0.01$ floor beat Bulk on completion in 20/24 grid cells (median +5%, worst +17%, gap growing with RTT) and on overhead in 24/24 (excess overhead 2-14% vs ~0-1%).

M2 – moderate loss. At $\epsilon \geq \sim 0.1$ the upper quantile sits above the 0.1 clamp FOREVER, so $\delta_{\text{eff}} = 0.1 < \hat{\epsilon}_{.95}$ permanently and $r^* > 0$ in the steady state: Bulk pays FEC $\approx$ the IT floor AND ARQ for the same losses – double payment, directly against the 14.25 cost model in which mid-stream lateness is free.

Both flaws share one root: any δ_{eff} strictly below $\hat{\epsilon}$ re-activates the margin machinery, whose entire purpose Bulk rejects mid-stream.

The completion-exposure kernel. What actually distinguishes a loss that costs completion time from one that does not is whether its ARQ round (~1.5 SRTT, Section 3.4) can still hide behind remaining sends. Let T_{rem} be the remaining send time (in seconds, like Section 5.4's timing quantities: $T_{\text{rem}} = \text{remaining source symbols} / \text{send rate}$). Reusing the Section 3.4/5.4 normal-RTT-tail machinery (Φ = normal survival function):

$$\chi(T_{\text{rem}}) = \Phi_{\text{bar}}((T_{\text{rem}} - 1.5 \times \text{SRTT}) / \sigma_{\text{arq}})$$

$$\sigma_{\text{arq}} = \max(4 \times \text{RTTVAR}, \text{SRTT} / 4)$$

χ is the probability that a loss suffered NOW is EXPOSED – that its recovery outlives the send stream and becomes serial completion time. The $1.5 \cdot \text{SRTT}$ center is the Section 14.25 serial-round cost; σ_{arq} aggregates detection and flight-time variance ($4 \times \text{RTTVAR}$, floored at $\text{SRTT}/4$ against degenerate RTTVAR estimates). Mid-stream ($T_{\text{rem}} \gg \text{SRTT}$) $\chi = 0$; over the final ~1.5 SRTT it rises smoothly to 1. No cutoff anywhere: χ inherits its continuity from Φ .

The δ glide. Bulk's effective tail target is the exposure-weighted blend of “the channel itself” and the 14.25 tail budget:

$$\delta_{\text{bulk}} = \text{eps_hat} + (\delta_{\text{tail}} - \text{eps_hat}) \times \chi, \quad \delta_{\text{tail}} = 0.05$$

- **Mid-stream ($\chi = 0$):** $\delta_{\text{bulk}} = \hat{\epsilon}$ exactly, so $z_{\{\delta/\epsilon\}} = \Phi^{-1}(0) = -\infty$ and $r^* = 0$ IDENTICALLY – independent of the estimator's uncertainty, because the target tracks the estimate ITSELF

rather than a constant the estimate is compared against. This kills M1 ($\hat{\epsilon}_{95} \approx 0.975 \rightarrow \delta_{\text{eff}} = 0.975 \rightarrow r^* = 0$) and M2 ($\hat{\epsilon}_{95} = 0.12 \rightarrow \delta_{\text{eff}} = 0.12 \rightarrow r^* = 0$) in one stroke. It is exactly the “late is fine” semantics of the 14.25 cost model: mid-stream recovery is parallel and therefore free, so NO channel state justifies steady-state FEC under Bulk.

- **Stream tail ($\chi \rightarrow 1$):** $\delta_{\text{bulk}} \rightarrow \delta_{\text{tail}} = 0.05$ regardless of $\hat{\epsilon}$, and r ramps continuously to the exact rate that meets the completion budget — reaching it about 1.5 SRTT before the last source symbol, precisely when losses start being exposed.
- **T_rem unknown:** a production tunnel is an endless stream — there is no “last symbol” the sender can see, so $\chi = 0$ permanently and the steady state is pure ARQ (existing production tail behavior is unchanged). Feeding χ from an application-known transfer size, or from an idle-onset heuristic (send queue drained = provisional end of stream, retracted if new data arrives), is future work; simulation drivers that know the transfer length feed χ directly.

The tail burst as the glide’s limiting case. The 14.25 one-shot burst is what the χ ramp degenerates to as $\sigma_{\text{arq}} \rightarrow 0$: a step from $r = 0$ to the tail rate at $T_{\text{rem}} = 1.5$ SRTT. The glide is therefore not a second mechanism but the burst made continuous — and it supersedes the burst wherever T_{rem} is known (firing both would double-pay the tail budget). Beyond removing the discontinuity, the ramp fixes a coverage gap of the burst at high RTT: with in-flight span (symbols per RTT) $\gg W$, the last-window burst cannot protect late-stream losses that fall OUTSIDE the final window yet inside the final serial-recovery horizon. The ramp’s repairs are emitted across the whole final ~ 1.5 SRTT of the send stream, each covering the sliding window at its emission time, so the entire exposed span gets coverage. The same mechanism suppresses the pure-ARQ ($r = 0$ floor) serial-retransmit tail outliers: a tail hole whose retransmit is itself lost pays full serial rounds (one observed 865-tick outlier vs 559 ticks with a 1% floor); ramped repairs give those holes a parallel recovery path.

Scope of the mid-stream guarantee. $\chi = 0$ requires T_{rem} to exceed the exposure horizon $\sim 1.5\text{-SRTT} + 8 \cdot \sigma_{\text{arq}} (\approx 3.5\text{-}5.5 \text{ SRTT})$. A transfer SHORTER than that never has a mid-stream phase: $\chi > 0$ from the first symbol, and during the estimator cold start the glide inherits M1’s inflated $\hat{\epsilon}_{95}$ (measured at 150 ms RTT with a 0.5 s transfer: neither mapping wins — the early rate is governed by the cold-start prior in both). The glide’s guarantee is for streams long enough to HAVE a middle; the cold-start prior itself is estimator work, not δ -mapping work.

Verification (wasm simulator, shared formula, same seeds per cell; $\epsilon = 0.05/0.10$, $q = 0.5$, RTT = 50 ms, $W = 64$): the glide vs the old mapping cuts completion $599 \rightarrow 562$ ticks and excess overhead $5.99\% \rightarrow 0.04\%$ at $\epsilon = 0.05$, and $674 \rightarrow 620$ ticks / $8.21\% \rightarrow 0.91\%$ at $\epsilon = 0.10$; Bulk now beats the fixed $r = 0.01$ floor on BOTH completion (562 vs 598 ticks) and overhead (5.31% vs 6.11%) at the reference cell — the mapping the 20/24 finding said it must at least match.

14.27 Block-Mode ARQ via Batch Acknowledgements

Section 5 defines corrections as repairs $\cup$ retransmits, both driven by P_{lost} ; Section 14.26 then makes mid-stream $r^* = 0$ for Bulk on the grounds that a mid-stream loss recovers “for free” through the reactive path. The production BLOCK pipeline implemented neither reactive leg: on a decode failure the sender only updated stats and congestion control, and the receiver evicted the incomplete decoder after a timeout. Nothing was ever retransmitted. Under the glide this is not a degraded mode but a contradiction — $r^* = 0$ is only correct BECAUSE the retransmit path exists — and the L1 harness measured the consequence directly: at C2 (100 Mbit, 10 ms RTT, GE 1.3%/50% $\approx 2.6\%$ loss) the production tunnel completed a 1.8 MB transfer

in ~8 s against quinn’s 0.175 s. The inner TCP saw the raw 2.6% loss and collapsed; every block with a hole waited out the 30 s decoder eviction. The window pipeline has its own retransmit buffer (Section 6.1); this section specifies the block-mode equivalent.

The Ack IS the P_lost evidence. The block receiver already acknowledges every Symbol-Batch it receives — each QUIC datagram is one batch, so batch loss is symbol loss. The v4 Ack echoes the batch’s `batch_seq`, which keys a sender-side ledger of (`batch_seq` → path, [(block, symbol)], send time). This is the SACK limiting case of the Section 3.4 P_lost model: an Ack for batch *j* on a path, with none for an earlier batch *i* on the same path, drives $P(\text{lost}_i) \rightarrow 1$ without waiting for a timer. Concretely a batch is declared lost when 3 later same-path batches have been acked (the dup-ACK analogue; tolerates datagram reordering) or when $\max(1.5\text{-SRTT}, 50 \text{ ms})$ elapses without an Ack (the timeout leg — needed for transfer tails, where no later traffic exists to accuse the loss; a 25 ms sweep task covers that case). The ledger is keyed by `batch_seq` and NOT by anything already in the v3 Ack: the echoed send timestamp is shared by every chunk of one drain call, and a batch may interleave symbols of several blocks, so the v3 fields identify neither the batch nor the victims. Adding the 8-byte field (protocol v4) was the robust option.

Fresh repairs beat resends. The sender retains source data for the last 64 blocks (LRU, ≤ 4 MB). On a loss event it re-derives the block encoder and, for rateless codes (RaptorQ, RLC), mints repair symbols at ESIs beyond anything previously sent: any repair fills any hole, so a fresh repair strictly dominates resending the specific lost symbol (which a burst may kill again — fresh repairs also compose across multiple holes in one block). Fixed-rate codes (RS, METTLE) cannot mint post-hoc, so they fall back to resending the exact missing symbols from the retained data, which every decoder accepts. Correction volume per event is missing + $\lfloor \text{accumulated } \hat{\epsilon} \text{ margin} \rfloor$ — the margin is fractional and carried continuously across events (at MTU-sized batches an event is typically ONE symbol; a per-event ceil would double the correction volume, i.e. cost 2ϵ instead of $\epsilon(1+\epsilon)$). Repair batches re-enter the ledger, so a lost repair triggers the next round with doubled margin, capped at 3 rounds per block; the receiver’s eviction timeout remains the final backstop.

Recovery bound. Detection costs one RTT (the Ack of the next surviving batch), repair transmission another half: a mid-stream hole closes in ~1.5 RTT wall time WITHOUT stalling the send stream — which is exactly the “parallel recovery” the 14.25 cost model prices at zero completion cost, now actually implemented. Ack loss ($\sim \hat{\epsilon}$ of batches) is indistinguishable from batch loss and costs one spurious repair event, bounded overhead $\sim \hat{\epsilon}^2$; the receiver drops symbols for already-decoded blocks, so spurious repairs are harmless. Corrections are charged against the same `in_flight` budget and pacing tokens as scheduled symbols (Section 12.5), and the estimator is NOT fed from the ledger — the receiver-side batch-gap accounting already reports these losses, and double-feeding would bias $\hat{\epsilon}$ upward.

Honest constants: 64 retained blocks / 4 MB, dup-ACK threshold 3, loss timeout $\max(1.5\text{-SRTT}, 50 \text{ ms})$ clamped to 2 s, sweep cadence 25 ms, margin $\hat{\epsilon} \cdot 2^{\text{round}}$ continuous, 3 rounds max, 4096-entry ledger cap. L1 verification of the C2 completion gap (8 s → competitive with the 0.175 s quinn bound) is pending; the mechanism is unit-verified at L0 (Ack-diff, dup-ACK and timeout legs, mixed-block batches, lost-Ack non-amplification, lost-repair second round, LRU caps, margin math, fresh-vs-resend per backend, decode-after-repair for RaptorQ and RS).

Appendix A: Summary of Key Formulas

Channel (Section 2):		
$e = p/(p+q)$	average loss rate	[probability]
$B = 1/q$	mean burst length	[symbols]
$P(\text{burst} \geq t) = (1-q)^{t-1}$	burst survival	[probability]
Taper function (Section 4):		
$\tau^*(t) = A^* \times (1-q)^t$	optimal taper function	[corrections/ symbol]
$A^* = r^* \times q$	taper amplitude ($\rho=100\%$)	[corrections/ symbol]
Burst variance correction (Section 8.3):		
$\sigma^2_{\text{burst}} = 1 + 2(1-p-q)/(p+q)$	burst variance inflation	[dimensionless]
$\text{Var}_{\text{GE}}(K) = W \times e \times (1-e) \times \sigma^2_{\text{burst}}$	loss count variance	[symbols ²]
Optimal correction rate (Section 8.4):		
$r^* = \max(0, e/(1-e) + z_{\{\delta/e\}} \times \sqrt{e \times s2_{\text{burst}} / (W \times (1-e))})$		[ratio]
$z_{\{\delta/e\}} = \text{normal_quantile}(1 - \delta/e)$	$(r^* \rightarrow 0 \text{ smoothly as } \delta \rightarrow e)$	
With codec: replace e with e_{hat} (see Section 9.2)		
$P_{\text{fec}} = \Phi(\sqrt{W} \times (r(1-e)-e) / \sqrt{e(1-e)(r+s2_{\text{burst}})})$		[probability]
Exact P_{fec} : transfer-matrix DP over the GE chain (Section 8.7)		
Codec overhead (Section 9.2):		
$e_{\text{codec_eff}} = e_{\text{codec}} \times (1-(1-e)^W)$	weighted codec overhead	[probability]
$e_{\text{hat}} = e + e_{\text{codec_eff}}$	effective loss rate	[probability]
Bulk completion-exposure glide (Section 14.26):		
$\chi(T_{\text{rem}}) = \Phi_{\text{bar}}((T_{\text{rem}} - 1.5 \times \text{SRTT}) / \sigma_{\text{arq}})$		[probability]
$\sigma_{\text{arq}} = \max(4 \times \text{RTTVAR}, \text{SRTT}/4)$		[seconds]
$\delta_{\text{bulk}} = e_{\text{hat}} + (\delta_{\text{tail}} - e_{\text{hat}}) \times \chi$	$\delta_{\text{tail}}=0.05$	[probability]
$(\chi = 0 \text{ mid-stream / unknown } T_{\text{rem}} \rightarrow \delta_{\text{bulk}} = e_{\text{hat}} \rightarrow r^* = 0)$		
Three-variable optimization (Section 8.6):		
Taper cutoff: $\tau(t) = 0$ for $t > T_{\text{cut}}$ ($\rho < 100\%$)		
Correction rate with cutoff: $r = A \times (1-(1-q)^{T_{\text{cut}}+1}) / q$		
Given $(\delta, \rho) \rightarrow r$: find T_{cut} from ρ , find A from δ , $r = A(1-(1-q)^{T_{\text{cut}}+1})/q$		
Given $(r, \rho) \rightarrow \delta$: find T_{cut} from ρ , compute A from r , $\delta = e(1-P_{\text{fec}})P_{\text{arq}}/\rho$		
Given $(r, \delta) \rightarrow \rho$: iterate T_{cut} until budget constraint met, $\rho = \text{recovery within } T_{\text{cut}}$		
Per-symbol delivery (Section 6.3):		
$P(\text{on-time}) = (1-e) + e \times P_{\text{fec}}$		[probability]
$P(\text{late}) = e \times (1-P_{\text{fec}}) \times P_{\text{arq}}$		[probability]
$P(\text{lost}) = e \times (1-P_{\text{fec}}) \times (1-P_{\text{arq}}) = 1-\rho$		[probability]
$P_{\text{arq}} = 1 - (1-\rho) / (e \times (1-P_{\text{fec}}))$		[probability]
Recovery latency (Section 3.4):		
$t_{\text{sym}} = \text{symbol_size} / \text{throughput}$	symbol transmission time	[seconds]
$t_{\text{fec}} = m \times (1+r) / (r \times (1-e)) \times t_{\text{sym}}$	FEC recovery time	[seconds]
$t_{\text{recovery}_i} = P_{\text{fec}_i} \times t_{\text{fec}_i} + (1-P_{\text{fec}_i}) \times L_{\text{arq}_i}$		[seconds]
$P_{\text{lost}}(t) = e / [e + (1-e) \times P(\text{RTT} > t)]$	loss confidence	[probability]
$P(\text{RTT} > t) = 1 - \Phi((t - \text{SRTT}) / \text{RTTVAR})$		[probability]
$L_{\text{actual}} = \min(t_{\text{fec}}, \text{retransmit arrival})$		[seconds]
Retransmit buffer (Section 6.1):		
$B_{\text{max}} = \text{ceil}(\ln(0.0001) / \ln(1-q))$		[symbols]
$\text{buffer_max} = \text{source_rate} \times (\text{RTT} + B_{\text{max}}/(r^*(1-e))*t_{\text{sym}})$		[symbols,

rho=100%]		
buffer_max = source_rate x T_cut		[symbols,
rho<100%]		
Per-slot decision (Section 5.4):		
P_retx = P_lost(t)		(time-based
mixing)		
with probability P_retx: send source retransmit		(immediate
decode)		
with probability 1-P_retx: send repair symbol		(FEC, any loss)
Optional refinement: P_retx = P_lost(t) x (1 - e_burst)		
for long-burst scenarios (Appendix C.6)		
Congestion control (Section 12):		
Copa: rate = 1 / (d_copa x dq)		[symbols/sec]
dq = RTT_current - RTT_min		[seconds]
source_rate = total_rate / (1 + r*)		[symbols/sec]
correction_rate = total_rate x r* / (1 + r*)		[symbols/sec]
Multi-path scheduling (Section 13):		
E_i = RTT_i/2 + e_i x t_recovery_i		[seconds]
B_eff_i = C_i / (1 + r_i)		[symbols/sec]
e_combined = SUM(C_i x e_i) / SUM(C_i)		[probability]
deficit = SUM_{un-ACKed s}(e_s)		[expected
corrections]		
cross-path: r = e_source / (1 - e_correction)		[ratio]
minimize: w_lat x SUM(x_i x E_i) + w_bw x SUM(x_i x r_i)		
P(cross-path retx) = P_lost(t, e_src)		
P(both paths fail) = e_src x e_retx		[probability]

Appendix B: Related Work

ACK-Based vs NACK-Based Recovery

Our model uses ACK-based (positive acknowledgment) feedback, the same approach that TCP has used successfully for over 40 years. In contrast, many real-time protocols (e.g., RTCP NACK [RFC4585]) use negative acknowledgment where the receiver explicitly reports losses.

Why ACK-based is more robust for unicast: - ACKs confirm what works; NACKs report what failed. If a NACK is lost, the sender never learns about the loss. If an ACK is lost, the sender simply retransmits after a timeout (self-healing). - SACK [RFC2018] provides the same information as NACKs (which symbols are missing) but derived from positive evidence (which symbols arrived). - The sender can infer losses from SACK gaps without requiring the receiver to detect and report them.

NACK-based approaches remain useful for multicast (one sender, many receivers) where ACK implosion is a concern [RFC4585]. For our unicast tunnel model, ACK+SACK is strictly superior.

Hybrid FEC-ARQ for Lossless Streaming

The closest prior work is Mehrotra, Li & Huang [Mehrotra2010], who solve the same core problem: minimize delivery delay for lossless, in-order streaming over a lossy channel using hybrid FEC+ARQ. Their key result: sometimes it is optimal to **preempt original data packets with FEC packets** — delaying new data to send proactive repair prevents an ARQ round-trip and reduces overall latency. This maps directly to our taper function concept.

Differences from our model: - They use a Markov Decision Process (MDP) formulation and compute the optimal policy numerically. We derive a closed-form taper function from the GE survival function, which is more directly implementable. - Their model doesn't account for burst correlation in the margin term (our σ^2 _burst correction). - They don't address multi-protocol traffic classes or interleaving.

The precursor paper [Mehrotra2009] establishes the basic hybrid FEC-ARQ protocol framework.

Streaming Codes over Gilbert-Elliott Channels

Vajha et al. [Vajha2020] provide the first **analytical** (not simulation-only) bounds on block-erasure probability for streaming codes over GE channels. Previously, streaming codes were designed for a simplified proxy channel (the delay-constrained sliding window model from [Badr2017]) and then evaluated via simulation on GE. Their upper and lower bounds could be used to verify our P_fec predictions without simulation.

Very recent work [RLC_GE2025] analyzes Random Linear Codes (our RLC backend) specifically over GE channels, providing analytical performance characterization that directly applies to our system. A related result [Vajha2020b] formalizes the sliding window approximation of the GE channel used in streaming code design.

The foundational work on delay-constrained burst erasure correction [Martinian2004] established the theoretical framework that later streaming code constructions build upon. Krishnan & Ramkumar [Krishnan2020] provide the simplest rate-optimal streaming code construction (staggered diagonal embedding), demonstrating that optimal streaming codes need not be complex. Tambur [Rudow2023] is a production-quality implementation of streaming codes for videoconferencing, demonstrating 26% fewer decode failures and 35% less redundancy bandwidth — validating that streaming code theory translates to practical gains.

Tail Latency Optimization with Proactive FEC

CloudBurst [Zeng2021] uses proactive FEC over multipath to reduce p99 tail latency by 60-75% in commodity datacenters. Different setting (datacenter LAN vs our WAN/wireless focus) but the same core insight: proactive FEC-coded packets spread across paths, recovered from the first arrivals. They use rateless fountain codes; we use windowed RLC/streaming codes with a taper.

Joint FEC/ARQ under Gilbert-Elliott

Razavi et al. [Razavi2008] develop adaptive heuristic algorithms that jointly select FEC redundancy and ARQ persistence under GE channel conditions, applied to video streaming. They search the joint parameter space numerically. Our approach is more principled: the GE parameters directly determine the taper shape, and the tail latency constraint determines the amplitude analytically.

Water-Filling and Optimal Resource Allocation

The water-filling principle [Gallager1968] — allocating resources proportional to channel quality — is foundational in information theory. Our taper function applies this principle in the time domain: allocate correction density proportional to the conditional loss probability $(1-q)^t$. While water-filling is well-known for power allocation in OFDM, its application to correction symbol density matching the GE burst survival function appears novel.

What This Model Contributes

Aspect	Prior work	This model
Optimization target	Throughput, avg delay, or weighted cost	Tail latency only (= tail loss under 100% reliability)
FEC/ARQ balance	MDP or heuristic search	Closed-form from GE params + tail constraint
Taper function	Not formalized	GE survival function $\tau(t) = A(1-q)^t$
Burst correction	Not addressed	$\sigma^2_{\text{burst}} = 1+2(1-p-q)/(p+q)$
Protocol hint	Separate FEC/ARQ tuning knobs	Single knob: tail latency target δ
Correction rate formula	Numerical	$r^* = \varepsilon/(1-\varepsilon) + z_{\delta}\sqrt{\varepsilon \cdot \sigma^2_{\text{burst}}/(W(1-\varepsilon))}$
Feedback mechanism	NACK-based or separate FEC/ARQ	Unified correction symbols with ACK+SACK

Appendix C: Model Extensions from Related Work

The following extensions are motivated by concrete results from related research and represent improvements over the base model in Sections 1-11.

C.1 Correction Symbol Preemption of Source Data [Mehrotra2010] (resolved)

Mehrotra & Li show via MDP that during detected bursts, the optimal policy shifts from sending new source data to sending correction symbols. Our model produces the same behavior through two mechanisms:

1. **P_{lost}(t) during bursts:** Since the ACK hasn't had time to return (RTT hasn't elapsed), P_{lost} stays at the base rate ε — almost all correction slots are FEC repair. This is exactly Mehrotra's recommendation: send repair (not retransmit) during bursts.
2. **P_{lost}(t) after bursts:** Once ACKs reveal gaps (after RTT), P_{lost} rises to ~ 1 for lost symbols — correction slots switch to retransmit. Immediate decode at receiver, targeted recovery.
3. **Source preemption via r:** When BOCD detects increased ε , the taper amplitude A increases, and $\tau(t)$ naturally exceeds 1.0 at small t . This means more correction slots than source slots — source is effectively paused. No special $\tau > 1.0$ rule needed; it emerges from the formula.

The P_{lost} model and Mehrotra's MDP agree on the optimal policy. The difference is derivation: we arrive at the answer from a Bayesian posterior (P_{lost}) and rate adaptation (r^*), while Mehrotra uses dynamic programming.

C.2 Information Debt for Exact P_{fec} [RLC_GE2025]

The base model uses a normal approximation for P_{fec}. The information debt framework tracks the running deficit between received and needed symbols:

$$I_d(t) = \text{symbols_needed}(t) - \text{symbols_received}(t)$$

Recovery occurs when I_d returns to 0. The slot error probability is:

$$p_e = E\{\text{error_slots}\} / E\{\text{debt_cycle_length}\}$$

where error_slots counts time steps where $I_d \geq \zeta$ (the maximum recoverable debt, determined by window size and code rate).

Key finding: For systematic codes, successfully receiving a source symbol can sometimes **reduce** decodability of earlier lost symbols through Gaussian elimination interactions. Our normal approximation misses this effect.

Extension: Replace the normal-approximation P_{fec} with debt-tracking:

Current model: $P_{\text{fec}} \approx 1 - \Phi(-z)$ where z depends on $(e, W, \sigma^2_{\text{burst}})$
 Extended model: $P_{\text{fec}} = 1 - p_e$ where p_e from debt Markov chain

The debt Markov chain has state space $\{0, 1, \dots, \zeta\}$ with transition probabilities depending on GE state and repair density. This is computable (finite-state Markov chain) though not as simple as the closed-form formula.

Status: The normal formula is used in the paper for analytical insight. For implementation, the exact $O(W^2)$ transfer matrix computation (Section 8.7, implemented as $p_{\text{fec_exact}}$) provides the same precision as the debt model. Both are finite-state Markov chain computations over the GE channel — the debt model tracks decoder state, the transfer matrix tracks the loss-minus-repair deficit. Either gives exact P_{fec} for implementation use.

C.3 Analytical P_{fec} Bounds [Vajha2020]

Vajha et al. derive upper and lower bounds on block-erasure probability for streaming codes over GE without simulation. Their bounds depend on: - GE parameters (p, q) - Code rate $R = K/N$ - Decoding delay constraint T

Extension to verification (Section 11):

For our taper with rate r^* and window W :
 Effective code rate $R = 1/(1+r^*)$
 Delay constraint $T = W$

Vajha lower bound $\leq P(\text{erasure}) \leq$ Vajha upper bound

If our predicted P_{fec} falls within these bounds: ✓ model validated
 If not: normal approximation is inadequate → use debt model (C.2)

This gives us an analytical verification path that complements simulation. Every verification method strengthens confidence in the model — simulation validates end-to-end behavior, Vajha bounds validate the P_{fec} formula analytically, and the exact $O(W^2)$ computation validates the normal approximation numerically.

C.4 Multipath Diversity Gain [Zeng2021]

CloudBurst sends FEC-coded packets across multiple paths and recovers from whichever path delivers first. This is a **diversity gain**: the same repair on two independent paths has survival probability $1 - \epsilon_1 \epsilon_2$ instead of $1 - \epsilon_1$.

Extension: For multipath, the effective repair arrival rate is higher than single-path. With M independent paths of loss rates $\epsilon_1 \dots \epsilon_m$:

$P(\text{repair arrives on at least one path}) = 1 - \prod(e_i)$

Single path: $P(\text{arrive}) = 1 - e$
 Dual path: $P(\text{arrive}) = 1 - e^2 \approx 1$ for small e

This means the multipath taper can use a **lower amplitude** A than single-path for the same P_{fec} target. The optimal multipath taper:

$$\tau_{\text{multi}}(t) = A_{\text{multi}} \times (1-q)^t$$

$$\text{where } A_{\text{multi}} = A_{\text{single}} \times (1-e) / (1-\Pi(e_i))$$

For two paths with 5% loss each: $A_{\text{multi}} = A_{\text{single}} \times 0.95/0.9975 \approx 0.95 \times A_{\text{single}}$. Modest gain for similar paths, but significant when paths have different characteristics (one lossy WiFi, one reliable Ethernet).

Not applicable to our model. CloudBurst duplicates the same repair symbol across paths. Our model uses shared-buffer cross-path retransmit instead (Section 13.10): each path generates its own corrections, and the shared retransmit buffer enables cross-path recovery. The diversity gain $P(\text{both fail}) = \epsilon_A \times \epsilon_B$ is already captured in Section 13.10 without repair duplication.

C.5 DCSW Worst-Case Taper Floor

The Delay-Constrained Sliding Window model [Badr2017], [Fong2019] provides a **hard guarantee**: within any window of W symbols, at most B consecutive erasures or N random erasures can occur, and recovery must happen within delay T .

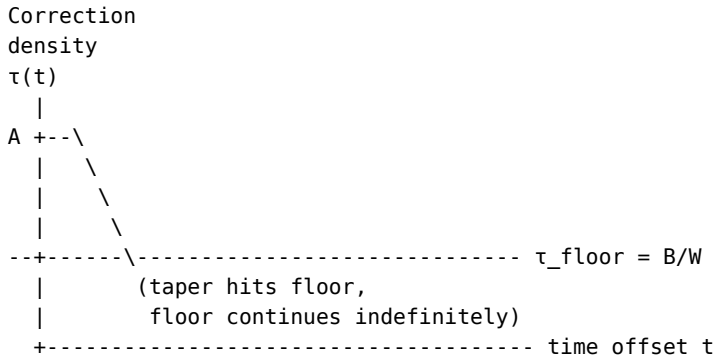
The streaming capacity for this model is $C(T,B) = T/(T+B)$ [Badr2017].

Extension: Regardless of the probabilistic taper, enforce a minimum correction density that survives the worst-case DCSW pattern:

$$\tau_{\text{floor}} = B / W \quad (\text{enough to survive one full burst per window})$$

$$\tau^*(t) = \max(A \times (1-q)^t, \tau_{\text{floor}}) \quad (\text{probabilistic taper with hard floor})$$

The floor ensures that even if the GE estimator underestimates burst length, we always have enough correction to survive at least B consecutive erasures.



Not applicable to our model. The DCSW floor is a hard guarantee from a worst-case adversarial channel model. Our model already handles burst protection through multiple mechanisms: (1) the taper's front-loaded density exceeds B/W for any reasonable r^* , (2) the retransmit buffer holds burst-lost symbols for ARQ recovery after detection, (3) cross-path diversity on multipath, and (4) backpressure guarantees delivery for $\rho=100\%$. Even if a burst overwhelms the taper entirely, ARQ kicks in — the retransmit buffer IS the hard guarantee. The DCSW floor is redundant.

Corrected total correction rate with floor (for reference):

$$r^* = \max(A/q, \tau_{\text{floor}} \times W) / W$$

In practice: A/q dominates when loss is high (large A).

Floor dominates when loss is low but bursts are long (large B , small e).

C.6 Burst-Aware Channel Discount (considered refinement)

The per-slot decision can optionally include a channel-state discount:

$$P_{\text{retx}} = P_{\text{lost}}(t) \times (1 - e_{\text{burst}})$$

where e_{burst} is a fast-reacting estimate of current channel loss (from GE state or fast EWMA). This suppresses retransmit during ongoing bursts, where repair is more flexible (covers any lost symbol).

Two separate loss estimates serve different purposes:

- ϵ (long-run, from BOCD): Bayesian prior for per-symbol loss confidence in P_{lost}
- ϵ_{burst} (fast, from GE state): current channel condition discount

This was considered for the primary model but deemed unnecessary because:

1. $P_{\text{lost}}(t)$ alone handles the primary FEC/ARQ regulation via timing
2. Before burst detection ($t < \text{RTT}$): P_{lost} is already low $\rightarrow$ mostly repair
3. After burst detection ($t > \text{RTT}$): burst is usually over $\rightarrow \epsilon_{\text{burst}}$ has dropped
4. The improvement is only significant for long bursts still ongoing at detection time ($\sim 1\text{ms}$ improvement)

The source/correction ratio was also considered for fast burst adjustment (two-speed taper: $A_{\text{effective}} = A_{\text{baseline}} \times (1 + \text{burst_boost})$), but the existing BOCD rate adaptation handles this with negligible delay.

Preserved here for future reference if extreme burst scenarios require it.

C.7 Summary of Extensions

Extension	From	Effect on correction rate	Effect on P_{fec} accuracy
Correction preemption	[Mehrotra2010]	Reduces delay during bursts	Improves burst recovery
Information debt	[RLC_GE2025]	More precise	Exact (Markov chain)
Analytical bounds	[Vajha2020]	Verification only	Bounds, not point estimate
Multipath diversity	[Zeng2021]	Reduces A_{multi}	Higher for same budget
DCSW taper floor	[Badr2017]	Hard minimum guarantee	Worst-case protection
Burst-aware discount	C.6	Modest improvement for long bursts	Suppresses retransmit during bursts

Appendix D: Open Questions

1. **Finite window truncation (resolved):** The taper is infinite-tailed but the encoder window has finite size W . After eviction, FEC repair is no longer possible — but the retransmit buffer (Section 5.3) still holds the exact source symbol for ARQ retransmission. The truncation error $(1-q)^W$ only affects the FEC component; the unified model's ARQ fallback covers it.

2. **Multi-path (resolved):** Each path has its own taper, Copa, and GE estimator (Section 13). Unified stream per path preserves interleaving. Scheduler adjusts per-path source/correction ratio (latency mode only). Global correction deficit tracks outstanding corrections. Cross-path retransmit via shared buffer. Burst protection during scheduler ratio adjustment is largely resolved: P_{lost} timing + BOCD adapts within $\sim 1\text{ms}$ (Section 13.7). Two-speed taper is an optional refinement for extreme scenarios (Appendix C.6).
3. **Interaction with congestion control (resolved):** Copa [Copa2018] controls total rate, taper controls source/correction split (Section 12). When $\text{source_rate} = \text{total_rate}/(1+r^*)$ drops below the app's minimum, the system signals back-pressure. Copa is preferred over BBR: no ProbeRTT, no FEC protection gaps, simpler formula, taper-compatible.
4. **Normal approximation validity (resolved).** The normal approximation uses the EXACT mean (We) and EXACT variance ($We(1-e)s2_burst$) from the GE model — only the distribution SHAPE is approximated as a bell curve. For $W \gg B$ (all practical scenarios: $W=50$, $B=2-3$), the CLT makes this accurate. For edge cases (W close to B), the exact GE tail probability is computable via transfer matrix dynamic programming in $O(W^2)$ — trivially cheap (2500 operations for $W=50$). The normal formula provides analytical insight (clean, closed-form); the exact computation is recommended for implementation and validation. IMPLEMENTED: Section 8.7 specifies the recursion; `p_fec_exact / compute_r_star_exact` in raptorpath-math implement it, verified against Monte Carlo and an independent-Binomial reference.
5. **Optimal retransmit timing (resolved):** Replaced by the $P_{\text{lost}}(t)$ model (Section 3.4). No hard timeout — the repair/retransmit mix is determined probabilistically by $P_{\text{lost}}(t) = \epsilon / [\epsilon + (1-\epsilon) \times P(\text{RTT} > t)]$. This smoothly transitions from repair to retransmit as confidence in loss grows, naturally handles proactive retransmit at high loss rates, and minimizes expected waste at $E[\text{waste}] = P_{\text{lost}} \times (1 - P_{\text{lost}})$.

Remaining Open Points (full audit)

The following were identified by systematic review of the paper. Rated by difficulty: LOW = can be resolved with a sentence or formula, MEDIUM = needs a paragraph or derivation, HIGH = needs new analysis or algorithm design.

Core model gaps (critical):

6. **RTT distribution for P_{lost} not specified (resolved).** (Section 3.4) [LOW] P_{lost} uses $P(\text{RTT} > t)$ but never specifies the distribution. Assuming normal with SRTT and RTTVAR gives $P(\text{RTT} > t) = 1 - \Phi((t - \text{SRTT})/\text{RTTVAR})$. Needs one sentence.
7. **P_{arq} never defined (resolved).** (Section 6.3, 8.6, Appendix A) [MEDIUM] Derived from reliability target: $P_{\text{arq}} = 1 - (1 - \rho)/(e \times (1 - P_{\text{fec}}))$. See Section 6.3.
8. **P_{fec} : two contradictory models (resolved).** (Appendix E vs Section 8.2) [MEDIUM] Appendix E sections marked as preliminary (Poisson, superseded). Section 8.2 marked as canonical (Binomial/Normal). Progression preserved to show why per-symbol model fails.
9. **Poisson model error not characterized (resolved).** (Appendix E.4) [LOW] The transition from Poisson (Appendix E.3) to corrected (Section 8.2) says “too pessimistic” but doesn't explain why. The error: per-symbol accounting counts only the lost symbol's

own taper ($\sim r$ corrections) instead of all window-covering repairs ($\sim rW$) — a factor-of- W undercount. See E.4.

10. **r^* formula: which version is canonical (resolved)?** (Section 8.4 vs 9.2) [LOW] Section 8.4 uses raw ϵ . Section 9.2 adds codec overhead ϵ_{codec} . The canonical formula should be r^* with $\epsilon_{\text{hat}} = \epsilon + \epsilon_{\text{codec}} \times P(\text{decoder})$. Needs a clarifying note in Section 8.4.
11. **T_{cut} computation algorithm (resolved).** (Section 8.6) [MEDIUM] Binary search on T_{cut} . $P(\text{recovered})$ is monotone in T_{cut} . Algorithm added to Mode 1. Converges in ~ 20 iterations.
12. **Mode 3 convergence (resolved).** (Section 8.6) [MEDIUM] Binary search on T_{cut} with monotonicity in ρ and r . Algorithm added to Mode 3. Convergence guaranteed.
13. **t_{recovery_i} undefined in multipath context (resolved).** (Section 13.5) [LOW] $E_i = RTT_i/2 + \epsilon_i \times t_{\text{recovery}_i}$ but t_{recovery_i} not defined for multipath. It should reference Section 3.4: $t_{\text{recovery}_i} = P_{\text{fec}_i} \times t_{\text{fec}_i} + (1 - P_{\text{fec}_i}) \times L_{\text{arq}_i}$. One sentence.

Implementation details (medium priority):

14. **Retransmit buffer saturation (resolved).** (Section 6.1) [LOW] Resolved: dual-mechanism model — T_{cut} age eviction + buffer_max size backpressure. Buffer is NOT bounded by encoder window. See Section 6.1.
15. **SACK timing and format (resolved).** (Section 6.2) [MEDIUM] Per-packet ACK (0.6% overhead). 5 fields: `cumulative_ack`, `sack_ranges` (cumulative within T_{cut}), `echo_timestamp`, `jitter_us` (u32), `cumulative_received`. Gap pruning at T_{cut} . ACK loss self-healing via cumulative semantics.
16. **GE parameter initialization (resolved).** (Section 7.5) [LOW] Initial state: all counters = 0, start in Good state, use the Beta prior (weak uniform) until enough transitions observed. `GE is_valid()` already gates usage (existing code). One sentence.
17. **Copa parameter d tuning (resolved).** (Section 12.4) [MEDIUM] Renamed to `d_copa` to avoid confusion with tail latency delta. Default 0.5 sufficient. Adaptive `d_copa` (Copa+) is a negligible optimization ($\sim 1\text{ms}$ gain).
18. **Copa min_rtt refresh (resolved).** (Section 12.4) [LOW] Copa's natural oscillation refreshes `min_rtt`. Sliding window of 10s (same as BBR). If `min_rtt` seems stale (RTT consistently above by 2x), force a brief rate reduction. One paragraph.
19. **Back-pressure signaling (resolved).** (Section 12.7) [MEDIUM] Blocking `write()` like TCP — sender stops reading from source when `buffer_max` reached. Optional stats API for advanced applications. Implementation-specific, not part of core model.
20. **Scheduler burst protection floor (resolved).** (Section 13.7) [LOW] No floor needed. Global correction deficit + cross-path diversity handle burst protection when the scheduler reduces a path's correction ratio. The three options and floor discussion were removed.
21. **Cross-path retransmit path selection (resolved).** (Section 13.10) [LOW] No explicit path selection needed. Cross-path retransmit emerges from shared buffer + per-path `P_lost`. Paths with spare capacity naturally pull retransmits (work-stealing). Weighted path preference for latency-sensitive traffic noted as potential refinement.

Minor (notation, justification):

22. **z_d vs z_δ inconsistency (resolved).** (Section 8.4) [LOW] Line uses “z_d” and “d” where it should be “z_δ” and “δ”.
 23. **Beta decay 0.995 not justified (resolved).** (Section 7.3) [LOW] Standard value for slow-forgetting Bayesian update. Half-life ≈ 138 samples. Sensitivity is low — values 0.99-0.999 give similar results.
 24. **BOCD “5-15 batches” — what’s a batch (resolved)?** (Section 4.5) [LOW] A batch = one ACK feedback cycle. With per-batch ACKs at ~10-100ms intervals, 5-15 batches = 50ms-1.5s adaptation time.
 25. **GE simplified model error bounds (resolved).** (Section 2.1) [MEDIUM] h_G=0, h_B=1 is exact for packet-level erasure (UDP checksums ensure no partial delivery). Not an approximation for our use case.
 26. **P_arq missing from Appendix A (resolved).** [LOW] Added P_arq formula to Appendix A.
-

Appendix E: Preliminary Poisson Model (superseded)

These sections were originally Section 6.2-6.5. They present a Poisson model that gives unrealistic results. The corrected Binomial/Normal model is in Section 8.2. This appendix is preserved for pedagogical value — it shows why the simpler per-symbol approach fails.

E.1 FEC Recovery Probability (preliminary — superseded by Section 8.2)

Note: Sections E.1-E.4 develop a preliminary Poisson model that gives unrealistic results. The model is superseded by the corrected Binomial model in Section 8.2. The preliminary derivation is kept to show why the simpler per-symbol approach fails and motivate the per-window model.

Consider a symbol lost at position 0. Correction symbols generated at offsets $t = 0, 1, 2, \dots$ each have:

- Probability $\tau(t)$ of being generated (fractional: may or may not generate one)
- Probability $(1-\epsilon)$ of surviving the channel

The expected number of correction symbols covering the lost position that arrive:

$$\begin{aligned} R(A, q) &= \sum_{t=0}^{W-1} \tau(t) \times (1-e) \\ &= A \times (1-e) \times \sum_{t=0}^{W-1} (1-q)^t \\ &= A \times (1-e) \times (1 - (1-q)^W) / q \end{aligned}$$

For large W (window much larger than burst length): $(1-q)^W \approx 0$, so:

$$R(A, q) \approx A \times (1-e) / q = r \times (1-e)$$

Recovery model: The number of useful correction symbols arriving is approximately Poisson(R). For FEC recovery, we need at least 1 repair symbol (plus codec overhead, see Section 9). Simplified to needing at least 1:

$$P_{\text{fec}}(A, q) = 1 - P(\text{Poisson}(R) = 0) = 1 - e^{-R}$$

E.2 Solving for A^* (preliminary)

Substituting into the constraint:

$$\begin{aligned}
e \times (1 - P_{\text{fec}}) &\leq \delta \\
e \times e^{-R} &\leq \delta \\
e^{-R} &\leq \delta/e \\
-R &\leq \ln(\delta/e) \\
R &\geq \ln(e/\delta) \quad (\text{note: } e > \delta, \text{ so } \ln(e/\delta) > 0)
\end{aligned}$$

Using $R \approx A \times (1-\epsilon) / q$:

$$\begin{aligned}
A \times (1-e) / q &\geq \ln(e/\delta) \\
A^* &= q \times \ln(e/\delta) / (1-e) \\
r^* &= A^*/q = \ln(e/\delta) / (1-e)
\end{aligned}$$

E.3 The Optimal Correction Rate Formula (preliminary — see Section 8.4)

$$\begin{aligned}
r^* &= \ln(e/\delta) / (1-e) \\
\text{where:} \\
e &= \text{average loss rate (from BOCD)} \\
\delta &= \text{tail latency target} \\
r^* &= \text{optimal correction rate}
\end{aligned}$$

Properties: - r^* depends only on ϵ and δ , not on q (burst length doesn't affect the TOTAL correction budget, only its distribution over time via the taper shape) - As $\delta \rightarrow 0$ (tighter tail): $r^* \rightarrow \infty$ (need infinite FEC for zero ARQ events) - As $\delta \rightarrow \epsilon$ (loose tail = every loss goes to ARQ): $r^* \rightarrow 0$ (no FEC needed) - As $\epsilon \rightarrow 0$ (perfect channel): $r^* \rightarrow 0$ (no correction needed)

The **taper amplitude** is:

$$A^* = r^* \times q = q \times \ln(e/\delta) / (1-e)$$

And the **complete taper function** is:

$$\tau^*(t) = A^* \times (1-q)^t = q \times \ln(e/\delta) / (1-e) \times (1-q)^t$$

E.4 Comparison with Information-Theoretic Minimum

The IT minimum (Shannon limit for the erasure channel [Shannon1948]) is:

$$r_{\text{IT}} = e / (1-e)$$

Why $\epsilon/(1-\epsilon)$, not just ϵ ? Correction symbols are also subject to channel loss. A correction sent to replace a lost source might itself be lost, needing another correction, which might also be lost:

$$e + e^2 + e^3 + \dots = e / (1-e)$$

At 30% loss: need 43% overhead (not 30%), because 30% of corrections are lost too. The $(1-\epsilon)$ denominator IS this geometric series. See also Section 13.4.

Our optimal rate:

$$r^* = \ln(e/\delta) / (1-e) = r_{\text{IT}} \times \ln(e/\delta) / e = r_{\text{IT}} \times \ln(1/\delta)/e + r_{\text{IT}} \times \ln(e)/e$$

The ratio $r^*/r_{\text{IT}} = \ln(\epsilon/\delta)/\epsilon$. This is the **unavoidable overhead** of targeting a tail latency of δ — it's the price of proactive protection.

For $\varepsilon = 0.025$ (WiFi), $\delta = 1e-4$ (Realtime):

$$r^*/r_{IT} = \ln(0.025/0.0001) / 0.025 = \ln(250) / 0.025 = 5.52/0.025 = 221$$

This seems very high. Let's check: $r_{IT} = 0.025/0.975 = 0.0256$, so $r^* = 0.0256 \times 221 = 5.66$. That's 566% overhead — clearly too much.

The issue: The dominant error is not Poisson-vs-Binomial — it is per-symbol accounting. E.1 counts only the corrections attributed to the lost symbol by its own taper, $\sum_t \tau(t) = r$ per source symbol, so the expected help is $R \approx r(1-e) < 1$ and the model concludes that even a single loss usually cannot be recovered. But every repair is a combination of the ENTIRE window (Section 3.2): while the lost symbol remains in the window, roughly rW repairs are generated that all cover it. The corrected model in Section 8.2 counts repairs per window — Binomial($rW, 1-e$) — and compares them against the per-window loss count K . (The secondary Poisson-vs-Binomial distinction matters far less than this factor-of- W undercount.)

References

Channel Models

- **[Gilbert1960]** E.N. Gilbert, “Capacity of a burst-noise channel,” *Bell System Technical Journal*, vol. 39, pp. 1253-1265, 1960. The original two-state channel model.
- **[Elliott1963]** E.O. Elliott, “Estimates of error rates for codes on burst-noise channels,” *Bell System Technical Journal*, vol. 42, pp. 1977-1997, 1963. Extension of Gilbert’s model with per-state loss probabilities.

Estimation and Detection

- **[Adams2007]** R.P. Adams, D.J.C. MacKay, “Bayesian Online Changepoint Detection,” arXiv:0710.3742, 2007. The BOCD algorithm used in Section 7.4 for regime-aware loss estimation. Maintains run-length distribution with $O(r_{\max})$ per update.
- **[RFC3550]** H. Schulzrinne, S. Casner, R. Frederick, V. Jacobson, “RTP: A Transport Protocol for Real-Time Applications,” IETF RFC 3550, 2003. Appendix A.8 defines the interarrival jitter calculation used in our estimator.
- **[RFC6298]** V. Paxson, M. Allman, J. Chu, M. Sargent, “Computing TCP’s Retransmission Timer,” IETF RFC 6298, 2011. Standard for TCP RTO computation using SRTT and RTTVAR.
- **[Abramowitz1964]** M. Abramowitz, I.A. Stegun, *Handbook of Mathematical Functions*, National Bureau of Standards, 1964. Rational approximation for the standard normal quantile (Section 26.2.23), used in our Beta quantile and z_δ computation.

FEC Codes

- **[RFC6330]** M. Luby, A. Shokrollahi, M. Watson, T. Stockhammer, L. Minder, “RaptorQ Forward Error Correction Scheme,” IETF RFC 6330, 2012. Fountain code with ~1% decode overhead. Our RaptorQ backend.
- **[RFC8681]** V. Roca, B. Teibi, “Sliding Window Random Linear Code (RLC) Forward Erasure Correction (FEC) Schemes,” IETF RFC 8681, 2020. Window-mode FEC over $GF(2^8)$. Our RLC backend.

- [Yu2025] S. Yu, J. Yang, Q. Meng, L. Xu, “METTLE: Streaming Codes Based on SC-MET-LDGM,” arXiv:2602.10020, 2025. Sparse random matrix streaming code. ~15% decode overhead at small windows.

Streaming Codes Theory

- [Martinian2004] E. Martinian, C.-E.W. Sundberg, “Burst erasure correction codes with low decoding delay,” *IEEE Trans. Information Theory*, 2004. Foundational work on delay-constrained erasure correction.
- [Badr2017] A. Badr, P. Patil, A. Tan, A. Dey, “Layered Constructions for Low-Delay Streaming Codes,” *IEEE Trans. Information Theory*, 2017, arXiv:1308.3827. Proves streaming capacity $C(T,B) = T/(T+B)$. Layered burst+random construction.
- [Fong2019] S.L. Fong, A. Khisti, B. Li, A. Tan, “Optimal Streaming Codes for Channels with Burst and Arbitrary Erasures,” *IEEE Trans. Information Theory*, vol. 65 no. 7, 2019, arXiv:1801.04241. Generalizes to adversarial model: $C(T,B,N) = (T-N+1)/(T-N+1+B)$.
- [Krishnan2020] M.N. Krishnan, D. Ramkumar, “Simple Streaming Codes for Reliable, Low-Latency Communication,” *IEEE Comm. Letters*, vol. 24 no. 2, 2020. Staggered diagonal embedding (SDE): simplest rate-optimal construction.
- [Rudow2023] M. Rudow et al., “Tambur: Efficient loss recovery for videoconferencing via streaming codes,” NSDI, 2023. Production implementation with ML-based rate adaptation. github.com/Thesys-lab/tambur

Multipath and Scheduling

- [Ferlin2016] S. Ferlin et al., “BLEST: Blocking Estimation-based MPTCP Scheduler,” IFIP Networking, 2016. Skip slow paths that would stall receiver.
- [Facenda2022] T. Facenda, M.N. Krishnan et al., “Error-correcting codes for low latency streaming over multiple link relay networks,” arXiv:2201.06609, 2022. Introduces delay spectrum concept for per-link rate allocation.

Hybrid FEC-ARQ

- [Mehrotra2010] S. Mehrotra, J. Li, Y. Huang, “Optimizing FEC Transmission Strategy for Minimizing Delay in Lossless Sequential Streaming,” *IEEE Trans. Multimedia*, 2010. MSR-TR-2010-134. Closest prior work: MDP-based optimization of when to send FEC vs data for lossless streaming. Key insight: preempting data with FEC can reduce latency.
- [Mehrotra2009] S. Mehrotra, J. Li, “A hybrid FEC-ARQ protocol for low-delay lossless sequential data streaming,” *IEEE MMSP*, 2009. Precursor establishing the hybrid FEC-ARQ protocol framework.
- [Razavi2008] R. Razavi et al., “Performance Evaluation of Joint FEC and ARQ Optimization Heuristic Algorithms under Gilbert-Elliott Wireless Channel,” *IEEE CCNC*, 2008. Adaptive joint FEC/ARQ parameter search under GE channels for video.

Streaming Code Analysis over GE

- [Vajha2020] M. Vajha, V. Ramkumar, M. Jhamtani, P.V. Kumar, “On the Performance Analysis of Streaming Codes over the Gilbert-Elliott Channel,” arXiv:2005.06921, 2020. ITW

2021. First analytical bounds on block-erasure probability for streaming codes over GE channels, replacing simulation-only evaluation.

- [RLC_GE2025] “On the Analysis of Random Linear Streaming Codes in Stochastic Channels,” arXiv:2509.01894, 2025. Analytical performance of RLC over GE channels — directly applicable to our RLC backend.

Tail Latency

- [Zeng2021] G. Zeng, L. Chen, B. Yi, K. Chen, “Optimizing Tail Latency in Commodity Datacenters using Forward Error Correction,” arXiv:2110.15157, 2021. (CloudBurst) Proactive FEC over multipath reduces p99 latency by 60-75% in datacenters.

Congestion Control

- [Copa2018] V. Arun, H. Balakrishnan, “Copa: Practical Delay-Based Congestion Control for the Internet,” NSDI 2018. Delay-based CC with natural queue draining (no ProbeRTT). Rate formula: $\text{rate} = 1/(\delta \times dq)$. Recommended for raptorpath (Section 12).

Information Theory

- [Shannon1948] C.E. Shannon, “A mathematical theory of communication,” *Bell System Technical Journal*, vol. 27, pp. 379-423, 623-656, 1948. The erasure channel capacity result $C = 1 - \epsilon$ gives the IT minimum repair rate $r_{IT} = \epsilon/(1 - \epsilon)$ used throughout this paper.
- [Gallager1968] R.G. Gallager, *Information Theory and Reliable Communication*, John Wiley & Sons, 1968. Water-filling theorem for optimal resource allocation across channels.

TCP SACK

- [RFC2018] M. Mathis, J. Mahdavi, S. Floyd, A. Romanow, “TCP Selective Acknowledgment Options,” IETF RFC 2018, 1996. Defines SACK for TCP: receiver reports non-contiguous blocks of received data, allowing sender to infer exactly which segments were lost. Our ACK+SACK feedback mechanism adapts this proven approach.

ECN and Multicast Feedback

- [RFC3168] K. Ramakrishnan, S. Floyd, D. Black, “The Addition of Explicit Congestion Notification (ECN) to IP,” IETF RFC 3168, 2001. ECN mechanism for router-signaled congestion without packet drops.
- [RFC4585] J. Ott, S. Wenger, N. Sato, C. Burmeister, J. Rey, “Extended RTP Profile for RTCP-Based Feedback,” IETF RFC 4585, 2006. RTCP NACK-based feedback for RTP multicast — the multicast feedback mechanism where ACK implosion is relevant.

Sliding Window Channel Models

- [Vajha2020b] M. Vajha, V. Ramkumar, P.V. Kumar, “On Sliding Window Approximation of Gilbert-Elliott Channel for Delay Constrained Setting,” arXiv:2005.06914, 2020. Formalizes the DCSW-to-GE approximation used in streaming code design.